

5.1人工智能与机器学习概述2025.pdf

5.2人工智能与机器学习—机器学习基础2025.pdf

5.3人工智能与机器学习—线性模型2025.pdf

5.4人工智能与机器学习—前馈神经网络2025.pdf

5.5人工智能与机器学习—卷积神经网络2025.pdf

6 基于深度学习智能感知应用与实践2025.pdf



智能感知

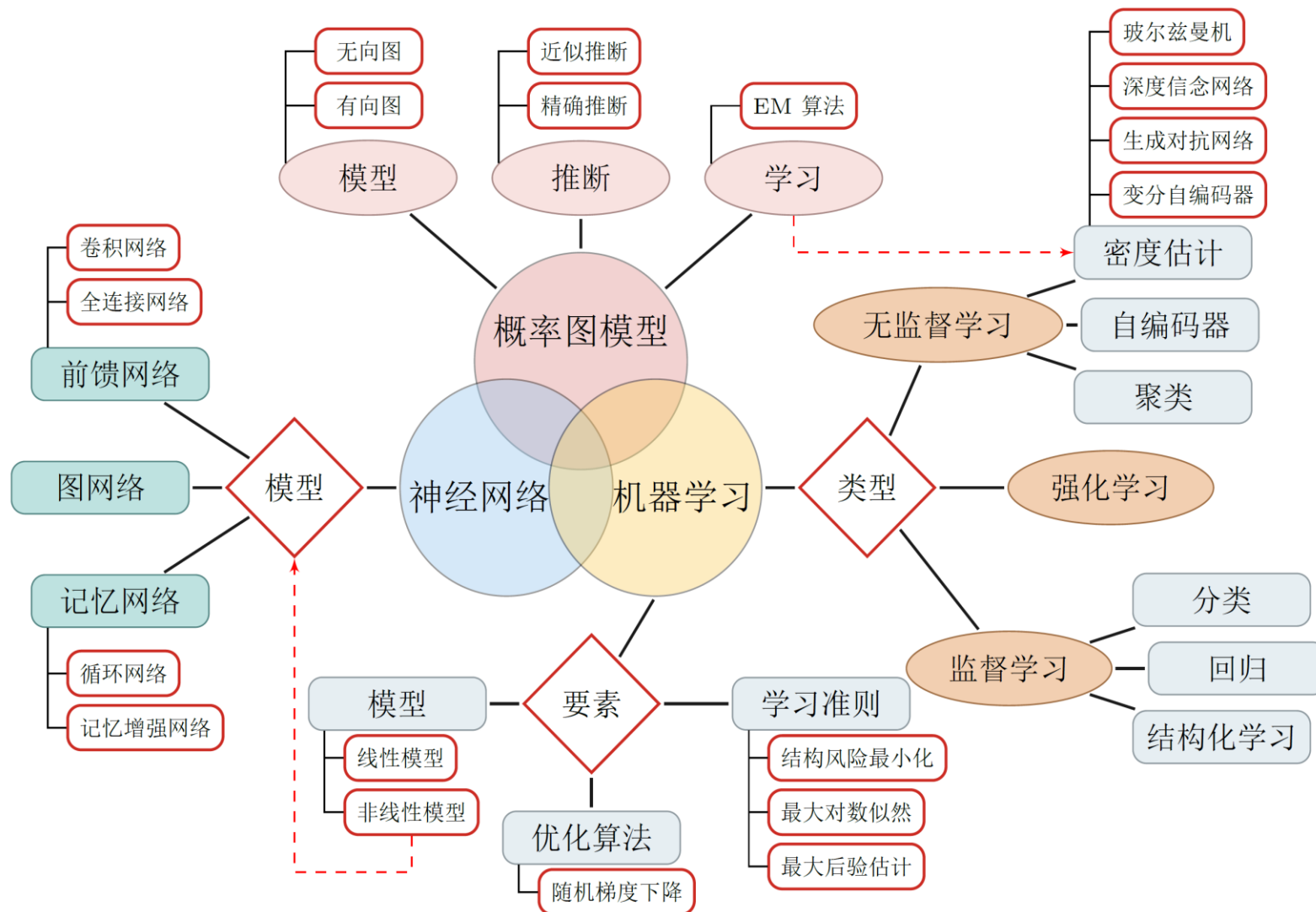
日期：2025年8月



目录

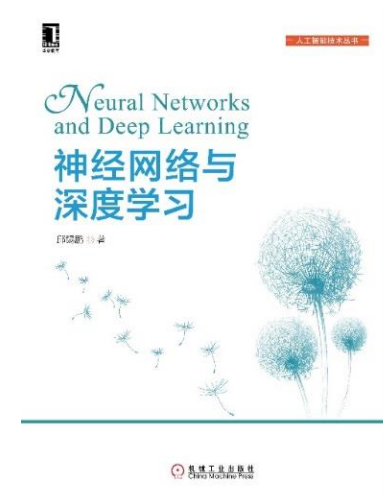
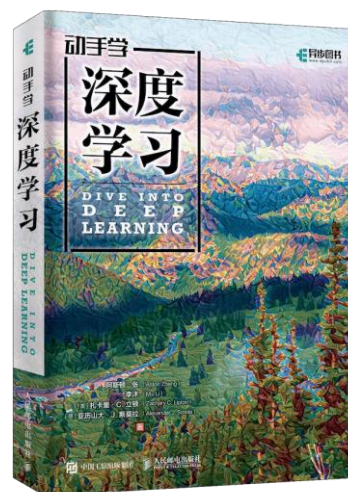
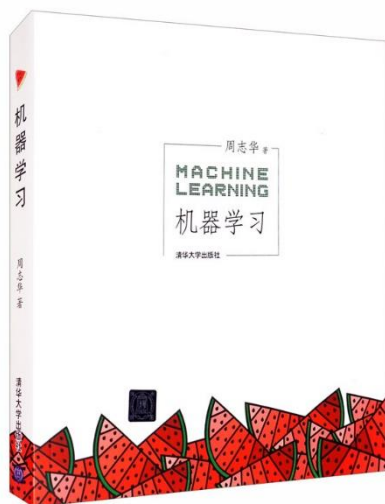
CONTENTS

- 1 绪论
- 2 机器学习基础
- 3 线性模型
- 4 前馈神经网络
- 5 卷积神经网络



- ▶ 线性代数
- ▶ 微积分
- ▶ 数学优化
- ▶ 概率论
- ▶ 信息论

- ▶ <https://www.bilibili.com/> 周志华老师 [bilibili](#)站亲讲-西瓜书
- ▶ <https://www.bilibili.com/> 李宏毅“机器学习2020”



- ▶ 人工智能（artificial intelligence, AI）就是让机器具有人类的智能。
- ▶ “计算机控制” + “智能行为”
- ▶ 人工智能这个学科的诞生有着明确的标志性事件，就是1956年的达特茅斯（Dartmouth）会议。在这次会议上，“人工智能”被提出并作为本研究领域的名称。

人工智能就是要让机器的行为看起来就像是人所表现出的智能行为一样。

John McCarthy（1927-2011）

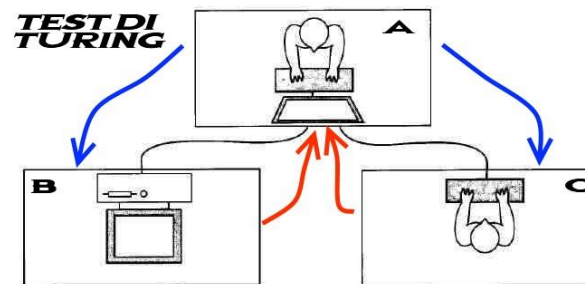
“一个人在不接触对方的情况下，通过一种特殊的方式，和对方进行一系列的问答。如果在相当长时间内，他无法根据这些问题判断对方是人还是计算机，那么就可以认为这个计算机是智能的”

。



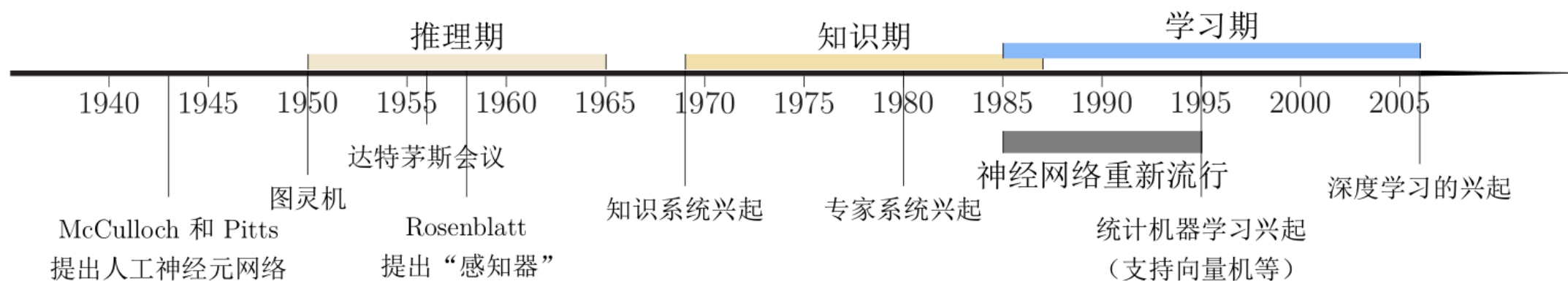
Alan Turing

---Alan Turing [1950]
《Computing Machinery and Intelligence》



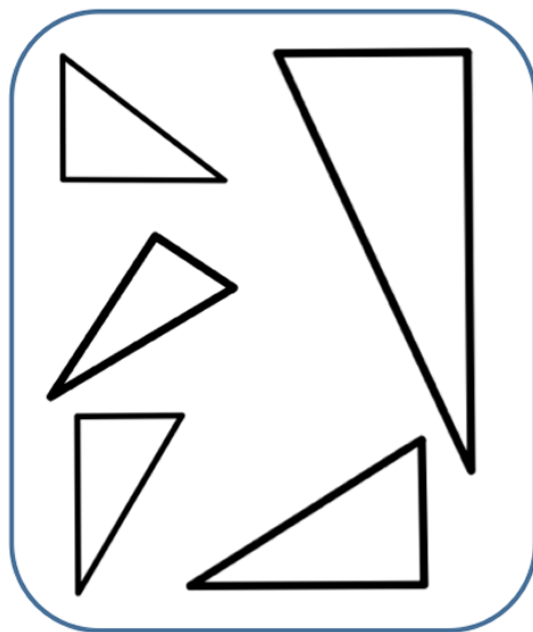
▶ 让机器具有人类的智能

- ▶ 机器感知（计算机视觉、语音信息处理）
- ▶ 学习（模式识别、机器学习、强化学习）
- ▶ 语言（自然语言处理）
- ▶ 记忆（知识表示）
- ▶ 决策（规划、数据挖掘）

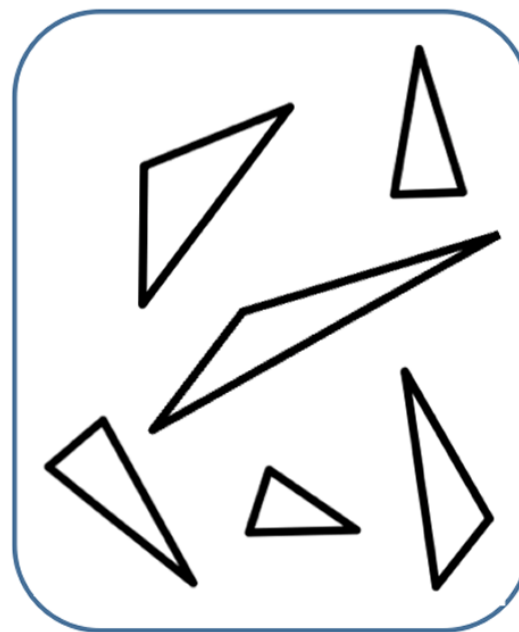


► 专家知识（人工规则）

What's My Rule?



Yes/True



No/False

What's the Rule?



2	6	8	9	3	4	7	5	6
3	4	7	9	5	5	6	7	2
5	8	7	0	9	4	3	5	4
5	2	3	4	9	5	6	7	8

机器学习

► 语音识别

$$f(\text{语音波形}) = \text{“你好”}$$

► 图像识别

$$f(\text{数字9}) = \text{“9”}$$

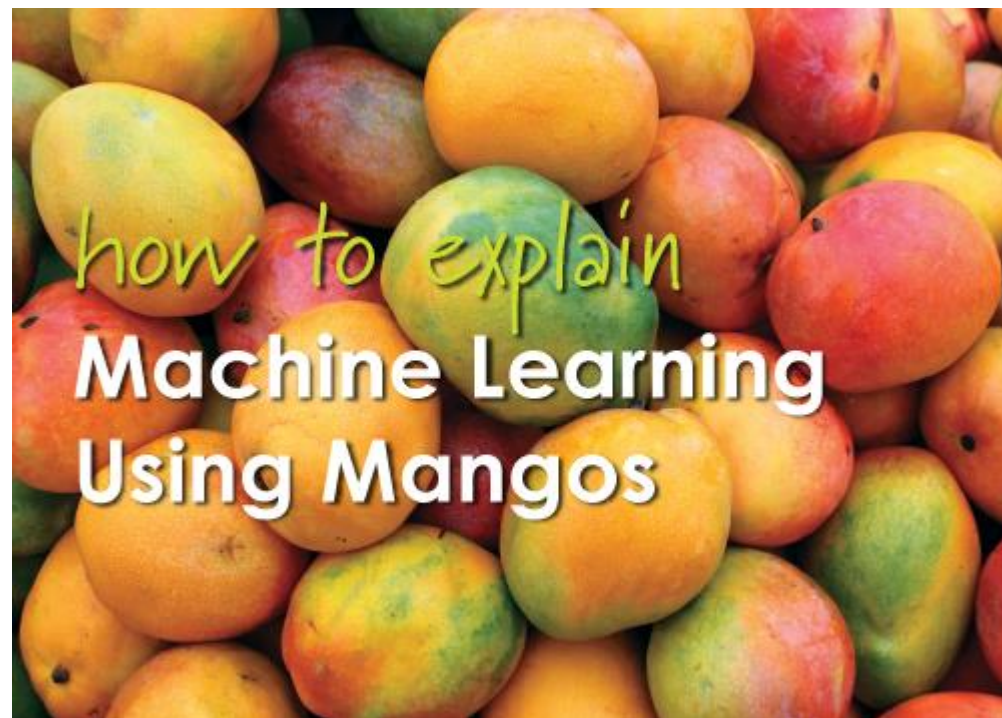
► 围棋

$$f(\text{围棋棋盘}) = \text{“6-5”} \quad (\text{落子位置})$$

► 机器翻译

$$f(\text{“你好!”}) = \text{“Hello!”}$$

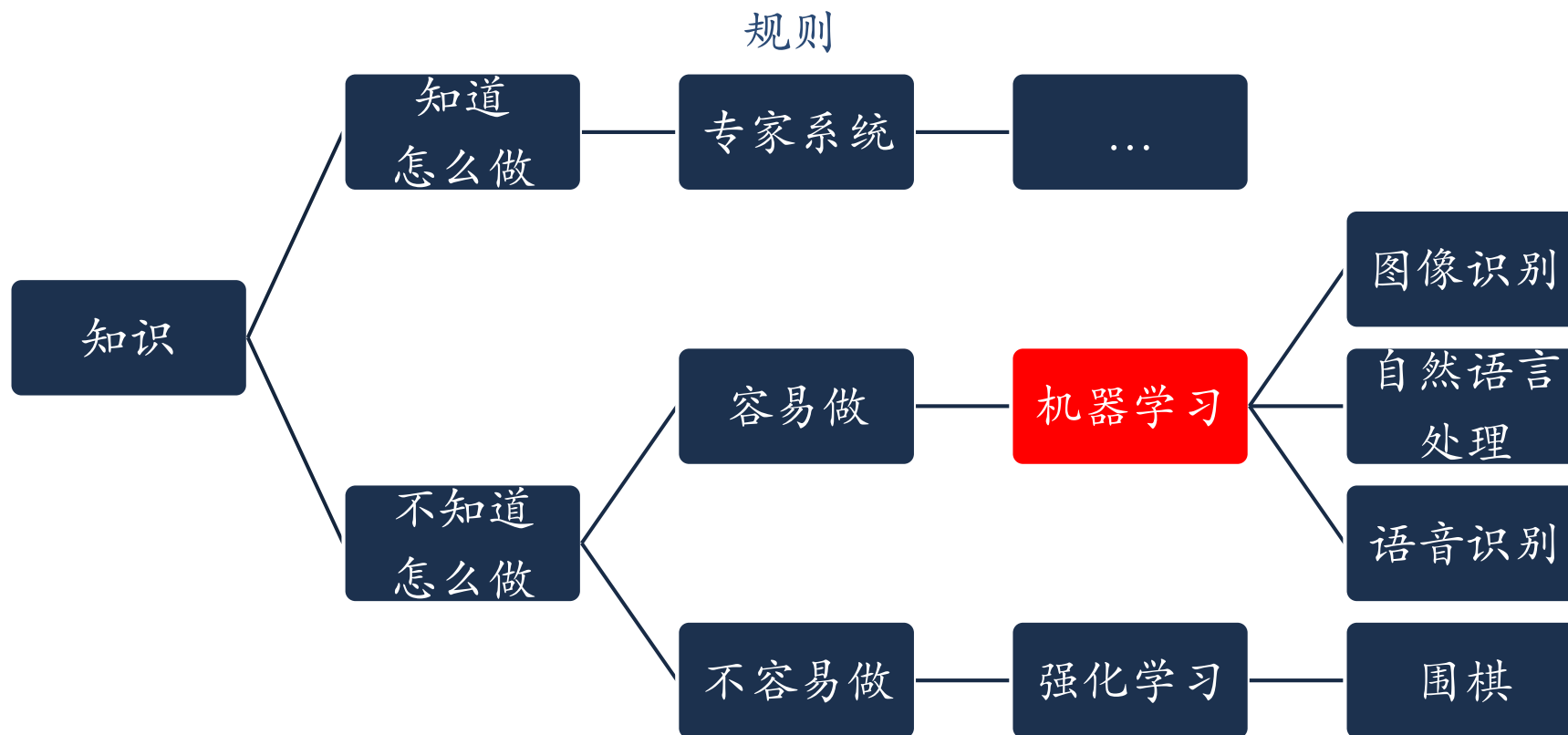
如果判断芒果是否甜蜜？



<https://www.quora.com/How-do-you-explain-Machine-Learning-and-Data-Mining-to-non-Computer-Science-people>

- ▶ 从市场上随机选取的芒果样本（训练数据），列出每个芒果的所有特征：
 - ▶ 如颜色，大小，形状，产地，品牌
 - ▶ 以及芒果质量（输出变量）：
 - ▶ 甜蜜，多汁，成熟度。
- ▶ 设计一个学习算法来学习芒果的特征与输出变量之间的相关性模型。
- ▶ 下次从市场上买芒果时，可以根据芒果（测试数据）的特征，使用前面计算的模型来预测芒果的质量。

如何开发一个人工智能系统?

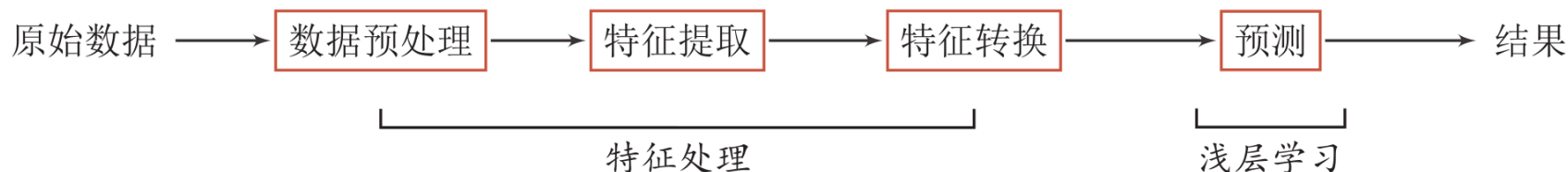




北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY

深度学习

- ▶ 当我们用机器学习来解决一些模式识别任务时，一般的流程包含以下几个步骤：



特征工程 (Feature Engineering)

- ▶ 浅层学习 (Shallow Learning)：不涉及特征学习，其特征主要靠人工经验或特征转换方法来抽取。

► 底层特征 VS 高层语义

- 人们对文本、图像的理解无法从字符串或者图像的底层特征直接获得



床前明月光，
疑是地上霜。
举头望明月，
低头思故乡。

- ▶ 数据表示是机器学习的核心问题。

- ▶ 特征工程：需要借助人类智能

- ▶ 表示学习

- ▶ 如何自动从数据中学习好的表示

- ▶ 难点

- ▶ 没有明确的目标

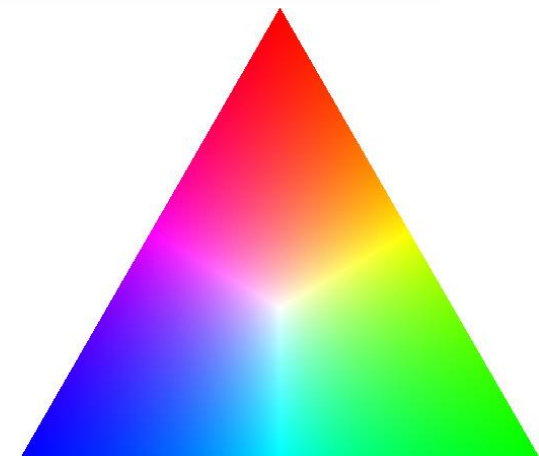
Bengio, Yoshua, Aaron Courville, and Pascal Vincent. "Representation learning: A review and new perspectives." IEEE transactions on pattern analysis and machine intelligence 35.8 (2013): 1798-1828.

- ▶ “好的表示”是一个非常主观的概念，没有一个明确的标准。
- ▶ 但一般而言，一个好的表示具有以下几个优点：
 - ▶ 应该具有很强的表示能力。
 - ▶ 应该使后续的学习任务变得简单。
 - ▶ 应该具有一般性，是任务或领域独立的。

▶ 局部表示

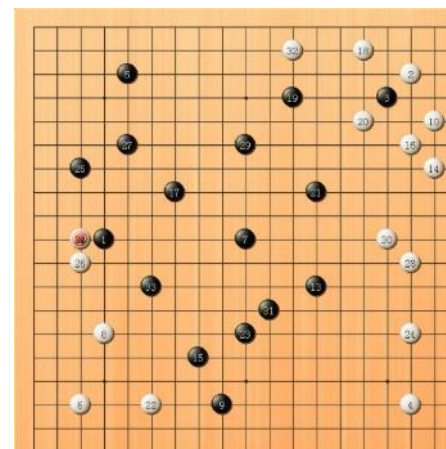
- ▶ 离散表示、符号表示
- ▶ One-Hot向量

颜色	局部表示	分布式表示
琥珀色	$[1, 0, 0, 0]^T$	$[1.00, 0.75, 0.00]^T$
天蓝色	$[0, 1, 0, 0]^T$	$[0.00, 0.5, 1.00]^T$
中国红	$[0, 0, 1, 0]^T$	$[0.67, 0.22, 0.12]^T$
咖啡色	$[0, 0, 0, 1]^T$	$[0.44, 0.31, 0.22]^T$



▶ 分布式(distributed)表示

- ▶ 压缩、低维、稠密向量
- ▶ 用 $O(N)$ 个参数表示 $O(2k)$ 区间
 - ▶ k 为非0参数, $k < N$

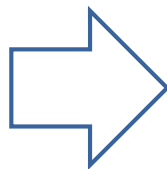


分布式表示

► 如何在计算机中表示语义？

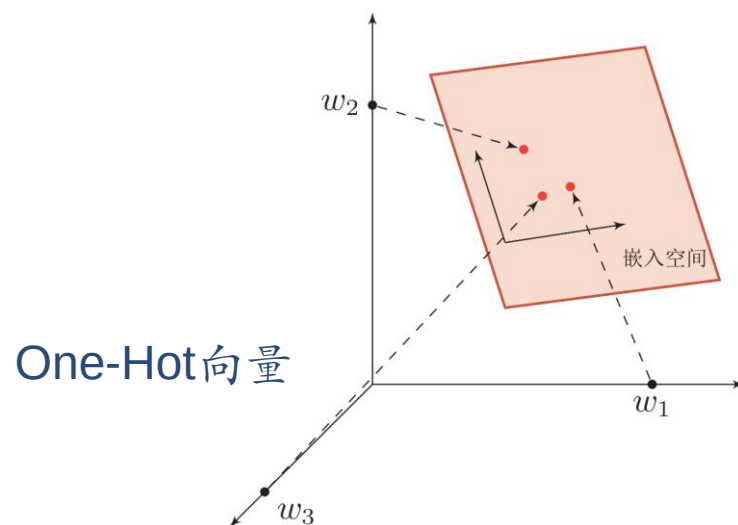
局部（符号）表示

知识库、规则

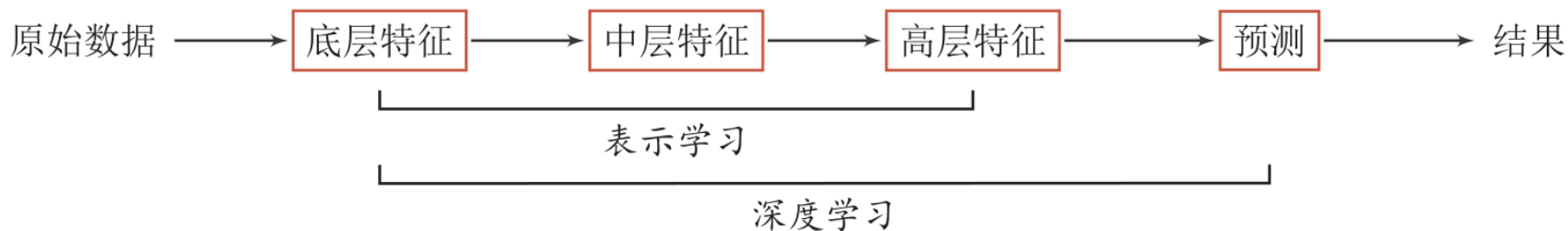


分布式表示

嵌入：压缩、低维、稠密向量



- ▶ 通过构建具有一定“深度”的模型，可以让模型来自动学习好的特征表示（从底层特征，到中层特征，再到高层特征），从而最终提升预测或识别的准确性。



▶ 特征提取

▶ 线性投影（子空间）

▶ PCA、LDA

▶ 非线性嵌入

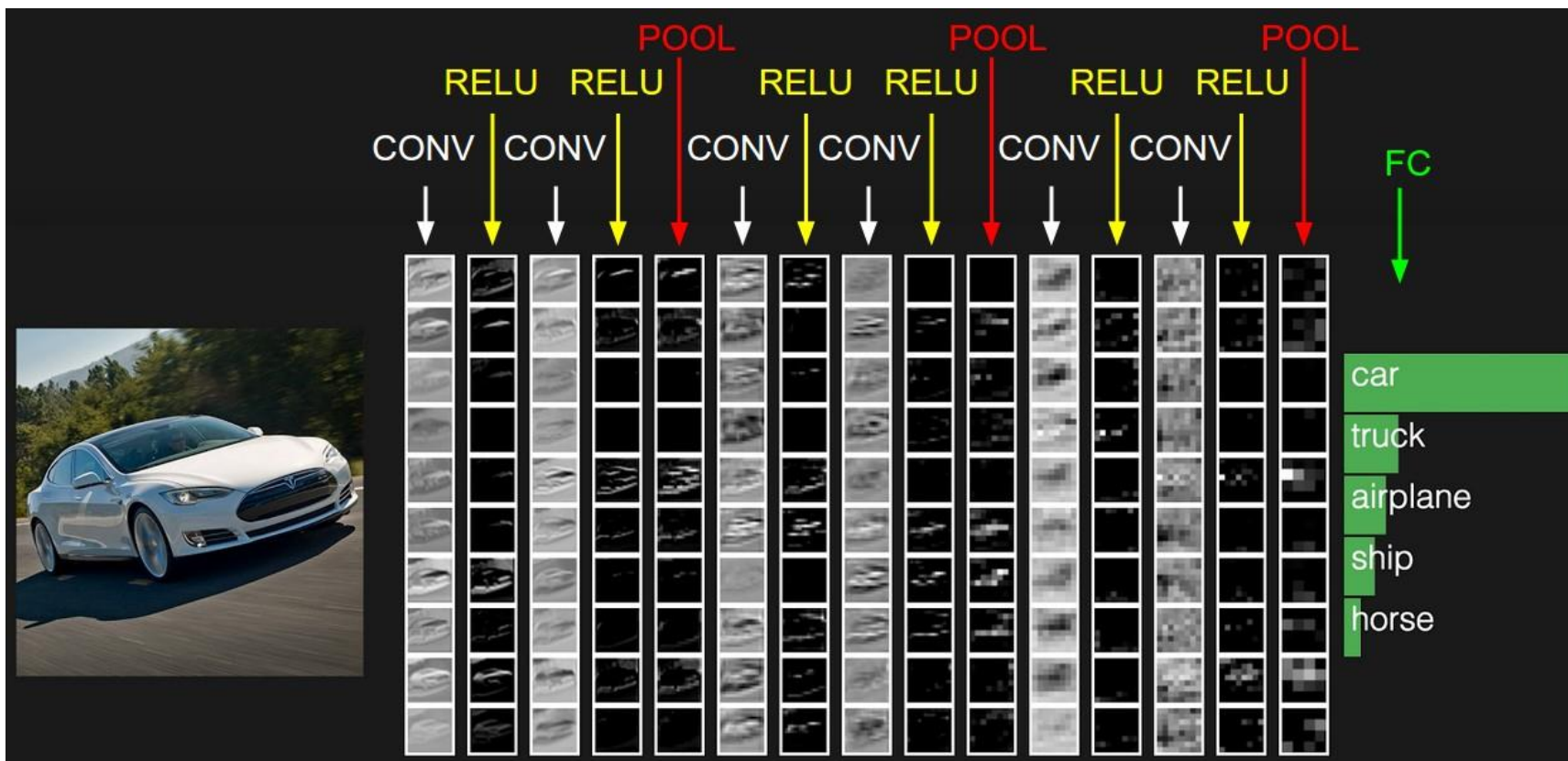
▶ LLE、Isomap、谱方法

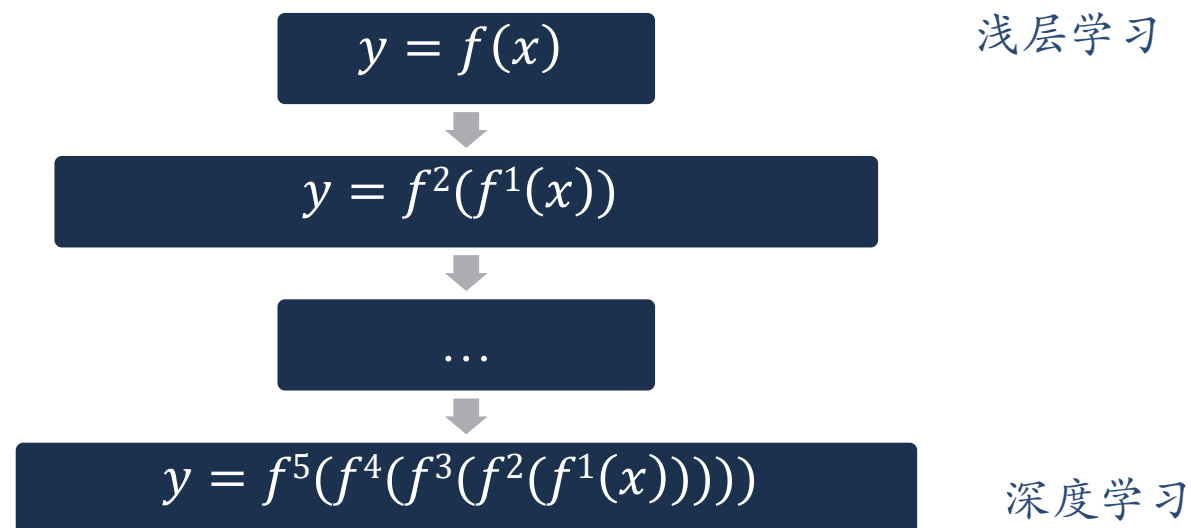
▶ 自编码器

▶ 特征提取VS表示学习

▶ 特征提取：基于任务或先验对去除无用特征

▶ 表示学习：通过深度模型学习高层语义特征





$f^1(x)$ 为非线性函数，不一定连续。

当 $f^1(x)$ 连续时，比如 $f^1(x) = \sigma(W^1 f^{1-1}(x))$ ，这个复合函数称为神经网络。

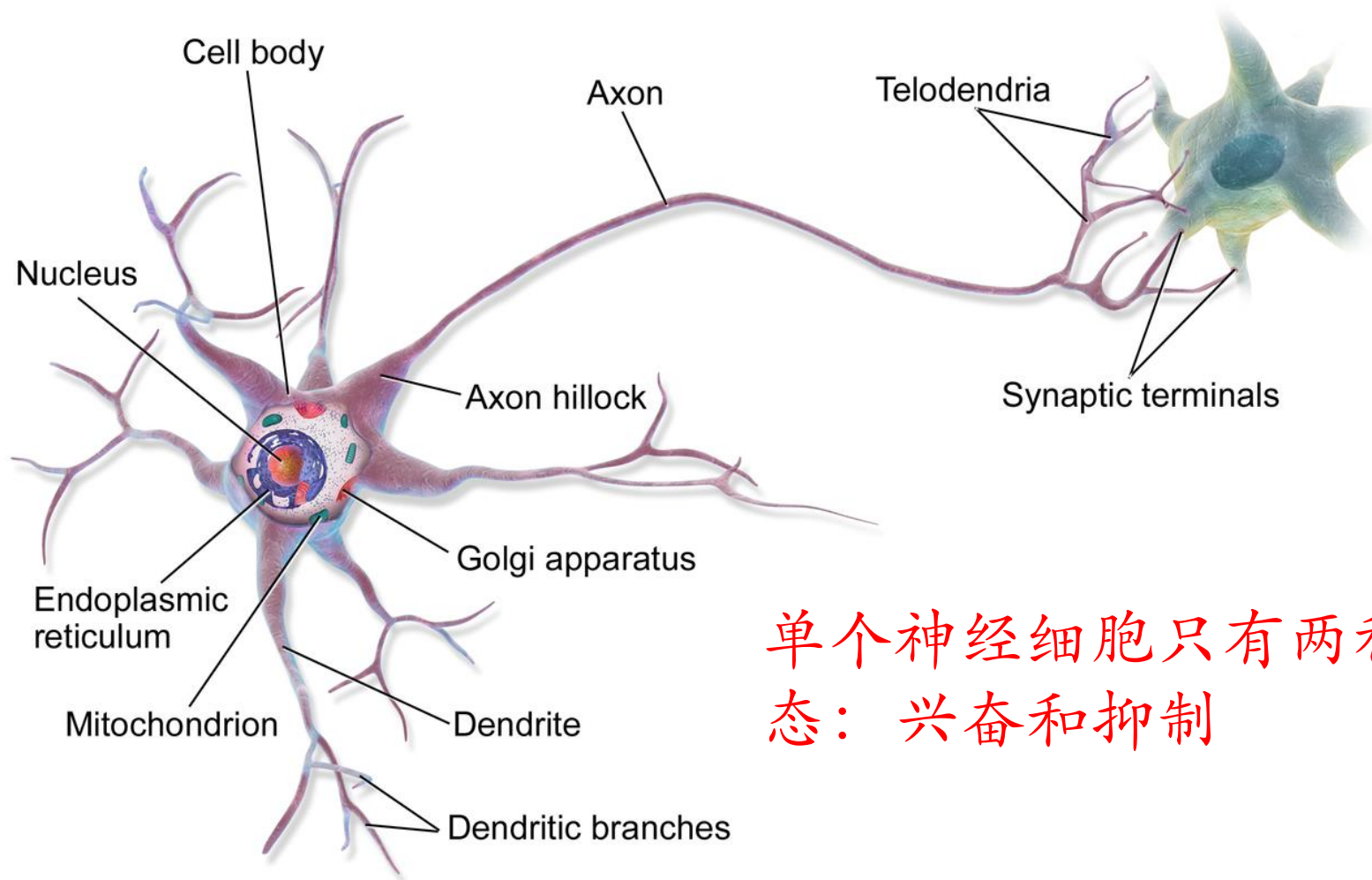


北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY

神经网络

人脑有860亿个神经元

[video: structure of brain](#)



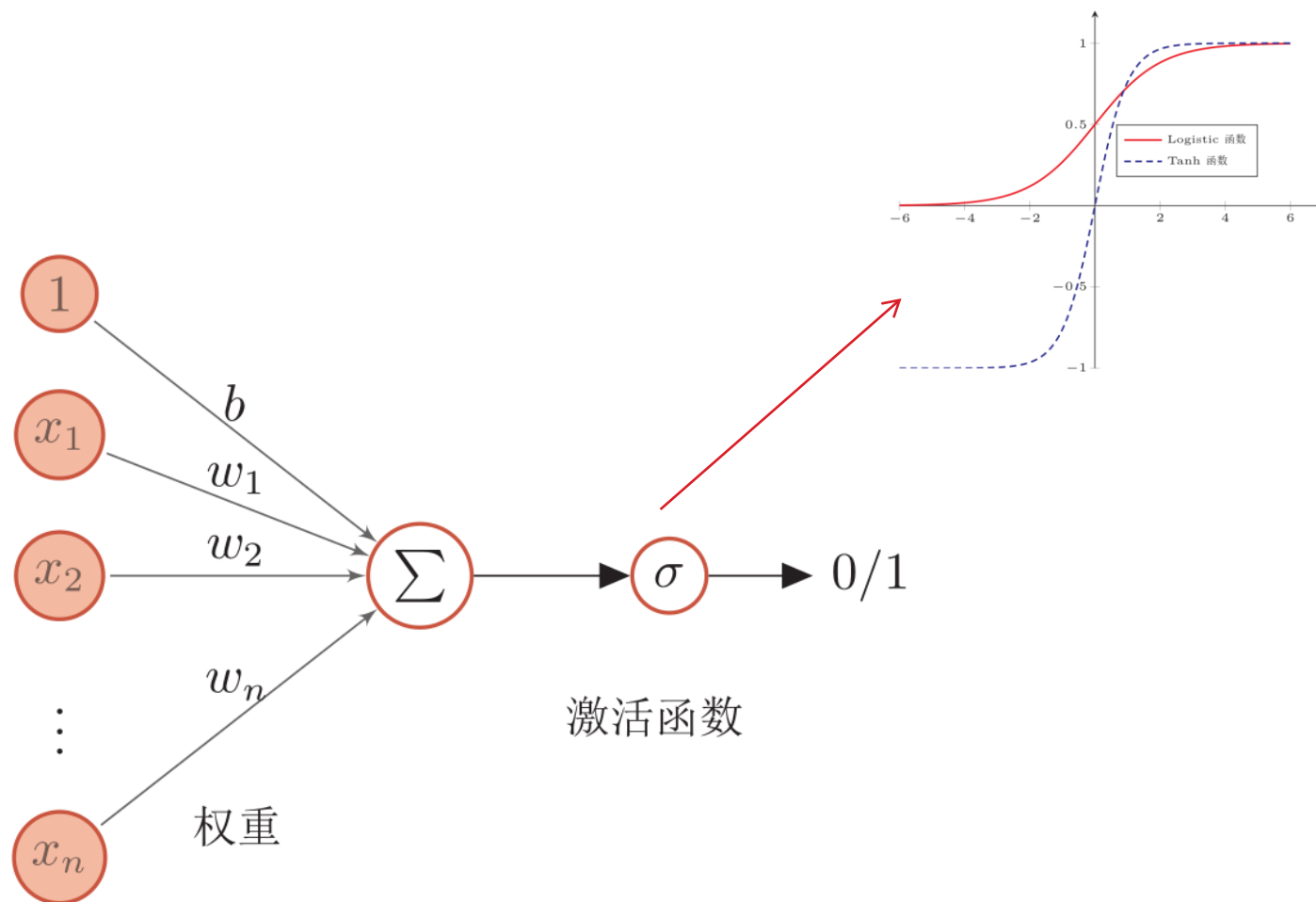
单个神经细胞只有两种状态：兴奋和抑制

▶ 赫布法则 Hebb's Rule

- ▶ “当神经元A的一个轴突和神经元B很近，足以对它产生影响，并且持续地、重复地参与了对神经元B的兴奋，那么在这两个神经元或其中之一会发生某种生长过程或新陈代谢变化，以致于神经元A作为能使神经元B兴奋的细胞之一，它的效能加强了。”

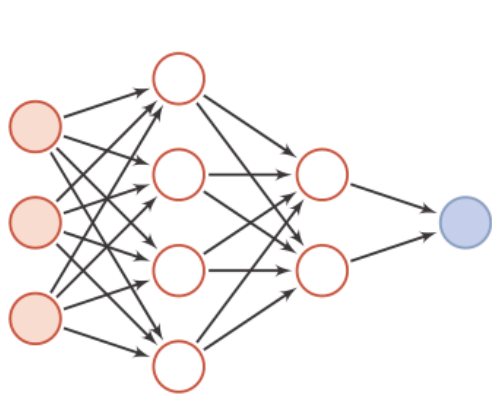
----加拿大心理学家Donald Hebb,
《行为的组织》，1949

- 人脑有两种记忆：**长期记忆**和**短期记忆**。短期记忆持续时间不超过一分钟。如果一个经验重复足够的次数，此经验就可储存在长期记忆中。
- 短期记忆转化为长期记忆的过程就称为**凝固作用**。
- 人脑中的海马区为大脑结构凝固作用的核心区域。

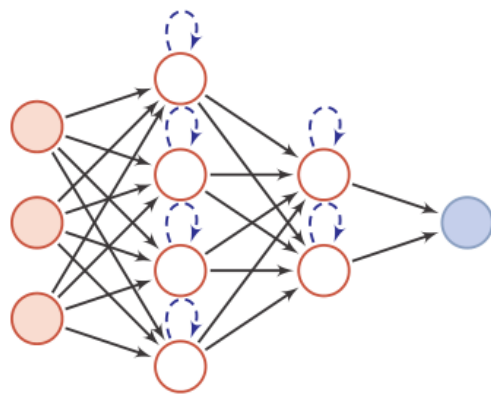


- ▶ 人工神经网络主要由大量的神经元以及它们之间的有向连接构成。因此考虑三方面：
 - ▶ 神经元的激活规则
 - ▶ 主要是指神经元输入到输出之间的映射关系，一般为非线性函数。
 - ▶ 网络的拓扑结构
 - ▶ 不同神经元之间的连接关系。
 - ▶ 学习算法
 - ▶ 通过训练数据来学习神经网络的参数。

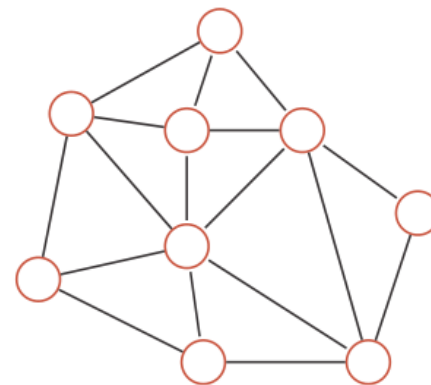
- ▶ 人工神经网络由神经元模型构成，这种由许多神经元组成的信息处理网络具有并行分布结构。
- ▶ 虽然这里将神经网络结构大体上分为三种类型，但是大多数网络都是复合型结构，即一个神经网络中包括多种网络结构。



(a) 前馈网络



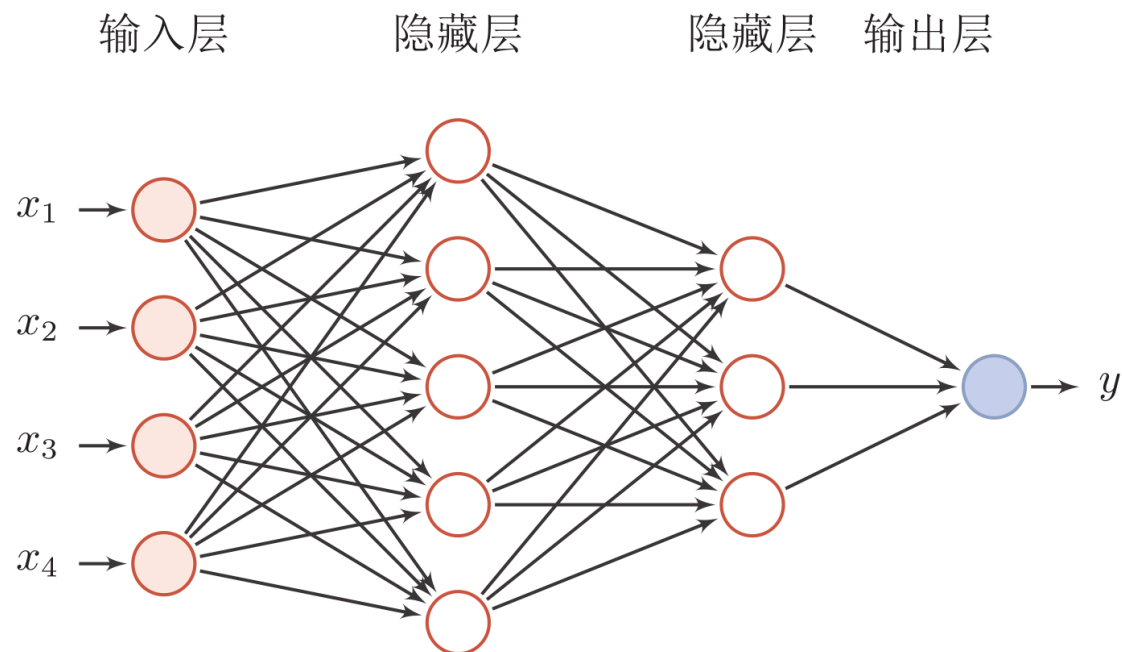
(b) 记忆网络



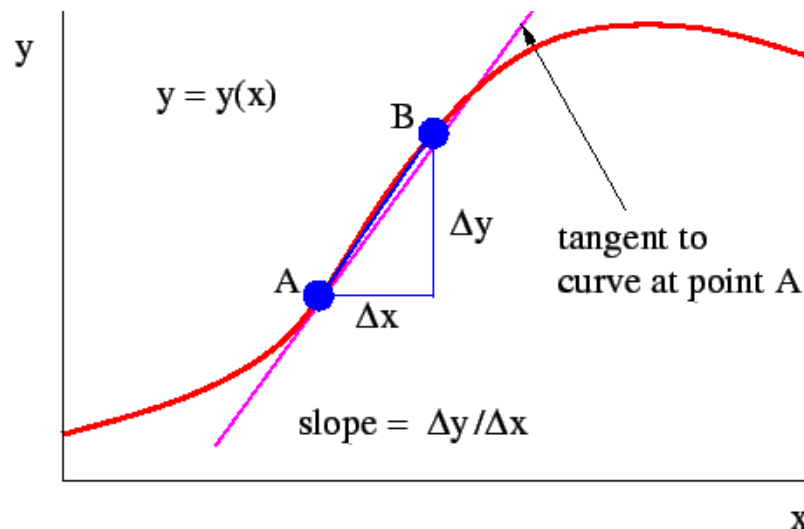
(c) 图网络

$$y = (f^3(f^2(f^1(x))))$$

$$f^1(x) = \sigma(W^1 f^{1-1}(x))$$



► 偏导数



► 贡献度

$$\frac{\partial y}{\partial W^l} = \frac{y(W^l + \Delta W^l) - y(W^l)}{\Delta W^l}$$

神经网络天然不是深度学习，但深度学习天然神经网络。

- ▶神经网络的发展大致经过五个阶段。
- ▶第一阶段：模型提出
 - ▶在1943年，心理学家Warren McCulloch和数学家Walter Pitts和最早描述了一种理想化的人工神经网络，并构建了一种基于简单逻辑运算的计算机制。他们提出的神经网络模型称为MP模型。
 - ▶阿兰·图灵在1948年的论文中描述了一种“B型图灵机”。(赫布型学习)
 - ▶1951年，McCulloch和Pitts的学生Marvin Minsky建造了第一台神经网络机，称为SNARC。
 - ▶Rosenblatt [1958]最早提出可以模拟人类感知能力的神经网络模型，并称之为感知器（Perceptron），并提出了一种接近于人类学习过程（迭代、试错）的学习算法。

▶ 第二阶段：冰河期

- ▶ 1969年，Marvin Minsky出版《感知器》一书，书中论断直接将神经网络打入冷宫，导致神经网络十多年的“冰河期”。他们发现了神经网络的两个关键问题：
 - ▶ 1) 基本感知器无法处理异或回路。
 - ▶ 2) 电脑没有足够的能力来处理大型神经网络所需要的很长的计算时间。
- ▶ 1974年，哈佛大学的Paul Webos发明反向传播算法，但当时未受到应有的重视。
- ▶ 1980年，Kunihiko Fukushima（福岛邦彦）提出了一种带卷积和子采样操作的多层神经网络：新知机（Neocognitron）

▶ 第三阶段：反向传播算法引起的复兴

- ▶ 1983年，物理学家John Hopfield对神经网络引入能量函数的概念，并提出了用于联想记忆和优化计算的神经网络（称为Hopfield网络），在旅行商问题上获得当时最好结果，引起轰动。
- ▶ 1984年，Geoffrey Hinton提出一种随机化版本的Hopfield网络，即玻尔兹曼机。
- ▶ 1986年，David Rumelhart和James McClelland对于联结主义在计算机模拟神经活动中的应用提供了全面的论述，并重新发明了反向传播算法。
- ▶ 1986年，Geoffrey Hinton等人将引入反向传播算法到多层感知器
- ▶ 1989年，LeCun等人将反向传播算法引入了卷积神经网络，并在手写体数字识别上取得了很大的成功。

▶ 第四阶段：流行度降低

- ▶ 在20世纪90年代中期，统计学习理论和以支持向量机为代表的机器学习模型开始兴起。
- ▶ 相比之下，神经网络的理论基础不清晰、优化困难、可解释性差等缺点更加凸显，神经网络的研究又一次陷入低潮。

▶ 第五阶段：深度学习的崛起

- ▶ 2006年，Hinton等人发现多层前馈神经网络可以先通过逐层预训练，再用反向传播算法进行精调的方式进行有效学习。
 - ▶ 深度神经网络在语音识别和图像分类等任务上的巨大成功。
- ▶ 2013年，AlexNet：第一个现代深度卷积网络模型，是深度学习技术在图像分类上取得真正突破的开端。
 - ▶ AlexNet不用预训练和逐层训练，首次使用了很多现代深度网络的技术
- ▶ 随着大规模并行计算以及GPU设备的普及，计算机的计算能力得以大幅提高。此外，可供机器学习的数据规模也越来越大。在计算能力和数据规模的支持下，计算机已经可以训练大规模的人工神经网络。

▶ AI领域

- ▶ 语音识别：可以使得词错误率从1/4下降到1/8
- ▶ 计算机视觉：目标识别、图像分类等
- ▶ 自然语言处理：分布式表示、机器翻译、问题回答等
- ▶ 信息检索、社会化网络

▶ 三个Deep:

- ▶ Deep Blue
- ▶ Deep QA
- ▶ Deep Learning

- ▶ 简易和快速的原型设计
- ▶ 自动梯度计算
- ▶ 无缝CPU和GPU切换

 PaddlePaddle


TensorFlow


Chainer


Microsoft
CNTK


Caffe2


Keras


theano


PYTORCH


APACHE
mxnet

德以明理 学以精工

谢谢



智能感知

日期：2025年8月



目录

CONTENTS

- 1 绪论
- 2 机器学习基础
- 3 线性模型
- 4 前馈神经网络
- 5 卷积神经网络

▶ 机器学习

- ▶ 概念

- ▶ 原理

▶ 线性回归

- ▶ 定义

- ▶ 经验风险最小化

 - ▶ 最小均方误差

- ▶ 结构风险最小化

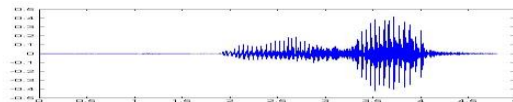
- ▶ 最大似然估计

- ▶ 最大后验估计

▶ 机器学习的几个关键点

▶ 语音识别

$$f(\text{语音波形图}) = \text{“你好”}$$



▶ 图像识别

$$f(\text{猫咪照片}) = \text{“猫”}$$



▶ 围棋

$$f(\text{围棋棋盘}) = \text{“5-5” (落子位置)}$$



▶ 对话系统

$$f(\text{“你好”}) = \text{“今天天气真不错”}$$

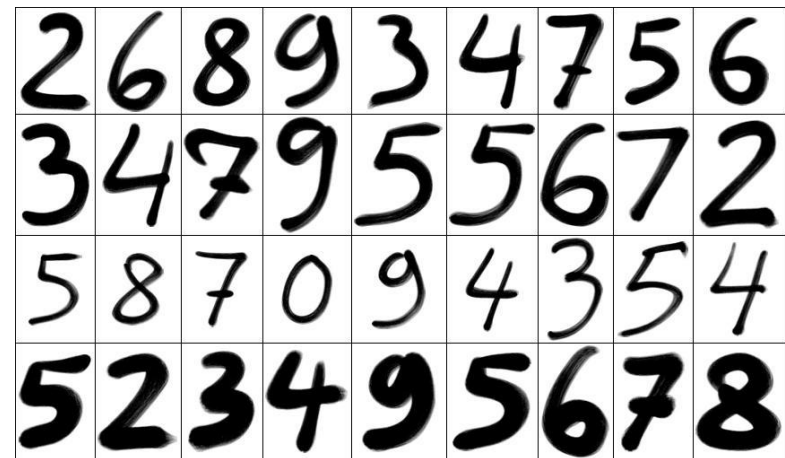
用户输入

机器

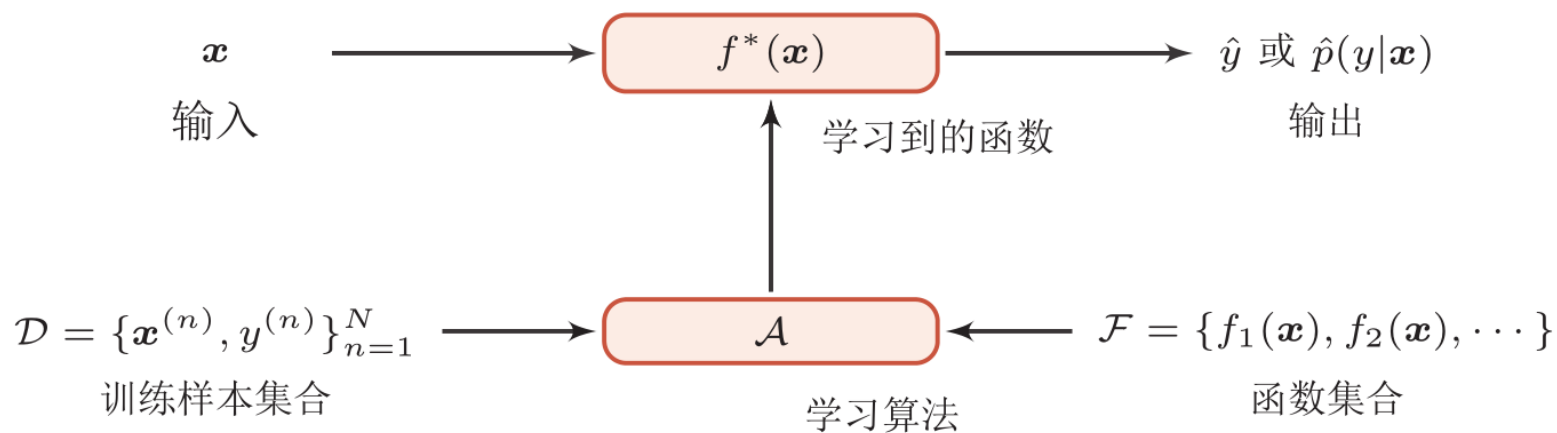
为什么要“机器学习”？



- ▶ 现实世界的问题都比较复杂
- ▶ 很难通过规则来手工实现



- ▶ 机器学习：通过算法使得机器能从大量数据中学习规律从而对新的样本做决策。
- ▶ 规律：决策（预测）函数



独立同分布 $p(\mathbf{x}, y)$

▶ 模型

▶ 线性方法: $f(\mathbf{x}, \theta) = \mathbf{w}^T \mathbf{x} + b$

▶ 广义线性方法: $f(\mathbf{x}, \theta) = \mathbf{w}^T \phi(\mathbf{x}) + b$

▶ 如果 $\phi(\mathbf{x})$ 为可学习的非线性基函数, $f(\mathbf{x}, \theta)$ 就等价于神经网络。

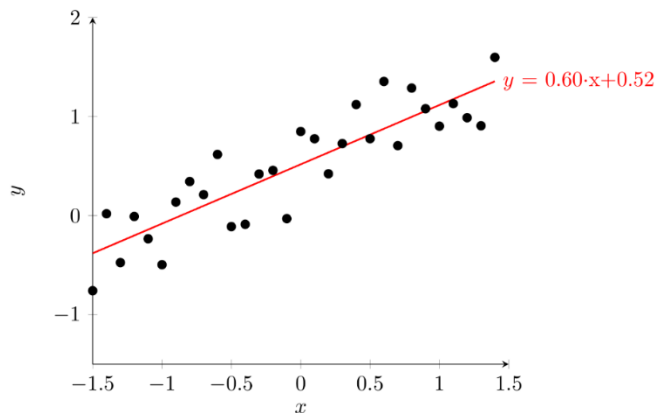
▶ 学习准则

▶ 期望风险

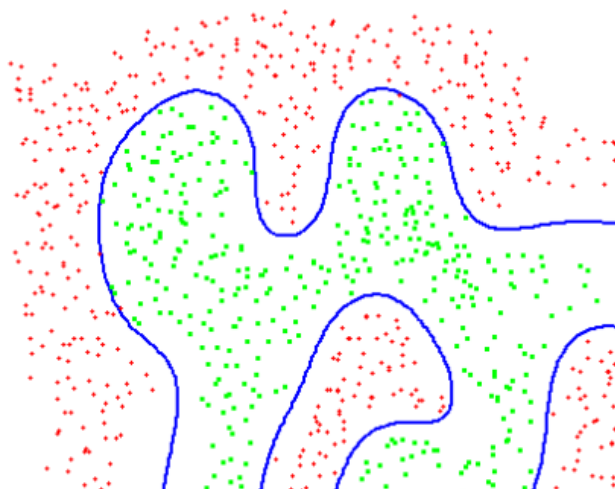
$$\mathcal{R}(f) = \mathbb{E}_{(\mathbf{x}, y) \sim p(\mathbf{x}, y)} [\mathcal{L}(f(\mathbf{x}), y)],$$

▶ 优化

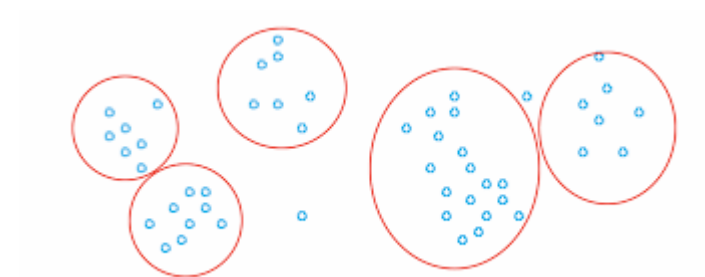
▶ 梯度下降



回归



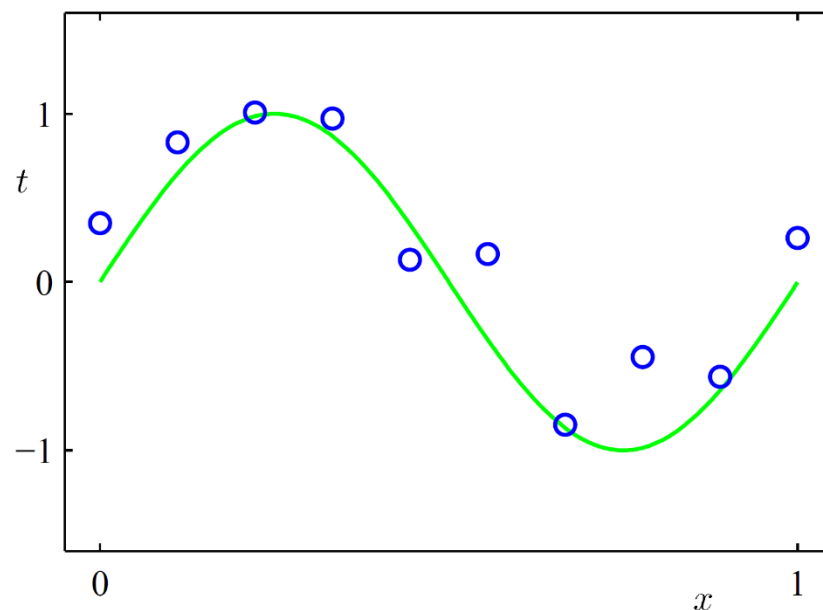
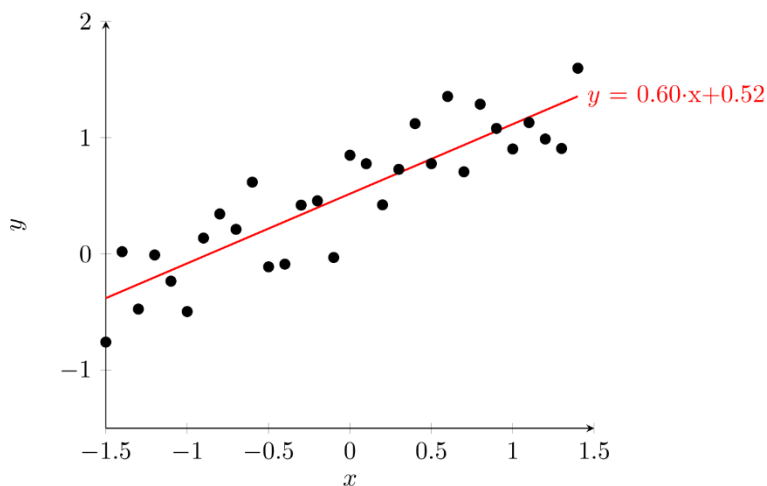
分类



聚类

- ▶ 以线性回归 (Linear Regression) 为例
- ▶ 模型:

$$f(\mathbf{x}; \mathbf{w}, b) = \mathbf{w}^T \mathbf{x} + b$$



模型

$$y(x, \mathbf{w}) = w_0 + w_1 x + w_2 x^2 + \dots + w_M x^M$$

▶ 损失函数

▶ 0-1损失函数

$$\mathcal{L}(y, f(x, \theta)) = \begin{cases} 0 & \text{if } y = f(x, \theta) \\ 1 & \text{if } y \neq f(x, \theta) \end{cases}$$

▶ 平方损失函数

$$\mathcal{L}(y, \hat{y}) = (y - f(x, \theta))^2$$

▶ 交叉熵损失函数

► 期望风险未知，通过经验风险近似

► 训练数据： $\mathcal{D} = \{x^{(n)}, y^{(n)}\}, i \in [1, N]$

$$\mathcal{R}_{\mathcal{D}}^{emp}(\theta) = \frac{1}{N} \sum_{n=1}^N \mathcal{L}(y^{(n)}, f(x^{(n)}, \theta))$$

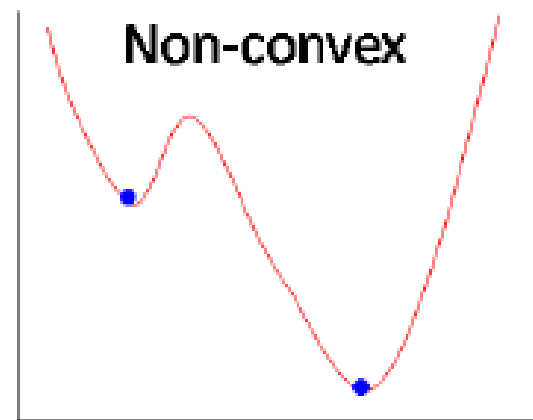
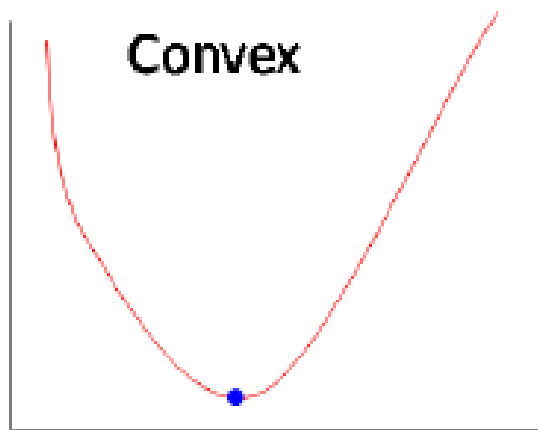
► 经验风险最小化

► 在选择合适的风险函数后，我们寻找一个参数 θ^* ，使得经验风险函数最小化。

$$\theta^* = \arg \min_{\theta} \mathcal{R}_{\mathcal{D}}^{emp}(\theta)$$

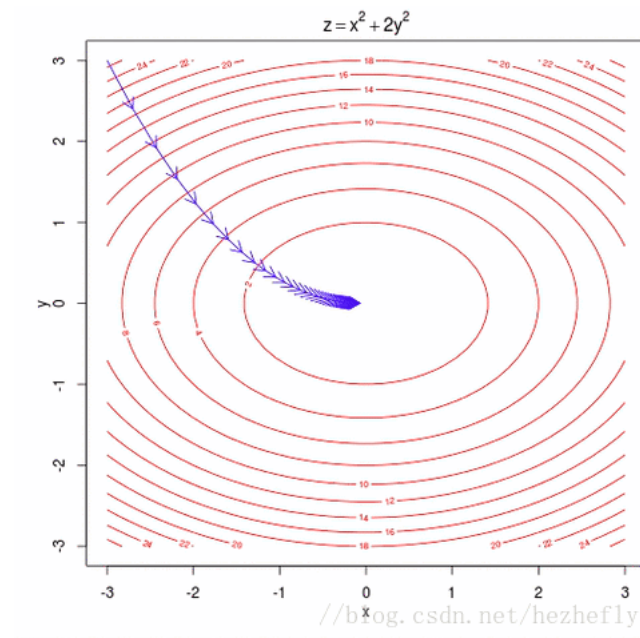
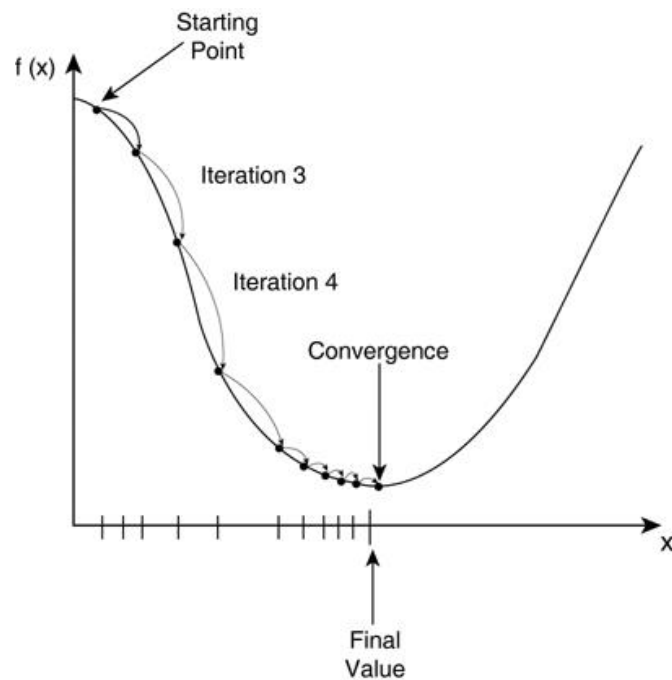
► 机器学习问题转化为一个最优化问题

► 机器学习问题转化成为一个最优化问题



$$\min_{\mathbf{x}} f(\mathbf{x})$$

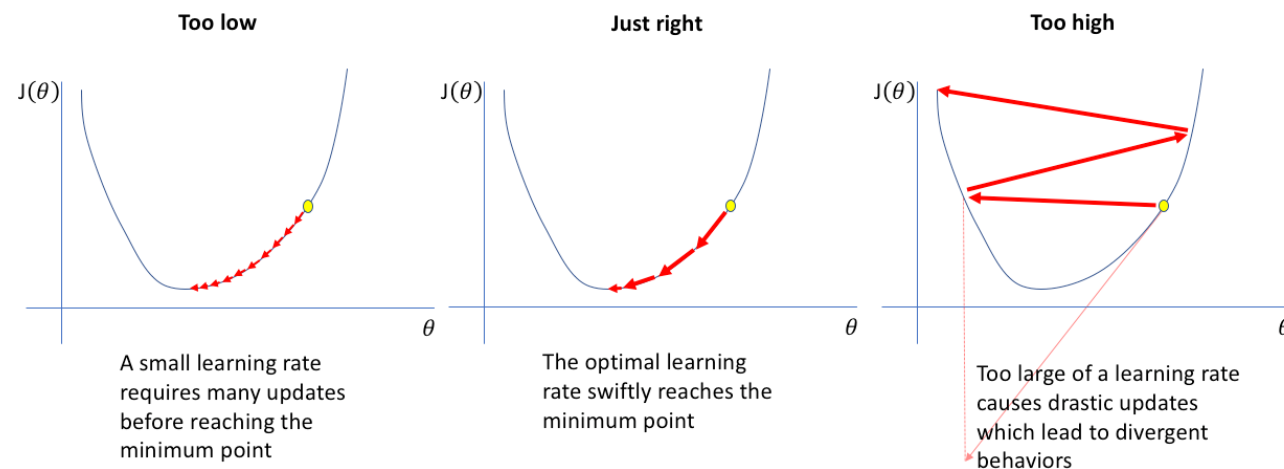
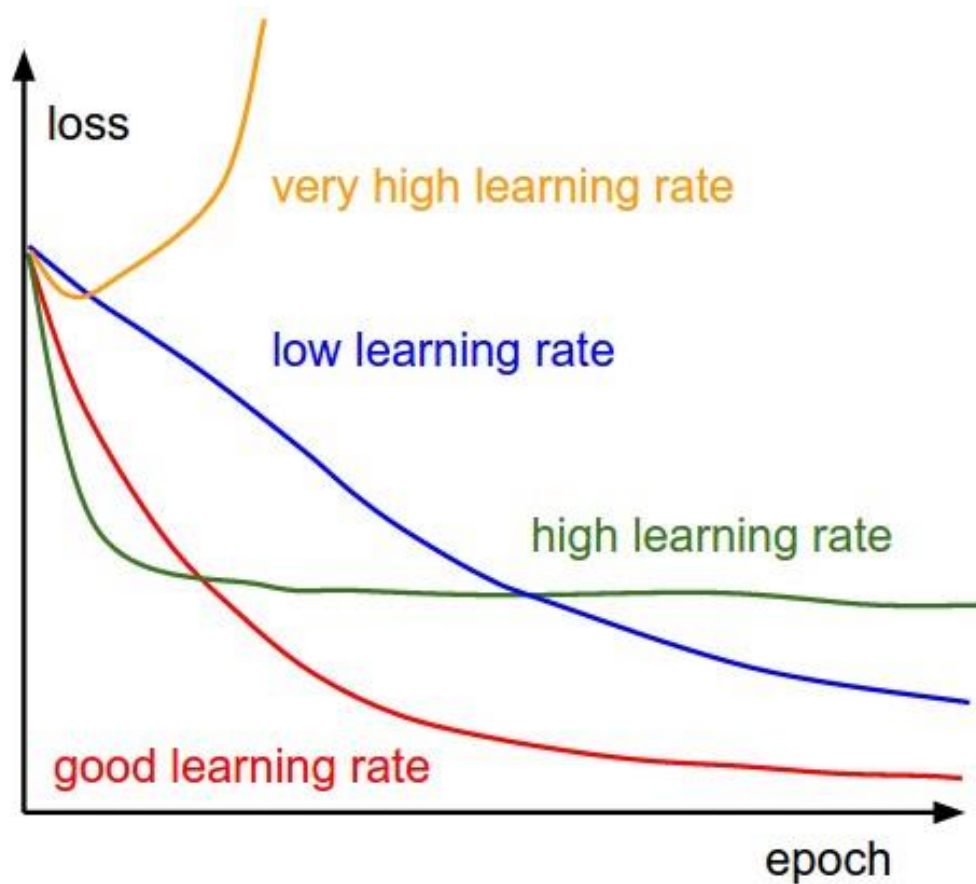
梯度下降法 (Gradient Descent)



$$\begin{aligned}\theta_{t+1} &= \theta_t - \alpha \frac{\partial \mathcal{R}(\theta)}{\partial \theta_t} \\ &= \theta_t - \alpha \frac{1}{N} \sum_{i=1}^N \frac{\partial \mathcal{L}(\theta_t; x^{(i)}, y^{(i)})}{\partial \theta}.\end{aligned}$$

搜索步长 α 中也叫作学习率 (Learning Rate)

学习率是十分重要的超参数！



- ▶ 随机梯度下降法 (Stochastic Gradient Descent, SGD) 也叫增量梯度下降, 每个样本都进行更新

$$\theta_{t+1} = \theta_t - \alpha \frac{\partial \mathcal{L}(\theta_t; x^{(t)}, y^{(t)})}{\partial \theta},$$

- ▶ 小批量 (Mini-Batch) 随机梯度下降法

算法 2.1: 随机梯度下降法

输入: 训练集 $\mathcal{D} = \{(\mathbf{x}^{(n)}, y^{(n)})\}_{n=1}^N$, 验证集 $\mathcal{V}$, 学习率 α

1 随机初始化 θ ;

2 repeat

3 对训练集 $\mathcal{D}$ 中的样本随机重排序;

4 for $n = 1 \cdots N$ do

5 从训练集 $\mathcal{D}$ 中选取样本 $(\mathbf{x}^{(n)}, y^{(n)})$;

 // 更新参数

6 $\theta \leftarrow \theta - \alpha \frac{\partial \mathcal{L}(\theta; x^{(n)}, y^{(n)})}{\partial \theta}$;

7 end

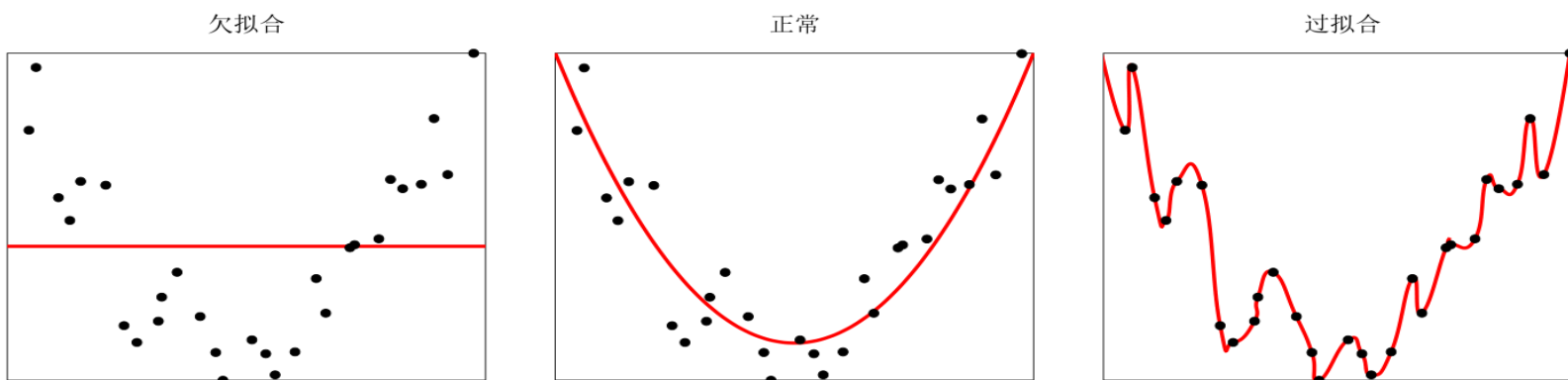
8 until 模型 $f(\mathbf{x}; \theta)$ 在验证集 $\mathcal{V}$ 上的错误率不再下降;

输出: θ



Why?

机器学习 = 优化? NO!



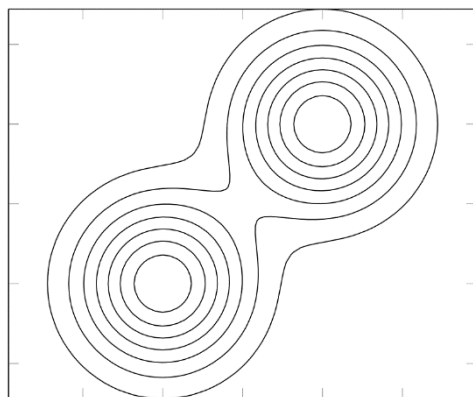
过拟合：经验风险最小化原则很容易导致模型在训练集上错误率很低，但是在未知数据上错误率很高。

过拟合问题往往是由于训练数据少和噪声等原因造成的。

期望风险

$$\mathcal{R}(f) = \mathbb{E}_{(\mathbf{x}, y) \sim p(\mathbf{x}, y)} [\mathcal{L}(f(\mathbf{x}), y)],$$

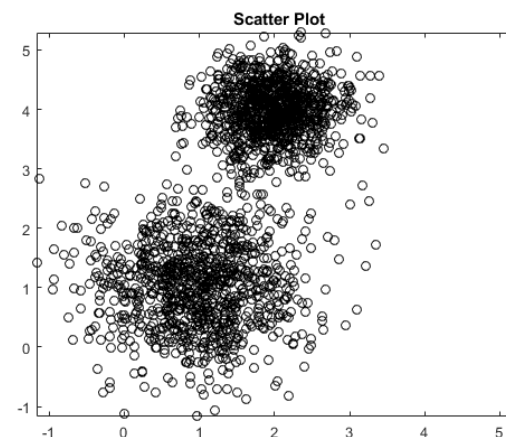
真实分布 p_r



$\neq$

经验风险

$$\mathcal{R}_D^{emp}(\theta) = \frac{1}{N} \sum_{n=1}^N \mathcal{L}(y^{(n)}, f(x^{(n)}, \theta))$$



$$\mathcal{G}_D(f) = \mathcal{R}(f) - \mathcal{R}_D^{emp}(f)$$

泛化错误

优化

经验风险最小

正则化

降低模型复杂度



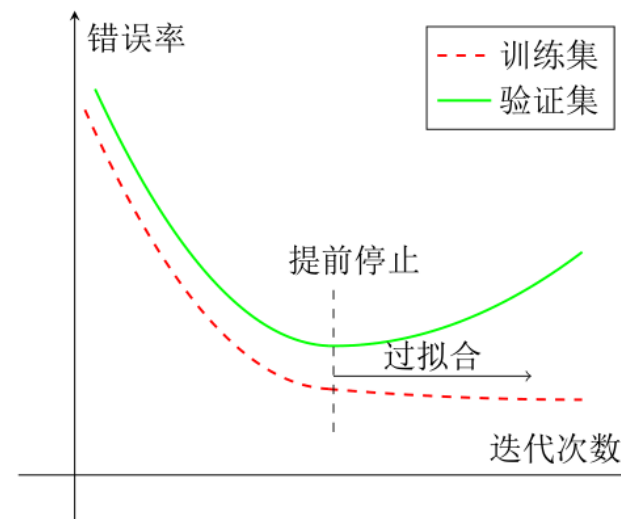
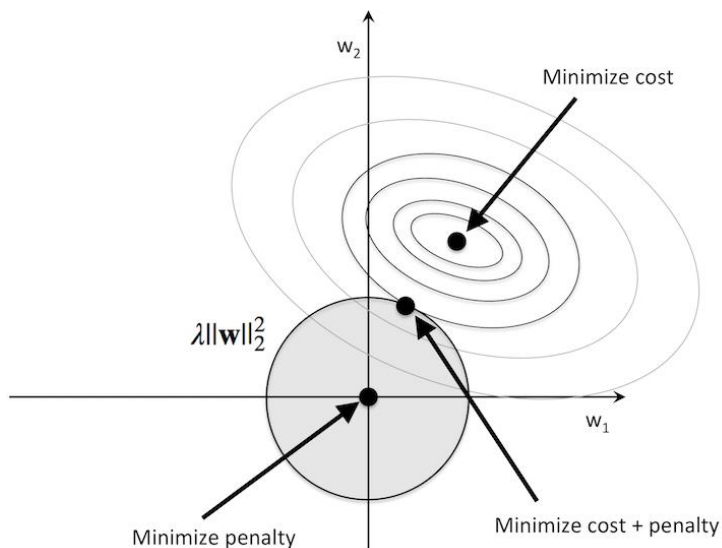
所有损害优化的方法都是正则化。

增加优化约束

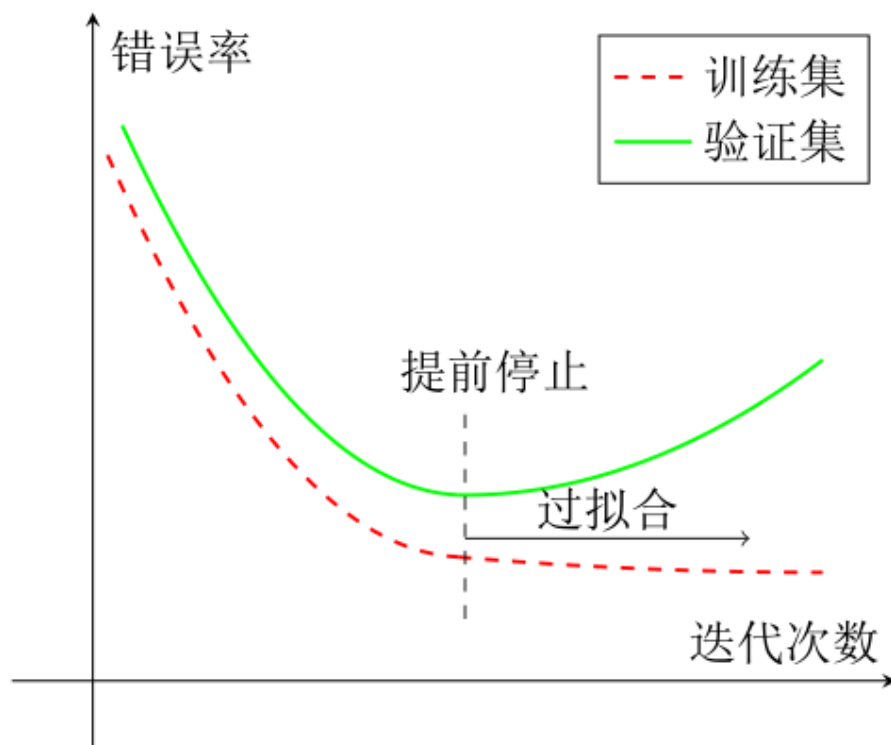
L1/L2约束、数据增强

干扰优化过程

权重衰减、随机梯度下降、提前停止



- 使用一个验证集 (Validation Dataset) 来测试每一次迭代的参数在验证集上是否最优。如果在验证集上的错误率不再下降, 就停止迭代。





北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY

线性回归

► 模型:

$$f(\mathbf{x}; \mathbf{w}, b) = \mathbf{w}^T \mathbf{x} + b$$

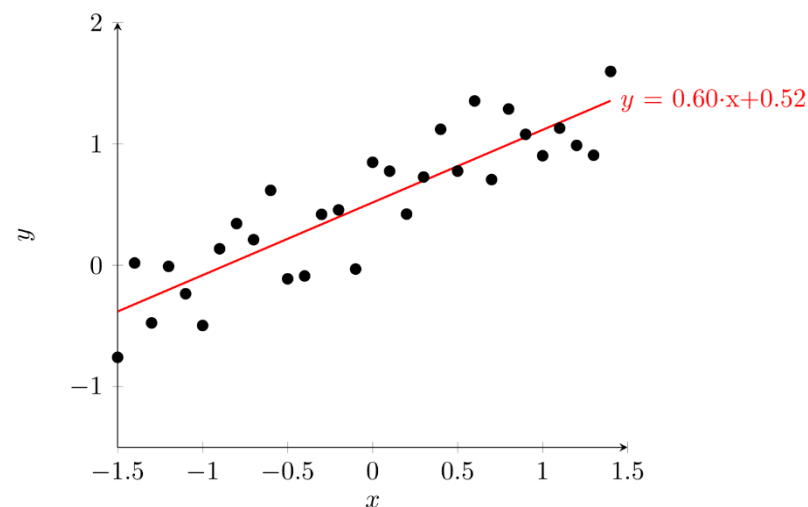
► 增广权重向量和增广特征向量

$$\hat{\mathbf{x}} = \mathbf{x} \oplus 1 \triangleq \begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix} = \begin{bmatrix} x_1 \\ \vdots \\ x_k \\ 1 \end{bmatrix},$$

$$\hat{\mathbf{w}} = \mathbf{w} \oplus b \triangleq \begin{bmatrix} \mathbf{w} \\ b \end{bmatrix} = \begin{bmatrix} w_1 \\ \vdots \\ w_k \\ b \end{bmatrix},$$



$$f(\mathbf{x}; \hat{\mathbf{w}}) = \hat{\mathbf{w}}^T \hat{\mathbf{x}},$$



- ▶ 经验风险最小化（最小二乘法）
- ▶ 结构风险最小化（岭回归）
- ▶ 最大似然估计
- ▶ 最大后验估计



北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY

经验风险最小化

► 标量关于向量的偏导数

$$\frac{\partial y}{\partial \mathbf{x}} = \left[\frac{\partial y}{\partial x_1}, \dots, \frac{\partial y}{\partial x_M} \right]^\top$$

► 向量关于向量的偏导数

$$\frac{\partial f(\mathbf{x})}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \dots & \frac{\partial y_N}{\partial x_1} \\ \vdots & \vdots & \vdots \\ \frac{\partial y_1}{\partial x_M} & \dots & \frac{\partial y_N}{\partial x_M} \end{bmatrix}$$

► 向量函数及其导数

$$\begin{aligned} \frac{\partial \mathbf{x}}{\partial \mathbf{x}} &= \mathbf{I}, \\ \frac{\partial \mathbf{A}\mathbf{x}}{\partial \mathbf{x}} &= \mathbf{A}^\top, \\ \frac{\partial \mathbf{x}^\top \mathbf{A}}{\partial \mathbf{x}} &= \mathbf{A} \end{aligned}$$

► 模型

$$f(\mathbf{x}; \mathbf{w}) = \mathbf{w}^T \mathbf{x}$$

► 学习准则

$$\begin{aligned}\mathcal{R}(\mathbf{w}) &= \sum_{n=1}^N \mathcal{L}(y^{(n)}, f(\mathbf{x}^{(n)}; \mathbf{w})) \\ &= \frac{1}{2} \sum_{n=1}^N \left(y^{(n)} - \mathbf{w}^T \mathbf{x}^{(n)} \right)^2 \\ &= \frac{1}{2} \|\mathbf{y} - X^T \mathbf{w}\|^2,\end{aligned}$$

► 优化

$$\frac{\partial}{\partial \mathbf{w}} \mathcal{R}(\mathbf{w}) = 0$$

$$\frac{\partial x^T x}{\partial x} = 2x$$

$$\frac{\partial Ax}{\partial x} = A^T$$

$$\begin{aligned} \frac{\partial \mathcal{R}(\mathbf{w})}{\partial \mathbf{w}} &= \frac{1}{2} \frac{\partial \|\mathbf{y} - X^T \mathbf{w}\|^2}{\partial \mathbf{w}} \\ &= -X(\mathbf{y} - X^T \mathbf{w}), \end{aligned}$$

▶ 结构风险最小化准则

$$\mathcal{R}(\mathbf{w}) = \frac{1}{2} \|\mathbf{y} - \mathbf{X}^\top \mathbf{w}\|^2 + \frac{1}{2} \lambda \|\mathbf{w}\|^2,$$

▶ 得到

$$\mathbf{w}^* = (\mathbf{X}\mathbf{X}^\top + \lambda \mathbf{I})^{-1} \mathbf{X}\mathbf{y},$$

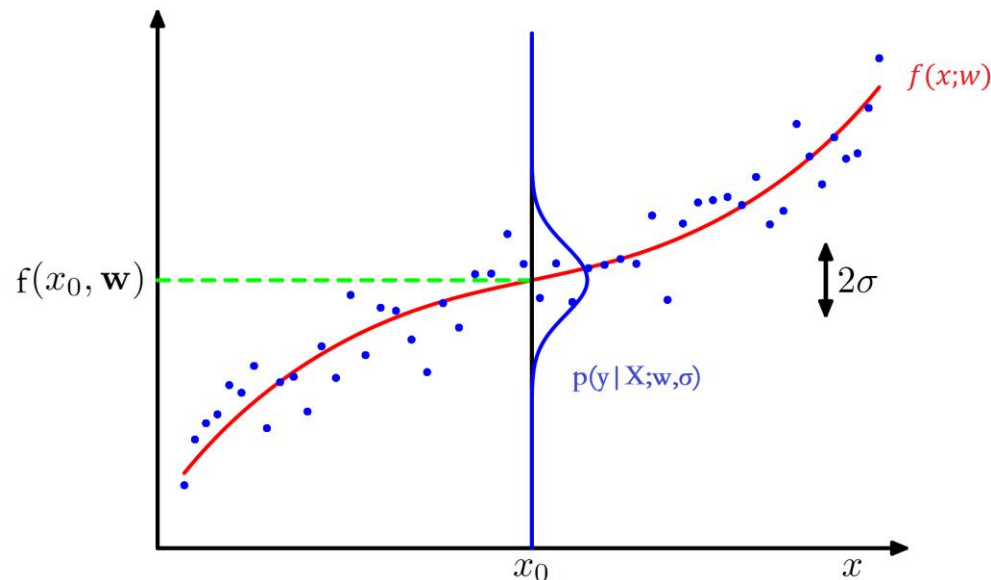


北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY

最大似然估计

- 假设标签 y 为一个随机变量，其服从以均值为 $f(x; \mathbf{w}) = \mathbf{w}^T \mathbf{x}$ ，方差为 σ^2 的高斯分布。

$$\begin{aligned} p(y|\mathbf{x}; \mathbf{w}, \sigma) &= \mathcal{N}(y; \mathbf{w}^T \mathbf{x}, \sigma^2) \\ &= \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y - \mathbf{w}^T \mathbf{x})^2}{2\sigma^2}\right). \end{aligned}$$



► 概率 (Probability)

► 一个随机事件发生的可能性大小，为0到1之间的实数。

► 随机变量 (Random Variable)

► 比如随机掷一个骰子，得到的点数就可以看成一个随机变量 X ，其取值为 $\{1, 2, 3, 4, 5, 6\}$ 。

► 概率分布 (Probability Distribution)

► 一个随机变量 X 取每种可能值的概率

$$P(X = x_i) = p(x_i), \quad \forall i \in \{1, \dots, n\}.$$

► 并满足

$$\sum_{i=1}^n p(x_i) = 1,$$

$$p(x_i) \geq 0, \quad \forall i \in \{1, \dots, n\}.$$

► 伯努利分布 (Bernoulli Distribution)

- 在一次试验中，事件A出现的概率为 μ ，不出现的概率为 $1 - \mu$ 。若用变量X表示事件A出现的次数，则X的取值为0和1，其相应的分布为

$$p(x) = \mu^x (1 - \mu)^{(1-x)}$$

► 二项分布 (Binomial Distribution)

- 在n次伯努利分布中，若以变量X表示事件A出现的次数，则X的取值为 $\{0, \dots, n\}$ ， $P(X = k) = \binom{n}{k} \mu^k (1 - \mu)^{n-k}$ ， $k = 1 \dots, n$

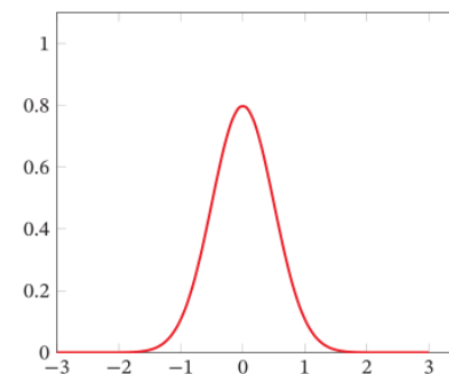
二项式系数，表示从n个元素中取出k个元素而不考虑其顺序的组合的总数。

- ▶ 连续随机变量 Y 的概率分布一般用概率密度函数 (Probability Density Function , PDF) $p(x)$ 来描述。

$$\int_{-\infty}^{+\infty} p(x)dx = 1$$

- ▶ 高斯分布 (Gaussian Distribution)

$$X \sim \mathcal{N}(\mu, \sigma^2) \quad p(x) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$



► 条件概率 (Conditional Probability)

- 对于离散随机向量 (X, Y) , 已知 $X = x$ 的条件下, 随机变量 $Y = y$ 的条件概率为:

$$p(y|x) = P(Y = y|X = x) = \frac{p(x, y)}{p(x)}$$

► 贝叶斯公式

- 两个条件概率 $p(y|x)$ 和 $p(x|y)$ 之间的关系

$$p(y|x) = \frac{p(x|y)p(y)}{p(x)}$$

- ▶ 似然函数是关于统计模型 $P(x;w)$ ，看做参数 w 的函数
- ▶ 概率 $P(x;w)$ 是描述固定参数 w 时，随机变量 x 的分布情况
- ▶ 似然 $P(x;w)$ 则是描述已知随机变量 x 时不同参数对 w 分布的影响

► 参数 $\mathbf{w}$ 在训练集 D 上的似然函数 (Likelihood) 为

$$\begin{aligned} p(\mathbf{y}|\mathbf{X}; \mathbf{w}, \sigma) &= \prod_{n=1}^N p(y^{(n)}|\mathbf{x}^{(n)}; \mathbf{w}, \sigma) \\ &= \prod_{n=1}^N \mathcal{N}(y^{(n)}; \mathbf{w}^T \mathbf{x}^{(n)}, \sigma^2) \end{aligned}$$

- ▶ 最大似然估计 (Maximum Likelihood Estimate, MLE)
- ▶ 是指找到一组参数 w 使得似然函数 $p(y|X;w,\sigma)$ 最大

$$\text{令 } \frac{\partial \log p(\mathbf{y}|X; \mathbf{w}, \sigma)}{\partial \mathbf{w}} = 0$$



$$\mathbf{w}^{ML} = (X X^T)^{-1} X \mathbf{y}.$$



北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY

最大后验估计

▶ 贝叶斯学习

▶ 将参数 w 也看做随机变量

▶ 目标：给定一组观测数据 X , 求参数 w 的分布 $P(w|X)$

▶ $P(w|X)$ 也称为**后验分布** (posterior)

贝叶斯公式：

$$p(Y|X) = \frac{p(X|Y)p(Y)}{p(X)}$$

$$p(w|X) \propto p(X|w)p(w)$$

后验	似然	先验
posterior	likelihood	prior

$$\text{posterior} \propto \text{likelihood} \times \text{prior}$$

- ▶ 采用贝叶斯估计的线性回归也称为贝叶斯线性回归
- ▶ 带入贝叶斯估计，可得参数 $\mathbf{w}$ 的后验分布

$$p(\mathbf{w}|\mathbf{X}, \mathbf{y}; \nu, \sigma) = \frac{p(\mathbf{w}, \mathbf{y}|\mathbf{X}; \nu, \sigma)}{\sum_{\mathbf{w}} p(\mathbf{w}, \mathbf{y}|\mathbf{X}; \nu, \sigma)} \quad p(\mathbf{w}; \nu) = \mathcal{N}(\mathbf{w}; \mathbf{0}, \nu^2 \mathbf{I})$$
$$\propto \underbrace{p(\mathbf{y}|\mathbf{X}, \mathbf{w}; \sigma)}_{\text{似然}} p(\mathbf{w}; \nu),$$

后验 似然 先验
posterior likelihood prior

$$\log p(\mathbf{w}|\mathbf{X}, \mathbf{y}; \nu, \sigma) \propto \log p(\mathbf{y}|\mathbf{X}, \mathbf{w}; \sigma) + \log p(\mathbf{w}; \nu)$$
$$\propto -\frac{1}{2\sigma^2} \sum_{n=1}^N \left(y^{(n)} - \mathbf{w}^\top \mathbf{x}^{(n)} \right)^2 - \frac{1}{2\nu^2} \mathbf{w}^\top \mathbf{w},$$
$$= -\frac{1}{2\sigma^2} \underbrace{\|\mathbf{y} - \mathbf{X}^\top \mathbf{w}\|^2}_{\text{正则化系数}} - \frac{1}{2\nu^2} \mathbf{w}^\top \mathbf{w}. \quad \lambda = \sigma^2/\nu^2$$

- ▶ 最大后验估计: $\mathbf{w}^{MAP} = \max_{\mathbf{w}} p(\mathbf{y}|\mathbf{X}, \mathbf{w}; \sigma) p(\mathbf{w}; \nu)$

	无先验	引入先验
平方误差	经验风险 最小化	结构风险 最小化
概率	最大似然估计	最大后验估计

$$\mathbf{w}^{ML} = (\mathbf{X}\mathbf{X}^\top)^{-1}\mathbf{X}\mathbf{y}$$

$$\mathbf{w}^* = (\mathbf{X}\mathbf{X}^\top + \lambda\mathbf{I})^{-1}\mathbf{X}\mathbf{y}$$



北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY

学习关键点

▶ 模型选择

- ▶ 拟合能力强的模型一般复杂度会比较高，容易过拟合。
- ▶ 如果限制模型复杂度，降低拟合能力，可能会欠拟合。

▶ 偏差与方差分解

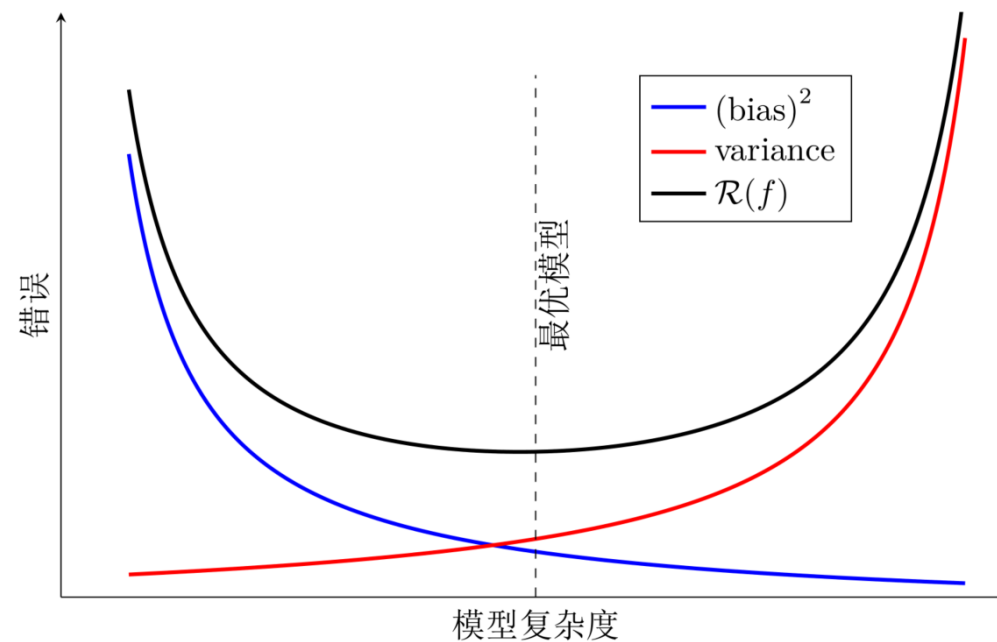
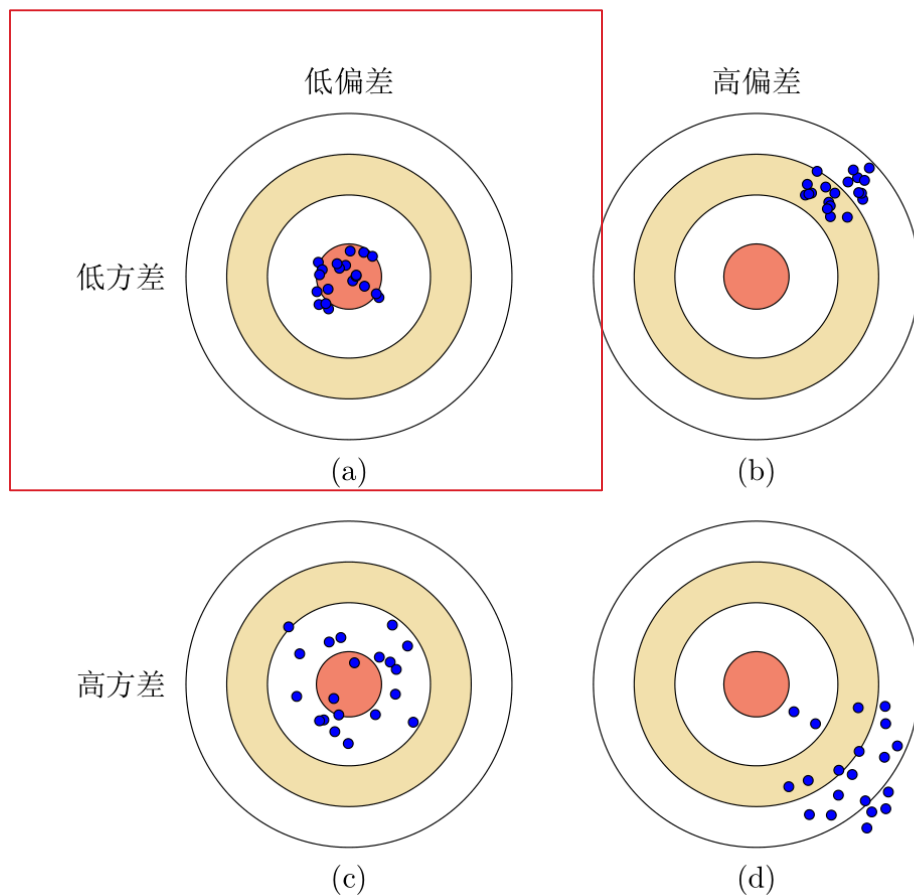
▶ 期望错误 $R(f) = E_{(x,y) \sim p_r(x,y)} [(y - f(x))^2]$

▶ 可以分解为 $R(f) = bias^2 + variance + \varepsilon$

$$\mathbb{E}_{\mathbf{x}} \left[\left(\mathbb{E}_{\mathcal{D}} [f_{\mathcal{D}}(\mathbf{x})] - f^*(\mathbf{x}) \right)^2 \right]$$

$$\mathbb{E}_{(\mathbf{x},y) \sim p_r(\mathbf{x},y)} \left[(y - f^*(\mathbf{x}))^2 \right]$$

$$\mathbb{E}_{\mathbf{x}} \left[\mathbb{E}_{\mathcal{D}} \left[(f_{\mathcal{D}}(\mathbf{x}) - \mathbb{E}_{\mathcal{D}} [f_{\mathcal{D}}(\mathbf{x})])^2 \right] \right]$$



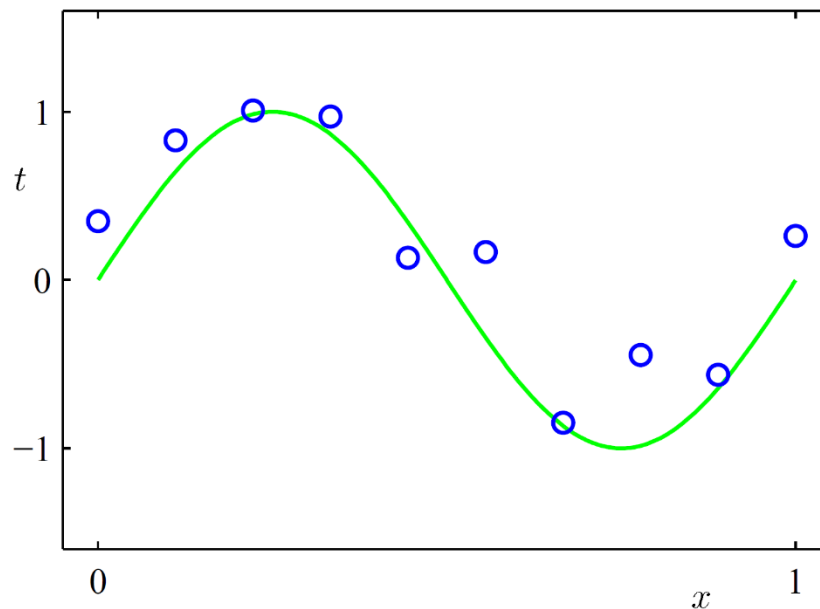
集成模型：有效的降低方差的方法



北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY

多项式回归

一个例子: Polynomial Curve Fitting



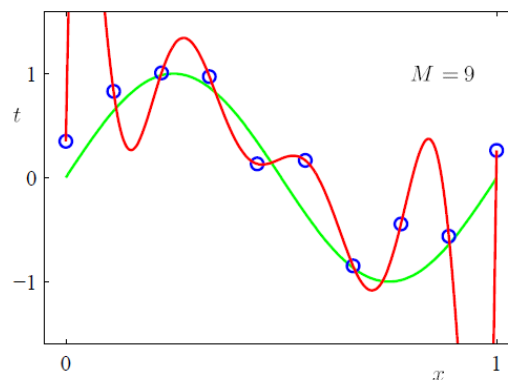
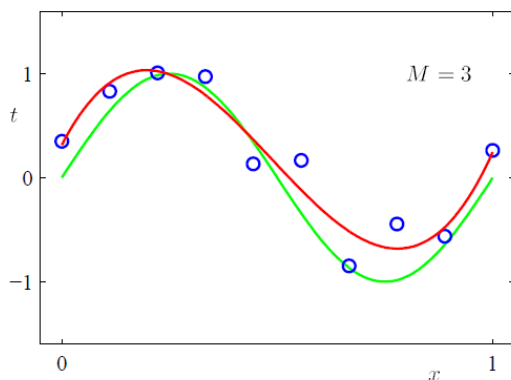
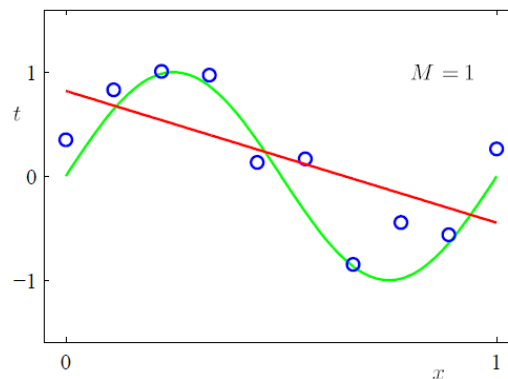
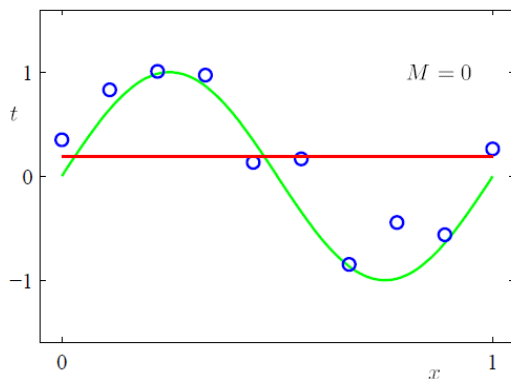
模型

$$y(x, \mathbf{w}) = w_0 + w_1x + w_2x^2 + \dots + w_Mx^M$$

损失函数

$$E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \{y(x_n, \mathbf{w}) - t_n\}^2$$

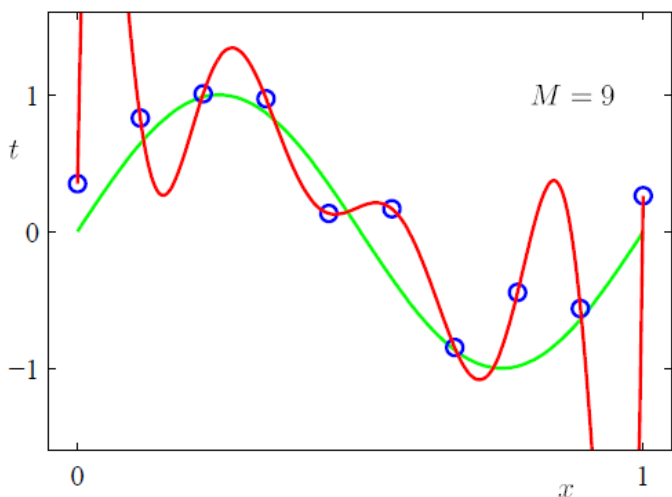
Which Degree of Polynomial?



A model selection problem

$M = 9 \rightarrow E(w) = 0$: This is **overfitting**

Controlling Overfitting: Regularization



	$M = 0$	$M = 1$	$M = 3$	$M = 9$
w_0^*	0.19	0.82	0.31	0.35
w_1^*		-1.27	7.99	232.37
w_2^*			-25.43	-5321.83
w_3^*			17.37	48568.31
w_4^*				-231639.30
w_5^*				640042.26
w_6^*				-1061800.52
w_7^*				1042400.18
w_8^*				-557682.99
w_9^*				125201.43

As order of polynomial M increases, so do coefficient magnitudes!

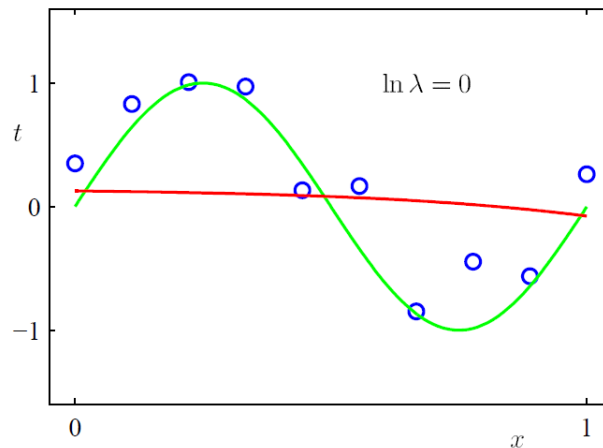
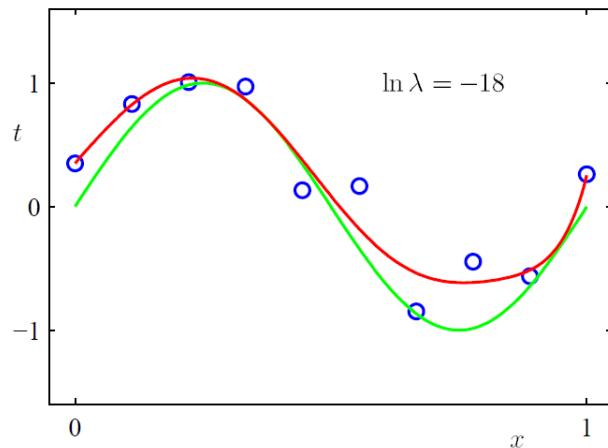
$$\tilde{E}(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \{y(x_n, \mathbf{w}) - t_n\}^2 + \frac{\lambda}{2} \|\mathbf{w}\|^2$$

对大的系数进行惩罚

Controlling Overfitting: Regularization

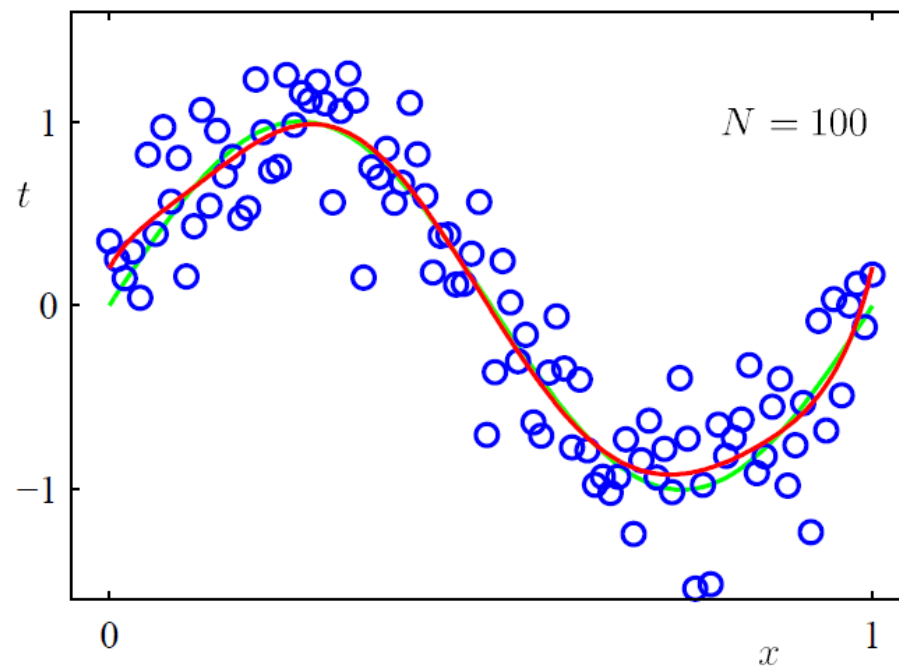
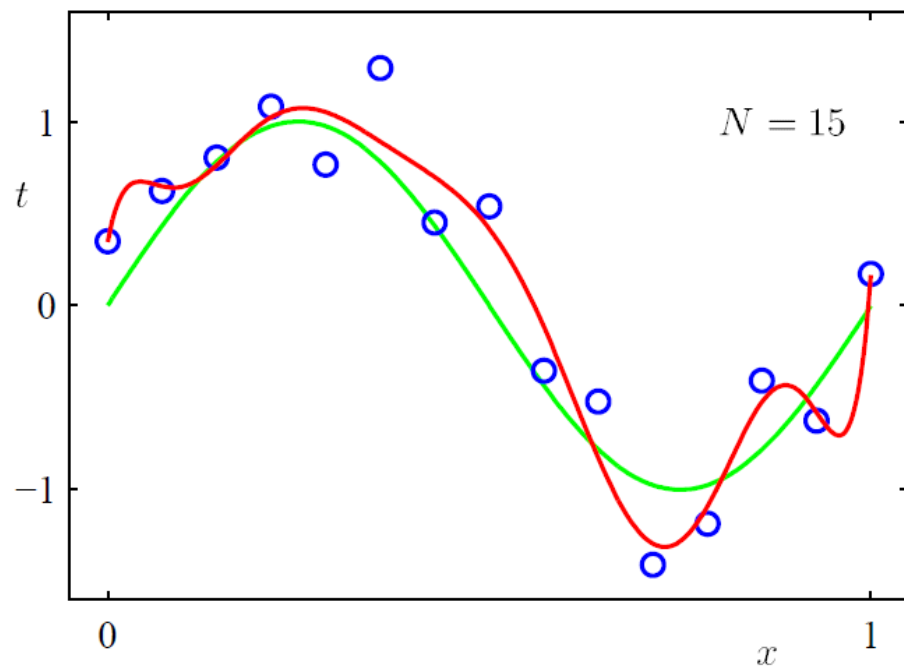


$$\tilde{E}(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \{y(x_n, \mathbf{w}) - t_n\}^2 + \frac{\lambda}{2} \|\mathbf{w}\|^2$$



	$\ln \lambda = -\infty$	$\ln \lambda = -18$	$\ln \lambda = 0$
w_0^*	0.35	0.35	0.13
w_1^*	232.37	4.74	-0.05
w_2^*	-5321.83	-0.77	-0.06
w_3^*	48568.31	-31.97	-0.05
w_4^*	-231639.30	-3.89	-0.03
w_5^*	640042.26	55.28	-0.02
w_6^*	-1061800.52	41.32	-0.01
w_7^*	1042400.18	-45.95	-0.00
w_8^*	-557682.99	-91.53	0.00
w_9^*	125201.43	72.68	0.01

Controlling Overfitting: Dataset size



德以明理 学以精工

谢谢



智能感知

日期：2025年8月



目录

CONTENTS

- 1 绪论
- 2 机器学习基础
- 3 线性模型
- 4 前馈神经网络
- 5 卷积神经网络

- ▶ 分类问题示例
- ▶ 线性分类模型
 - ▶ Logistic Regression
 - ▶ Softmax Regression
 - ▶ Perceptron
 - ▶ SVM



分类示例

示例：图像分类



▶ 数据集：CIFAR-10

▶ 60000张32x32色彩图像，共10类

▶ 每类6000张图像

airplane



automobile



bird



cat



deer



dog



frog



horse



ship



truck

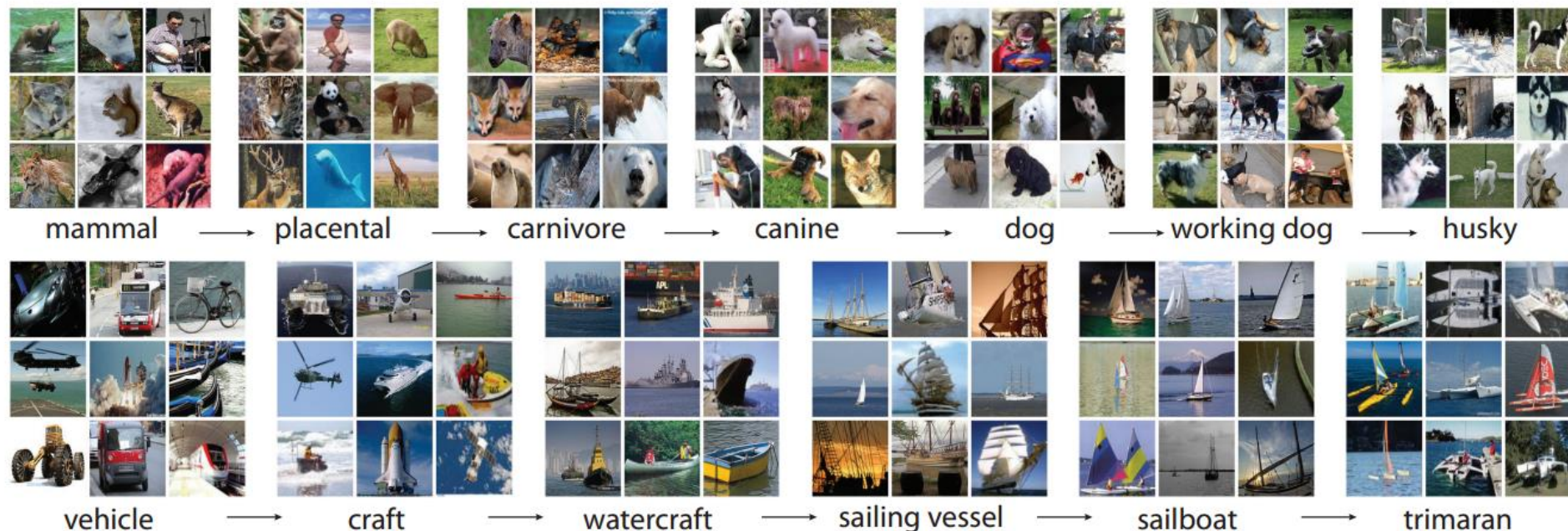


示例：图像分类



► 数据集：ImageNet

► 14,197,122 images, 21841 synsets



示例：图像分类

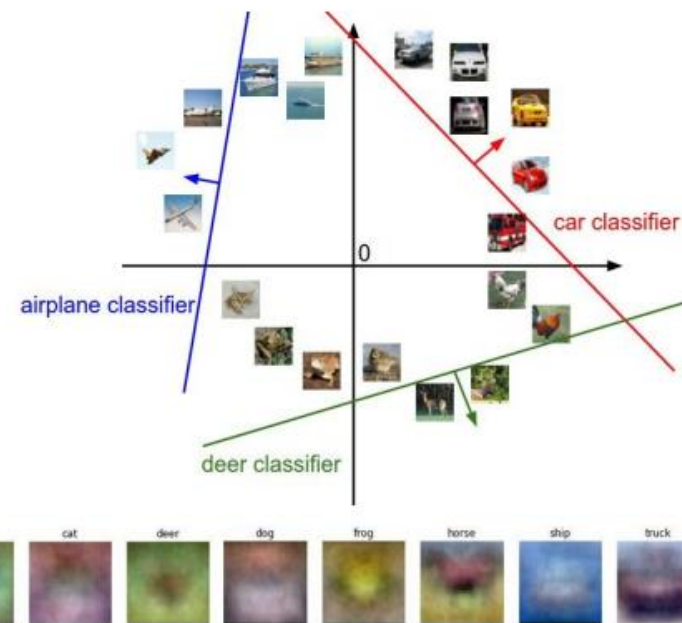
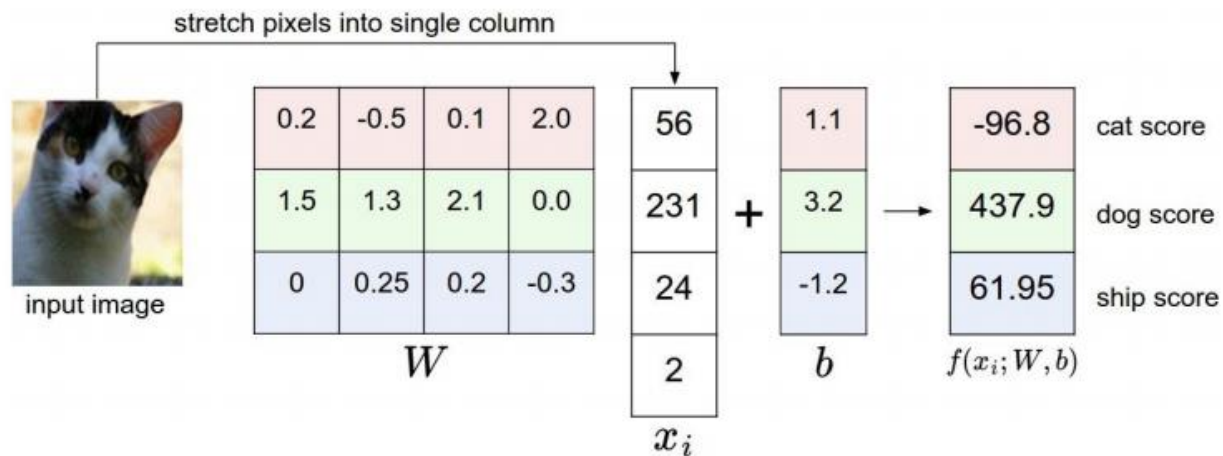


[32x32x3]

array of numbers 0...1
(3072 numbers total)

image parameters
 $f(\mathbf{x}, \mathbf{W})$

10 numbers, indicating
class scores



示例：图像分类、目标检测、实例分割



北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY

Classification



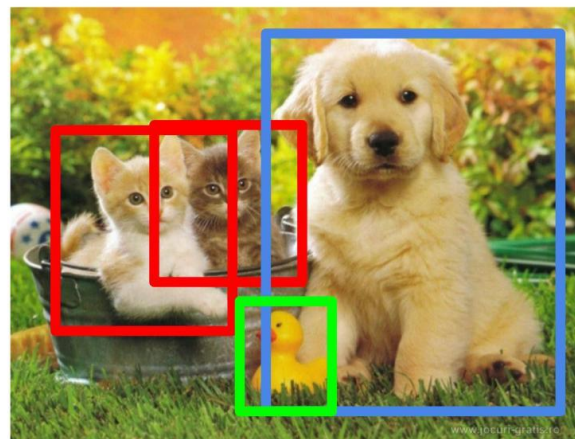
CAT

Classification + Localization



CAT

Object Detection



CAT, DOG, DUCK

Instance Segmentation



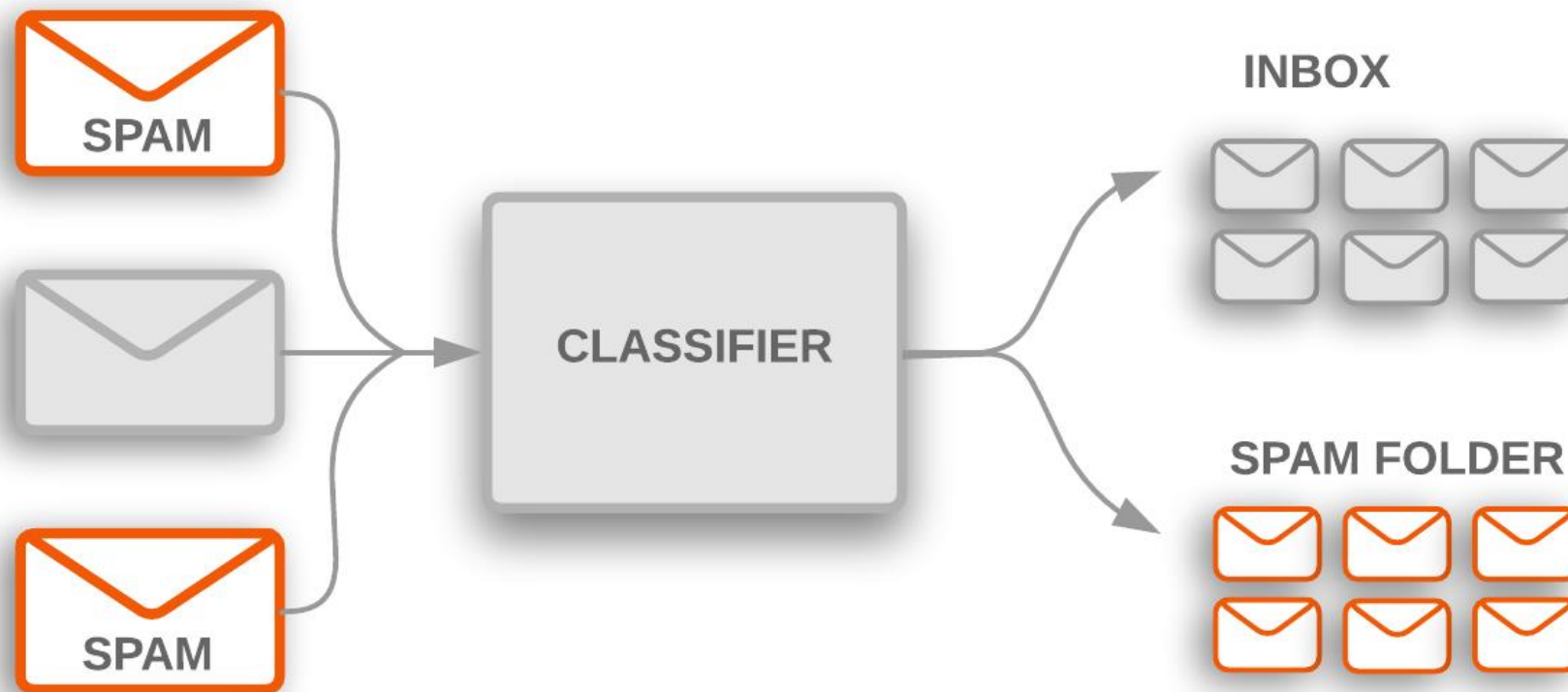
CAT, DOG, DUCK

Single object

Multiple objects

<https://medium.com/zylapp/review-of-deep-learning-algorithms-for-object-detection-c1f3d437b852>

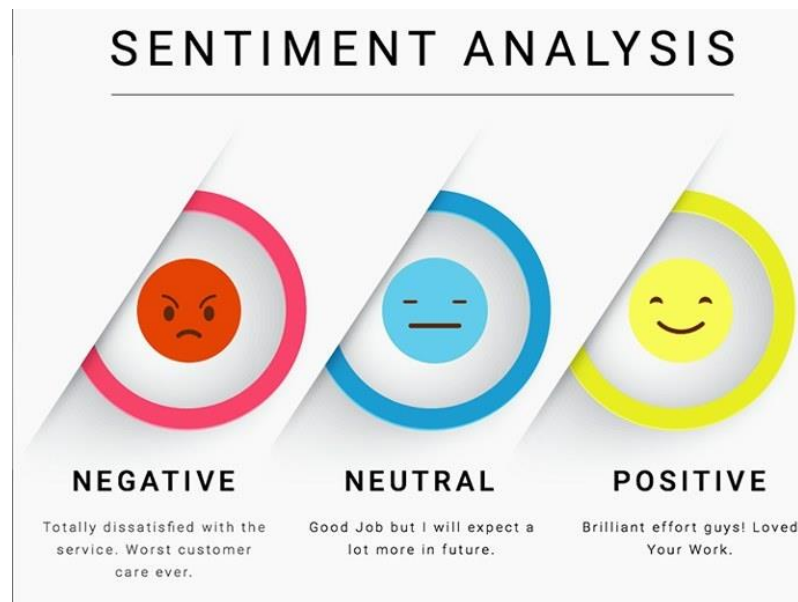
示例：垃圾邮件过滤



示例：文档归类



<https://towardsdatascience.com/automated-text-classification-using-machine-learning-3df4f4f9570b>



Review (X)

"This movie is fantastic! I really like it because it is so good!"

"Not to my taste, will skip and watch another movie"

"This movie really sucks! Can I get my money back please?"

Rating (Y)



- ▶ 将样本 x 从文本形式转为向量形式
- ▶ 词袋模型 (Bag-of-Words, BoW) 模型

the dog is on the table

0	0	1	1	0	1	1	1
are	cat	dog	is	now	on	table	the

比如两个文本“我 喜欢 读书”和“我 讨厌 读书”中共有“我”、“喜欢”、“讨厌”、“读书”四个词，它们的BoW表示分别为

$$\mathbf{v}_1 = [1 \ 1 \ 0 \ 1]^T,$$

$$\mathbf{v}_2 = [1 \ 0 \ 1 \ 1]^T.$$

示例：文本情感分类



根据文本内容来判断文本的相应类别

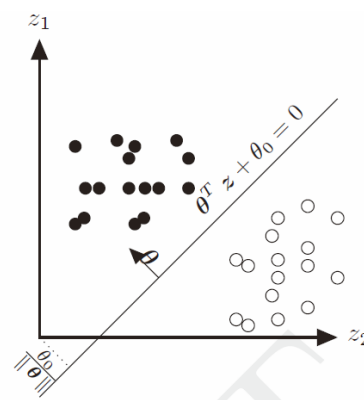
D_1 : “我喜欢读书”

D_2 : “我讨厌读书”

	我	喜欢	讨厌	读书
D_1	1	1	0	1
D_2	1	0	1	1

+

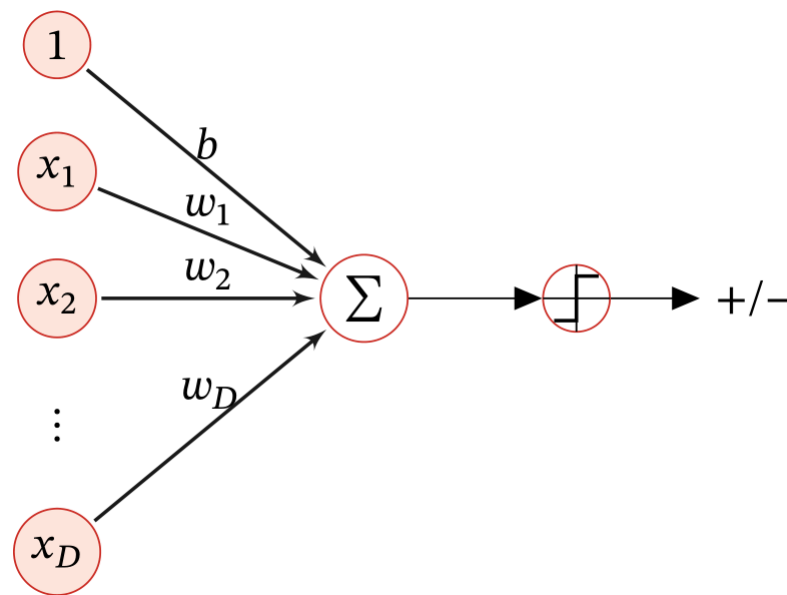
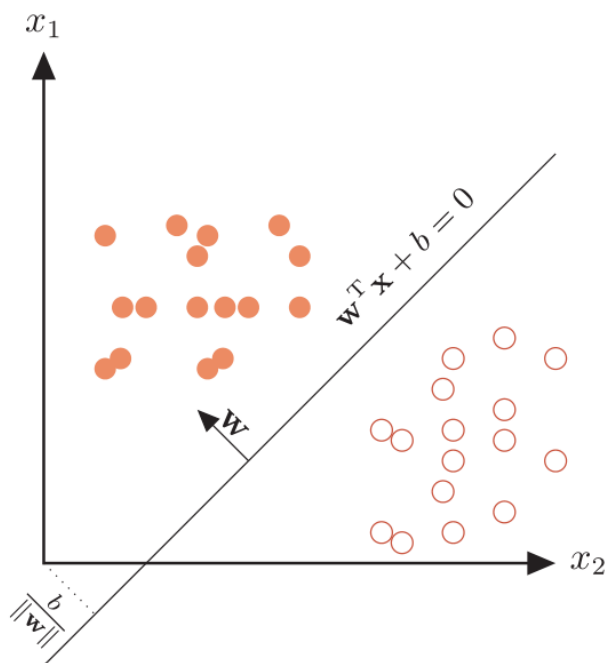
-





北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY

线性分类模型



- ▶ Logistic 回归
- ▶ Softmax 回归
- ▶ 感知器
- ▶ 支持向量机



北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY

学习准则 交叉损失函数

► 在信息论中，熵用来衡量一个随机事件的不确定性。

► 自信息 (Self Information) $I(x) = -\log(p(x))$

► 熵

$$\begin{aligned} H(X) &= \mathbb{E}_X[I(x)] \\ &= \mathbb{E}_X[-\log p(x)] \\ &= - \sum_{x \in \mathcal{X}} p(x) \log p(x) \end{aligned}$$

► 熵越高，则随机变量的信息越多；

► 熵越低，则随机变量的信息越少。

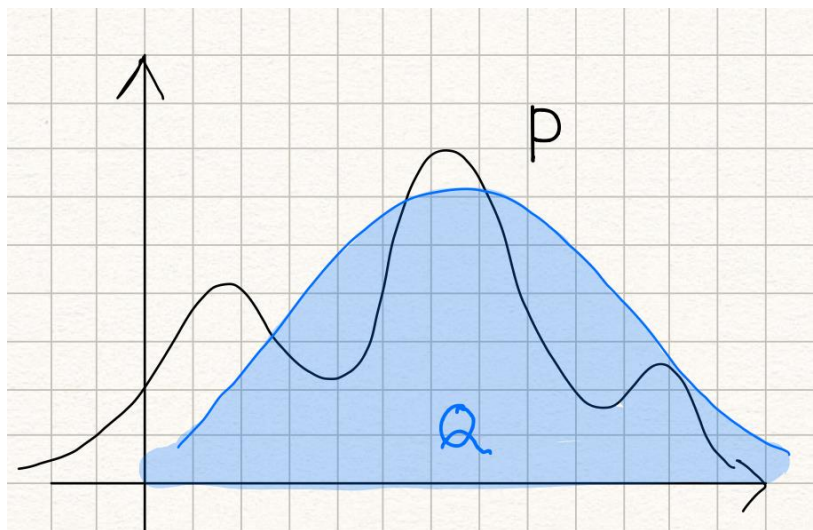
► 在对分布 $q(y)$ 的符号进行编码时，熵 $I(q)$ 也是理论上最优的平均编码长度，这种编码方式称为熵编码 (Entropy Encoding)

- ▶ 交叉熵是按照概率分布 q 的最优编码对真实分布为 p 的信息进行编码的长度。

$$\begin{aligned} H(p, q) &= \mathbb{E}_p[-\log q(x)] \\ &= -\sum_x p(x) \log q(x) \end{aligned}$$

- ▶ 在给定 q 的情况下，如果 p 和 q 越接近，交叉熵越小；
- ▶ 如果 p 和 q 越远，交叉熵就越大。

- ▶ KL散度是用概率分布 q 来近似 p 时所造成的信息损失量。
- ▶ KL散度是按照概率分布 q 的最优编码对真实分布为 p 的信息进行编码，其平均编码长度（即交叉熵） $H(p, q)$ 和 p 的最优平均编码长度（即熵） $H(p)$ 之间的差异。



$$\begin{aligned} \text{KL}(p, q) &= H(p, q) - H(p) \\ &= \sum_x p(x) \log \frac{p(x)}{q(x)} \end{aligned}$$

▶ 模型预测条件概率 $p_{\theta}(y|\mathbf{x})$

$$p_{\theta}(y = 1|\mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x})$$

▶ 真实条件概率 $p_r(y|\mathbf{x})$

▶ 对于一个样本 $(\mathbf{x}, y^*)$, 其真实条件概率为

$$\begin{aligned} p_r(y = 1|\mathbf{x}) &= y^* \\ p_r(y = 0|\mathbf{x}) &= 1 - y^* \end{aligned}$$

如何衡量两个条件分布的差异?

► KL散度和交叉熵在机器学习定义学习准则

$$D_{KL}(p_r(y|x)||p_\theta(y|x)) = \sum_{y=0}^1 p_r(y|x) \log \frac{p_r(y|x)}{p_\theta(y|x)} \quad \text{KL散度}$$

$$\propto - \sum_{y=0}^1 p_r(y|x) \log p_\theta(y|x) \quad \text{交叉熵损失}$$

y^* 为x的真实标签

$$= -I(y^* = 1) \log p_\theta(y = 1|x) - I(y^* = 0) \log p_\theta(y = 0|x)$$

$$= -y^* \log p_\theta(y = 1|x) - (1 - y^*) \log p_\theta(y = 0|x)$$

$$= -\log p_\theta(y^*|x) \quad \text{负对数似然}$$



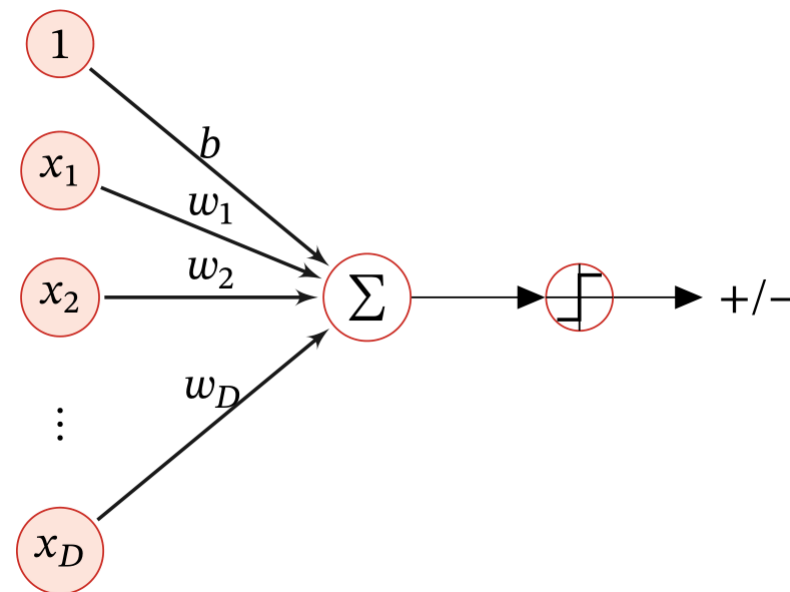
北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY

对率回归

Logistic Regression

$$g(f(\mathbf{x}; \mathbf{w})) = \begin{cases} 1 & \text{if } f(\mathbf{x}; \mathbf{w}) > 0 \\ 0 & \text{if } f(\mathbf{x}; \mathbf{w}) < 0 \end{cases}$$

损失函数?



▶ 将分类问题看作条件概率估计问题

▶ 引入非线性函数 g 来预测类别标签的条件概率 $p(y = c|x)$ 。

▶ 以二分类为例，

$$p(y = 1|\mathbf{x}) = g(f(\mathbf{x}; \mathbf{w}))$$

▶ 函数 f : 线性函数

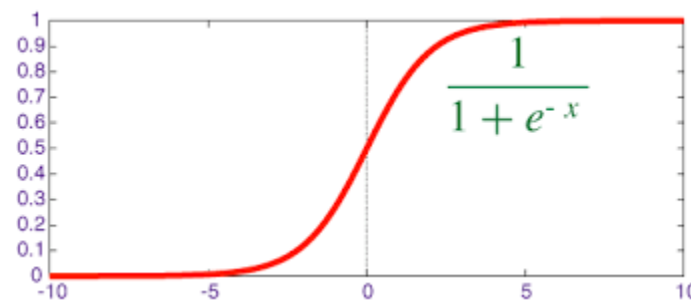
▶ 函数 g : 把线性函数的值域从实数区间“挤压”到了 $(0,1)$ 之间，可以用来表示概率。

如何构造函数 g ?

► Logistic函数

除了logistic, 还有什么函数 $f: \mathbb{R} \rightarrow (0,1)$?

$$\sigma(x) = \frac{1}{1 + \exp(-x)}$$



► Logistic回归

$$p(y = 1|x) = \sigma(\omega^T x)$$

$$\triangleq \frac{1}{1 + \exp(-\omega^T x)}$$

$$p(y = 0|x) = 1 - p(y = 1|x)$$

$$\triangleq \frac{\exp(-\omega^T x)}{1 + \exp(-\omega^T x)}$$

$$\omega^T x = \log \frac{p(y = 1|x)}{1 - p(y = 1|x)}$$

对数几率

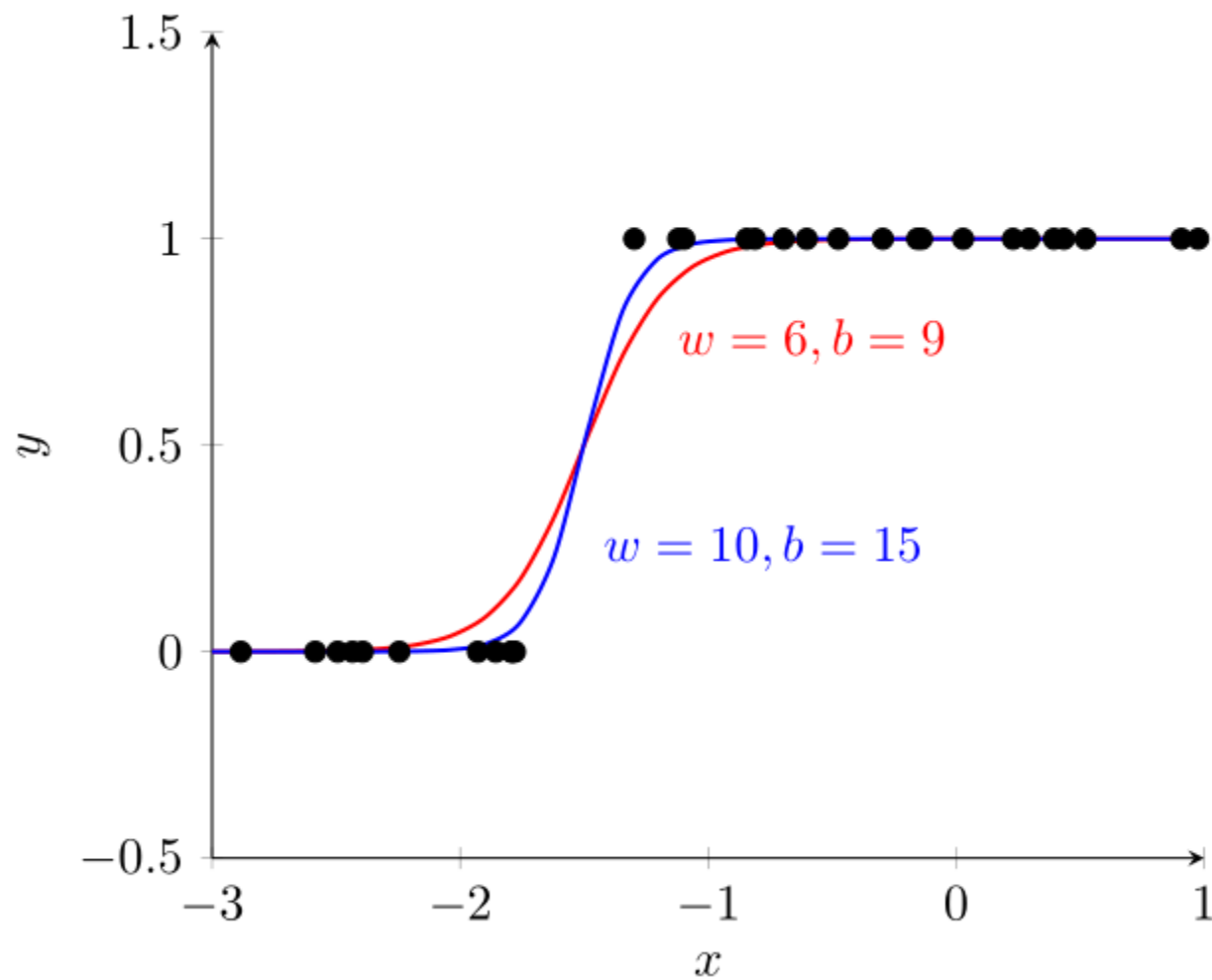
► 交叉熵损失函数，模型在训练集的风险函数为

$$\mathcal{R}(\mathbf{w}) = -\frac{1}{N} \sum_{i=1}^N \left(y^{(i)} \log \left(\sigma(\mathbf{w}^T \mathbf{x}^{(i)}) \right) + (1 - y^{(i)}) \log \left(1 - \sigma(\mathbf{w}^T \mathbf{x}^{(i)}) \right) \right).$$

► 梯度为

$$\frac{\partial \mathcal{R}(\mathbf{w})}{\partial \mathbf{w}} = \frac{1}{N} \sum_{i=1}^N \left(\mathbf{x}^{(i)} \cdot \left(\sigma(\mathbf{w}^T \mathbf{x}^{(i)}) - y^{(i)} \right) \right)$$

$$\begin{aligned}\frac{\partial \mathcal{R}(\mathbf{w})}{\partial \mathbf{w}} &= -\frac{1}{N} \sum_{n=1}^N \left(y^{(n)} \frac{\hat{y}^{(n)}(1 - \hat{y}^{(n)})}{\hat{y}^{(n)}} \mathbf{x}^{(n)} - (1 - y^{(n)}) \frac{\hat{y}^{(n)}(1 - \hat{y}^{(n)})}{1 - \hat{y}^{(n)}} \mathbf{x}^{(n)} \right) \\ &= -\frac{1}{N} \sum_{n=1}^N \left(y^{(n)}(1 - \hat{y}^{(n)}) \mathbf{x}^{(n)} - (1 - y^{(n)}) \hat{y}^{(n)} \mathbf{x}^{(n)} \right) \\ &= -\frac{1}{N} \sum_{n=1}^N \mathbf{x}^{(n)} (y^{(n)} - \hat{y}^{(n)}).\end{aligned}$$

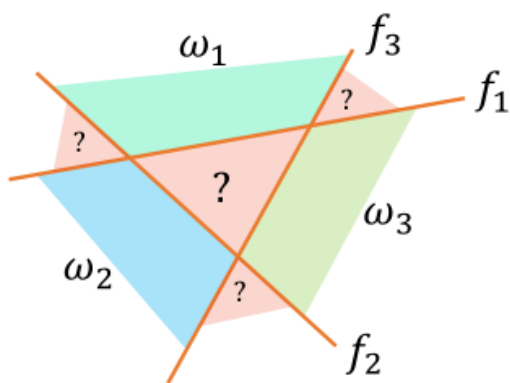




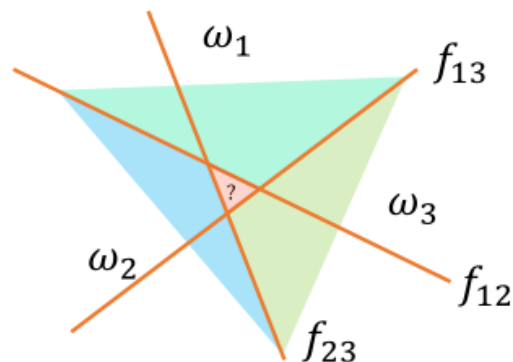
多分类问题

Softmax

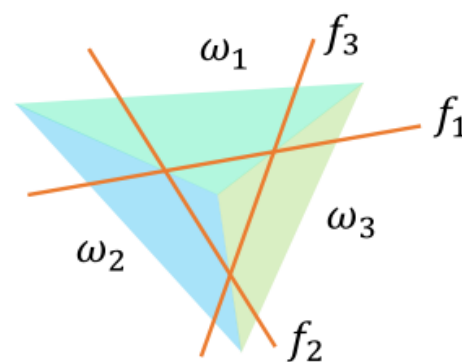
多分类 (Multi-class Classification)



(a) “一对其余” 方式



(b) “一对一” 方式



(c) “argmax” 方式

“argmax” 方式：这是一种改进的“一对其余”方式，共需要 C 个判别函数

$$f_c(\mathbf{x}; \mathbf{w}_c) = \mathbf{w}_c^T \mathbf{x} + b_c, \quad c = [1, \dots, C] \quad (3.10)$$

如果存在类别 c , 对于所有的其他类别 $\tilde{c} (\tilde{c} \neq c)$ 都满足 $f_c(\mathbf{x}; \mathbf{w}_c) > f_{\tilde{c}}(\mathbf{x}, \mathbf{w}_{\tilde{c}})$, 那么 $\mathbf{x}$ 属于类别 c 。即

$$y = \arg \max_{c=1}^C f_c(\mathbf{x}; \mathbf{w}_c). \quad (3.11)$$

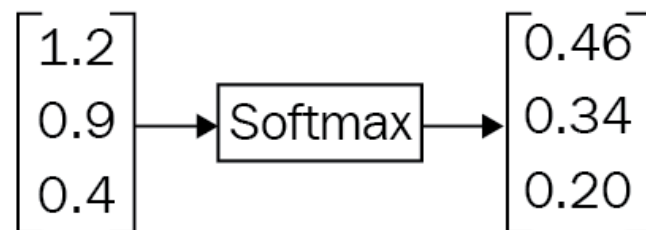
► 多分类问题

► $f_c(x; \omega_c) = \omega_c x + b$

$$y = \arg \max_{c=1}^C f_c(\mathbf{x}; \mathbf{w}_c)$$

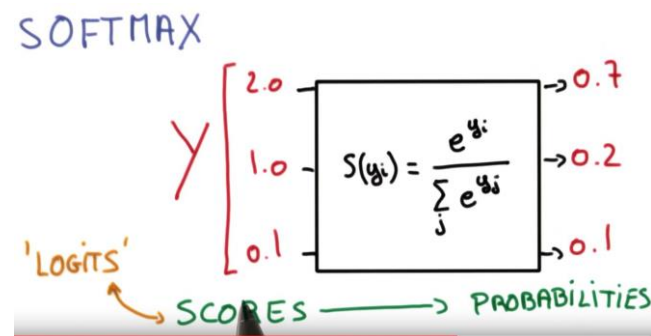
► Softmax函数

$$\text{softmax}(x_k) = \frac{\exp(x_k)}{\sum_{i=1}^K \exp(x_i)}$$



► 利用softmax函数，目标类别 $y = c$ 的条件概率为：

$$\begin{aligned} P(y = c|\mathbf{x}) &= \text{softmax}(\mathbf{w}_c^T \mathbf{x}) \\ &= \frac{\exp(\mathbf{w}_c^T \mathbf{x})}{\sum_{i=1}^C \exp(\mathbf{w}_i^T \mathbf{x})}. \end{aligned}$$



► 模型：Softmax 回归

► 学习准则：交叉熵

$$\mathcal{R}(W) = -\frac{1}{N} \sum_{n=1}^N (\mathbf{y}^{(n)})^T \log \hat{\mathbf{y}}^{(n)}$$

► 优化：梯度下降

$$\frac{\partial \mathcal{R}(W)}{\partial W} = -\frac{1}{N} \sum_{n=1}^N \mathbf{x}^{(n)} \left(\mathbf{y}^{(n)} - \hat{\mathbf{y}}^{(n)} \right)^T$$

► 负对数似然损失函数

$$\mathcal{L}(\mathbf{y}, f(\mathbf{x}, \theta)) = - \sum_{c=1}^C y_c \log f_c(\mathbf{x}, \theta)$$

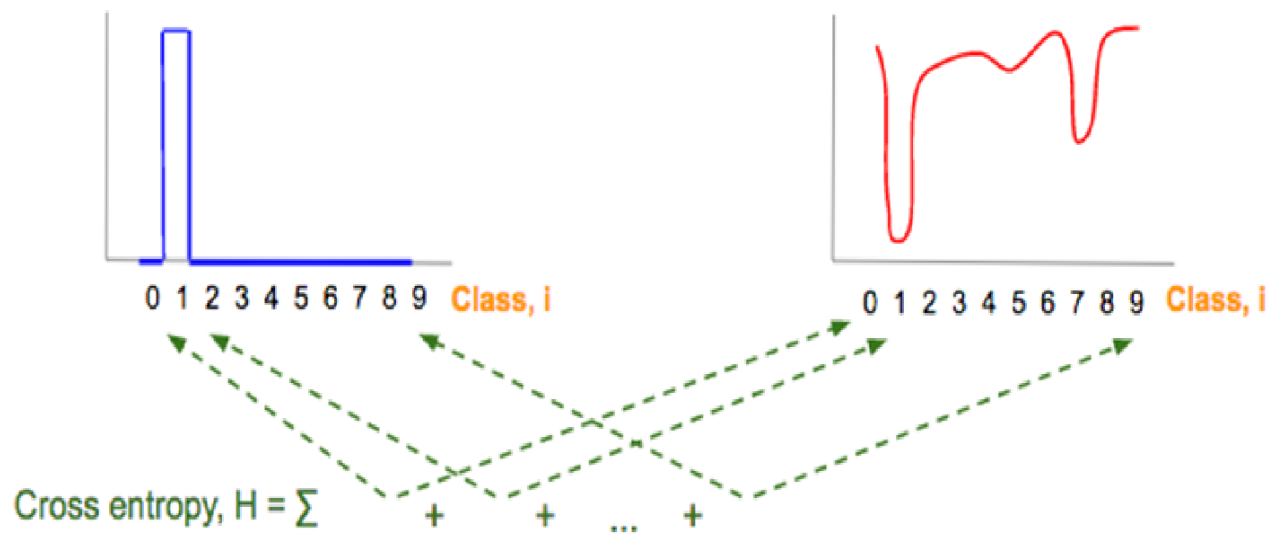
► 对于一个三类分类问题，类别为[0,0,1]，预测类别概率为[0.3,0.3,0.4]，则

$$\begin{aligned}\mathcal{L}(\theta) &= -(0 \times \log(0.3) + 0 \times \log(0.3) + 1 \times \log(0.4)) \\ &= -\log(0.4).\end{aligned}$$

$$-\sum_{y=1}^c p_r(y|x) \log p_{\theta}(y|x)$$

真实概率 $p_r(y|x)$

预测概率的负对数 $-\log p_{\theta}(y|x)$





北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY

感知器

Psychological Review
Vol. 65, No. 6, 1958

THE PERCEPTRON: A PROBABILISTIC MODEL FOR INFORMATION STORAGE AND ORGANIZATION IN THE BRAIN¹

F. ROSENBLATT

Cornell Aeronautical Laboratory

HAVING told you about the giant digital computer known as I.B.M. 704 and how it has been taught to play a fairly creditable game of chess, we'd like to tell you about an even more remarkable machine, the perceptron, which, as its name implies, is capable of what amounts to original thought. The first perceptron has yet to be built,

The New Yorker, December 6, 1958 P. 44



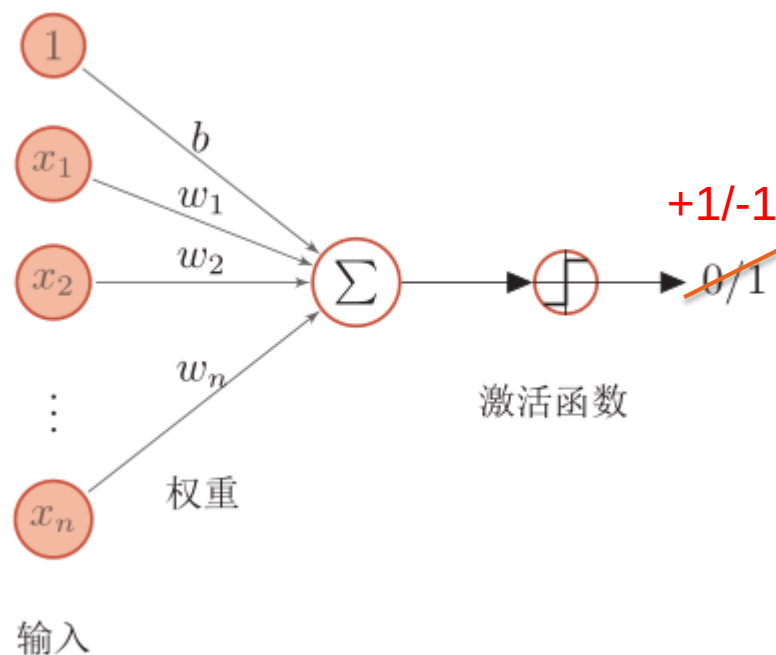
The IBM 704 computer

- ▶ 模拟生物神经元行为的机器，有与生物神经元相对应的部件，如权重（突触）、偏置（阈值）及激活函数（细胞体），输出为+1或-1。

- ▶
$$\hat{y} = \text{sgn}(\omega^T x)$$
$$\hat{y} = \begin{cases} +1 & \text{当 } \mathbf{w}^T \mathbf{x} > 0 \\ -1 & \text{当 } \mathbf{w}^T \mathbf{x} \leq 0 \end{cases},$$

- 学习目标寻找权重 $\mathbf{w}$ ，满足：

$$y\mathbf{w}^T \mathbf{x} > 0$$



▶ 学习算法

- ▶ 一种错误驱动的在线学习算法：
- ▶ 先初始化一个权重向量 $\mathbf{w} \leftarrow \mathbf{0}$ （通常是全零向量）；
- ▶ 每次分错一个样本 $(\mathbf{x}, y)$ 时，即

$$y\mathbf{w}^T \mathbf{x} < 0$$

- ▶ 用这个样本来更新权重

$$\mathbf{w} \leftarrow \mathbf{w} + y\mathbf{x}$$

- ▶ 根据感知器的学习策略，可以反推出感知器的损失函数为

$$\mathcal{L}(\mathbf{w}; \mathbf{x}, y) = \max(0, -y\mathbf{w}^T \mathbf{x})$$

算法 3.1 两类感知器的参数学习算法

输入: 训练集 $\mathcal{D} = \{(\mathbf{x}^{(n)}, y^{(n)})\}_{n=1}^N$, 最大迭代次数 T

1 初始化: $\mathbf{w}_0 \leftarrow 0, k \leftarrow 0, t \leftarrow 0$;

2 **repeat**

3 对训练集 $\mathcal{D}$ 中的样本随机排序;

4 **for** $n = 1 \cdots N$ **do**

5 选取一个样本 $(\mathbf{x}^{(n)}, y^{(n)})$;

6 **if** $\mathbf{w}_k^\top (y^{(n)} \mathbf{x}^{(n)}) \leq 0$ **then**

7 $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k + y^{(n)} \mathbf{x}^{(n)}$;

8 $k \leftarrow k + 1$;

9 **end**

10 $t \leftarrow t + 1$;

11 **if** $t = T$ **then** break;

12 **end**

13 **until** $t = T$;

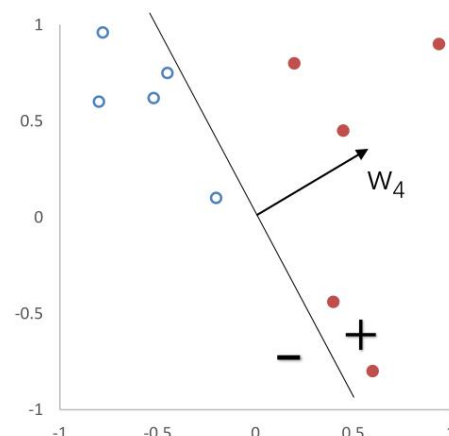
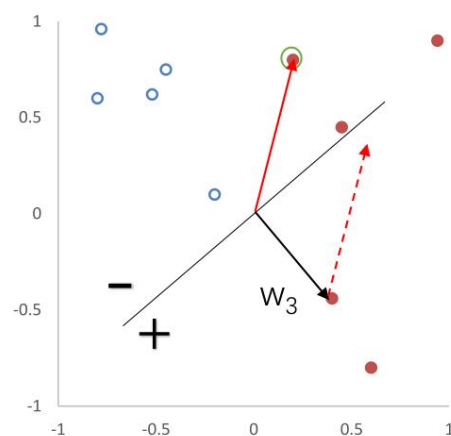
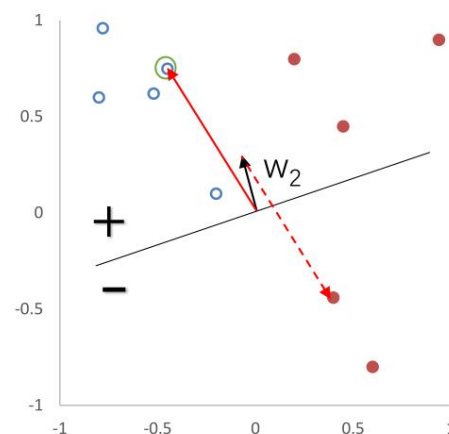
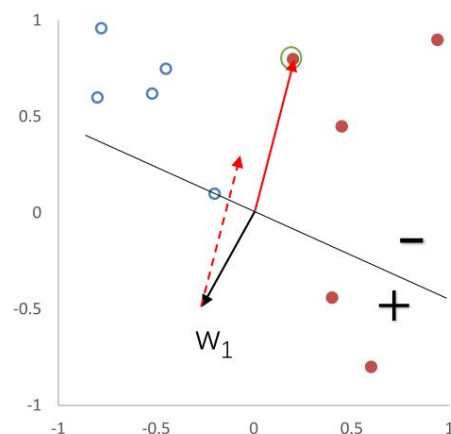
输出: $\mathbf{w}_k$

表示分错

对比Logistic回归的更新方式:

$$\mathbf{w}_{t+1} \leftarrow \mathbf{w}_t + \alpha \frac{1}{N} \sum_{n=1}^N \mathbf{x}^{(n)} (y^{(n)} - \hat{y}_{\mathbf{w}_t}^{(n)})$$

// 达到最大迭代次数



当数据集是两类线性可分时,对于训练集 $\mathcal{D} = \{(\mathbf{x}^{(n)}, y^{(n)})\}_{n=1}^N$, 其中 $\mathbf{x}^{(n)}$ 为样本的增广特征向量, $y^{(n)} \in \{-1, 1\}$, 那么存在一个正的常数 $\gamma (\gamma > 0)$ 和权重向量 $\mathbf{w}^*$, 并且 $\|\mathbf{w}^*\| = 1$, 对所有 n 都满足 $(\mathbf{w}^*)^\top (y^{(n)} \mathbf{x}^{(n)}) \geq \gamma$. 我们可以证明如下定理.

定理 3.1 – 感知器收敛性: 给定训练集 $\mathcal{D} = \{(\mathbf{x}^{(n)}, y^{(n)})\}_{n=1}^N$, 令 R 是训练集中最大的特征向量的模, 即

$$R = \max_n \|\mathbf{x}^{(n)}\|.$$

如果训练集 $\mathcal{D}$ 线性可分, 两类感知器的参数学习算法 3.1 的权重更新次数不超过 $\frac{R^2}{\gamma^2}$.

- ▶ 感知器的权重向量的更新方式为

$$\mathbf{w}_k = \mathbf{w}_{k-1} + y^{(k)} \mathbf{x}^{(k)}$$

- ▶ 在第 K 次更新时感知器的权重向量为

$$\mathbf{w}_K = \sum_{k=1}^K y^{(k)} \mathbf{x}^{(k)}$$

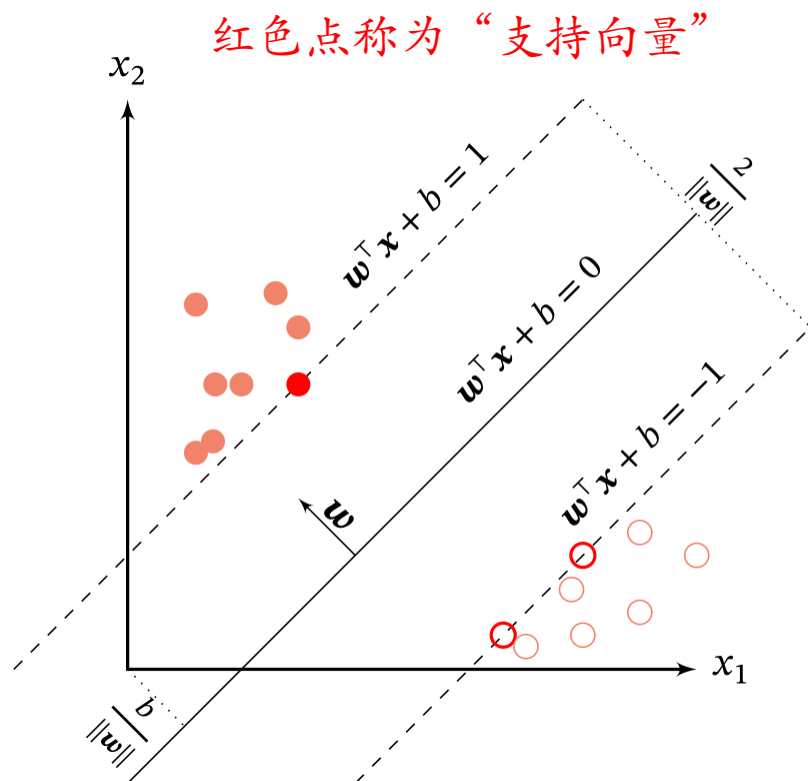
- ▶ $\|\mathbf{w}_K\|^2$ 的上界

- ▶ $\|\mathbf{w}_K\|^2$ 的下界



北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY

支持向量机



支持向量机的目标是寻找一个超平面 $(\mathbf{w}^*, b^*)$ 使得 γ 最大, 即

$$\begin{aligned} \max_{\mathbf{w}, b} \quad & \gamma \\ \text{s.t.} \quad & \frac{y^{(n)}(\mathbf{w}^\top \mathbf{x}^{(n)} + b)}{\|\mathbf{w}\|} \geq \gamma, \forall n \in \{1, \dots, N\} \end{aligned}$$



$$\begin{aligned} \max_{\mathbf{w}, b} \quad & \frac{1}{\|\mathbf{w}\|^2} \\ \text{s.t.} \quad & y^{(n)}(\mathbf{w}^\top \mathbf{x}^{(n)} + b) \geq 1, \forall n \in \{1, \dots, N\} \end{aligned}$$

数据集 $\mathcal{D}$ 中每个样本 $\mathbf{x}^{(n)}$ 到分割超平面的距离为:

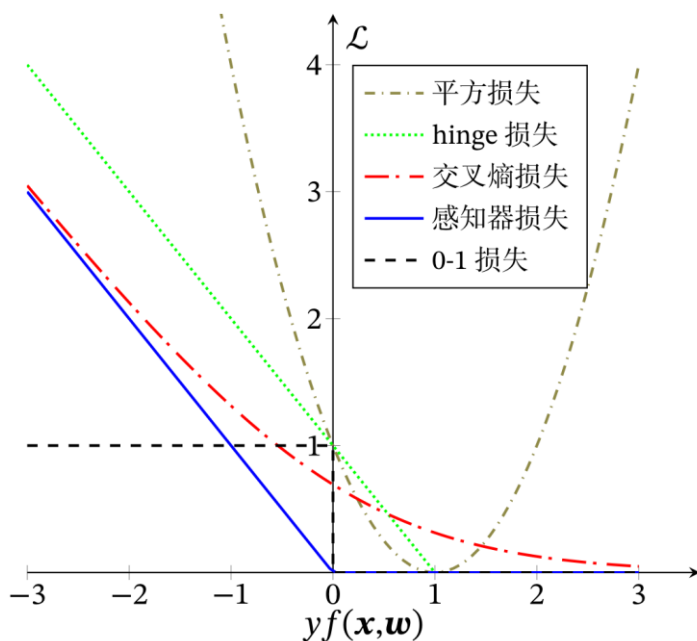
$$\gamma^{(n)} = \frac{\|\mathbf{w}^\top \mathbf{x}^{(n)} + b\|}{\|\mathbf{w}\|} = \frac{y^{(n)}(\mathbf{w}^\top \mathbf{x}^{(n)} + b)}{\|\mathbf{w}\|}.$$



小结

线性模型	激活函数	损失函数	优化方法
线性回归	-	$(y - \mathbf{w}^\top \mathbf{x})^2$	最小二乘、梯度下降
Logistic 回归	$\sigma(\mathbf{w}^\top \mathbf{x})$	$\mathbf{y} \log \sigma(\mathbf{w}^\top \mathbf{x})$	梯度下降
Softmax 回归	$\text{softmax}(\mathbf{W}^\top \mathbf{x})$	$\mathbf{y} \log \text{softmax}(\mathbf{W}^\top \mathbf{x})$	梯度下降
感知器	$\text{sgn}(\mathbf{w}^\top \mathbf{x})$	$\max(0, -y\mathbf{w}^\top \mathbf{x})$	随机梯度下降
支持向量机	$\text{sgn}(\mathbf{w}^\top \mathbf{x})$	$\max(0, 1 - y\mathbf{w}^\top \mathbf{x})$	二次规划、SMO 等

为了比较这些损失函数，我们统一定义类别标签 $y \in \{+1, -1\}$ ，并定义 $f(\mathbf{x}; \mathbf{w}) = \mathbf{w}^T \mathbf{x} + b$ 。这样对于样本 $(\mathbf{x}, y)$ ，若 $yf(\mathbf{x}; \mathbf{w}) > 0$ ，则分类正确，相反则分类错误。这样为了方便比较这些模型，我们可以将它们的损失函数都表述为定义在 $yf(\mathbf{x}; \mathbf{w})$ 上的函数。



$$\mathcal{L}_{LR} = \log(1 + \exp(-yf(\mathbf{x}; \mathbf{w})))$$

$$\mathcal{L}_{hinge} = \max(0, 1 - yf(\mathbf{x}; \mathbf{w}))$$

$$\mathcal{L}_p = \max(0, -yf(\mathbf{x}; \mathbf{w}))$$

$$\mathcal{L}_{squared} = (1 - yf(\mathbf{x}; \mathbf{w}))^2$$

德以明理 学以精工

谢谢



智能感知

日期：2025年8月



目录

CONTENTS

- 1 绪论
- 2 机器学习基础
- 3 线性模型
- 4 前馈神经网络
- 5 卷积神经网络

▶ 神经网络

- ▶ 神经元

- ▶ 网络结构

▶ 前馈神经网络

- ▶ 参数学习

- ▶ 计算图与自动微分

- ▶ 优化问题



北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY

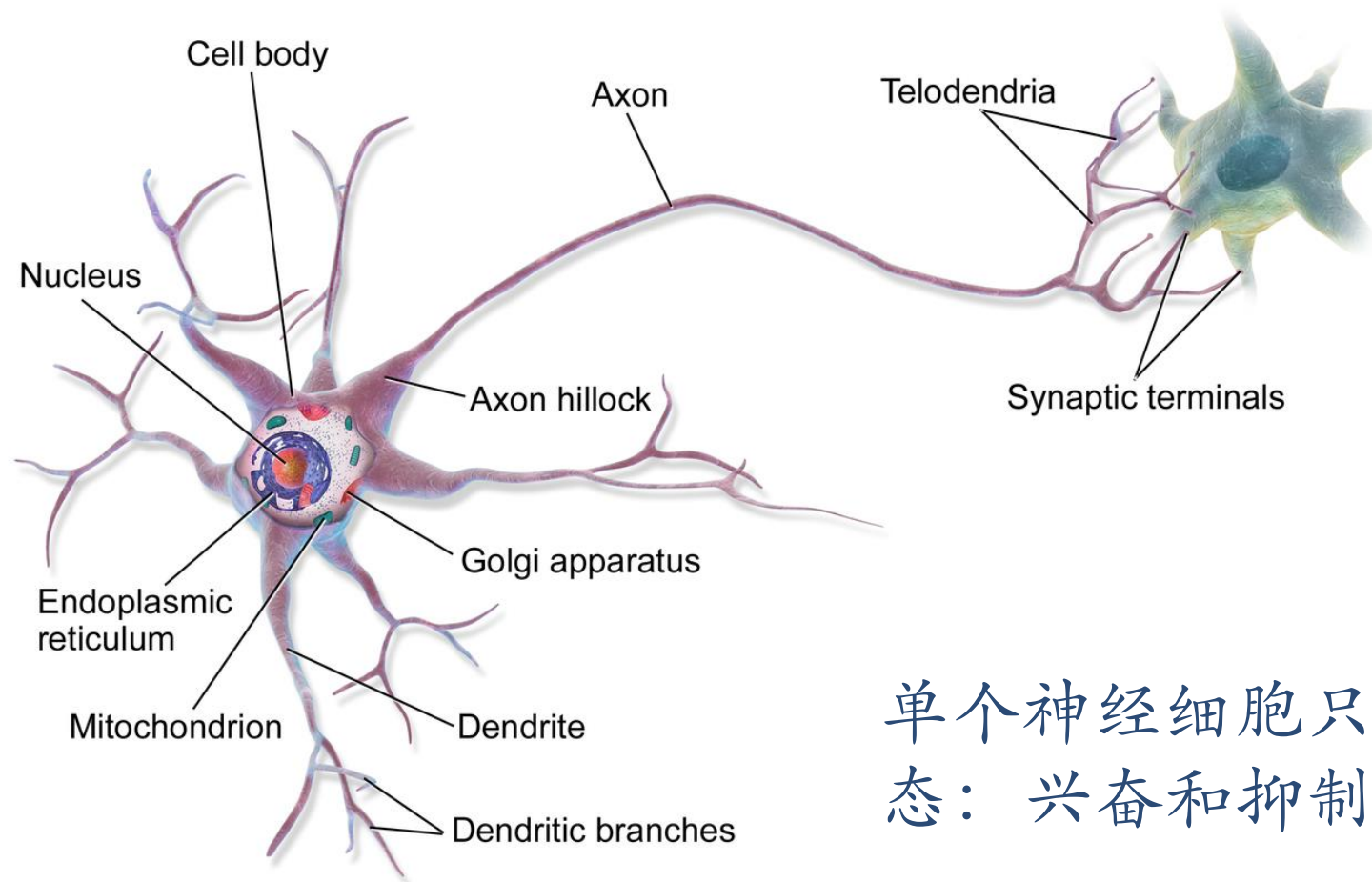
神经网络

- ▶ 神经网络最早是作为一种主要的连接主义模型。
- ▶ 20世纪80年代后期，最流行的一种连接主义模型是分布式并行处理（Parallel Distributed Processing, PDP）网络，其有3个主要特性：
 - ▶ 1) 信息表示是分布式的（非局部的）；
 - ▶ 2) 记忆和知识是存储在单元之间的连接上；
 - ▶ 3) 通过逐渐改变单元之间的连接强度来学习新的知识。
- ▶ 引入误差反向传播来改进其学习能力之后，神经网络也越来越多地应用在各种机器学习任务上。



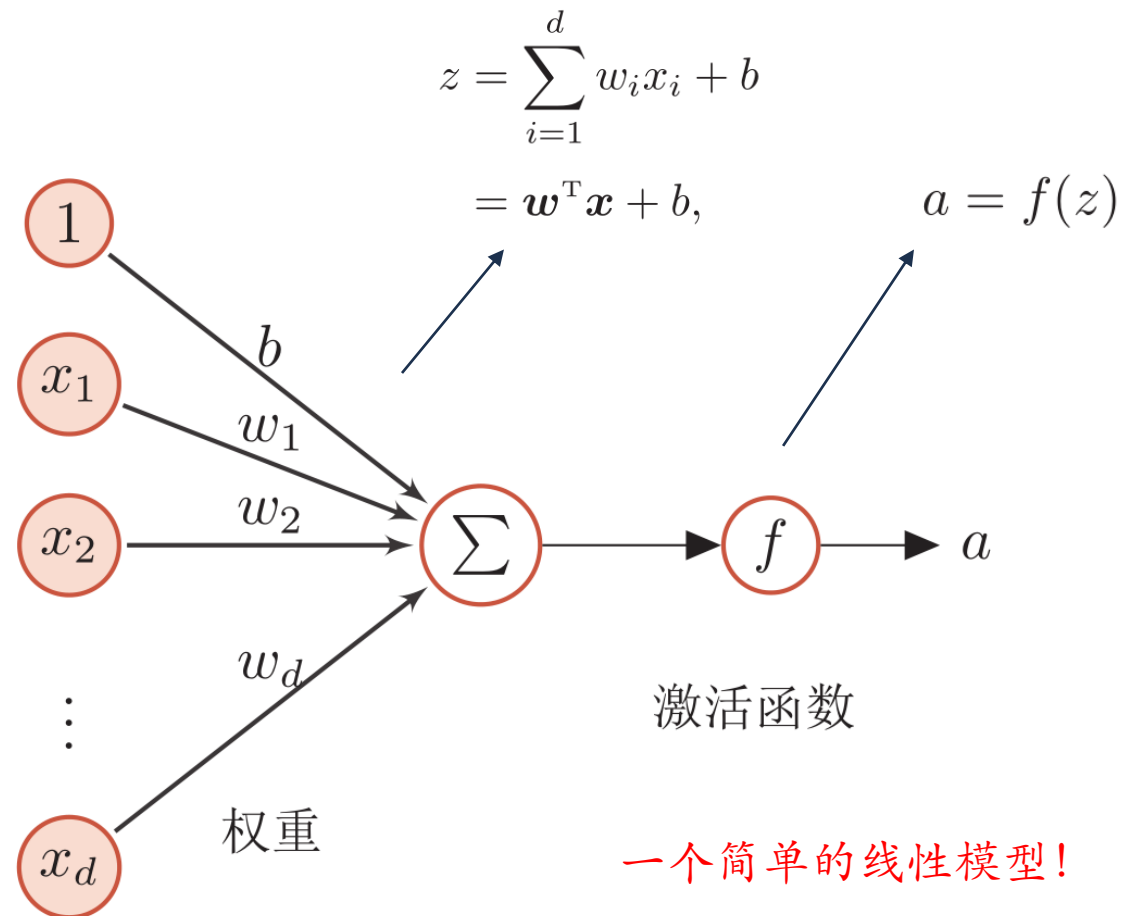
北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY

神经元



单个神经细胞只有两种状态：兴奋和抑制

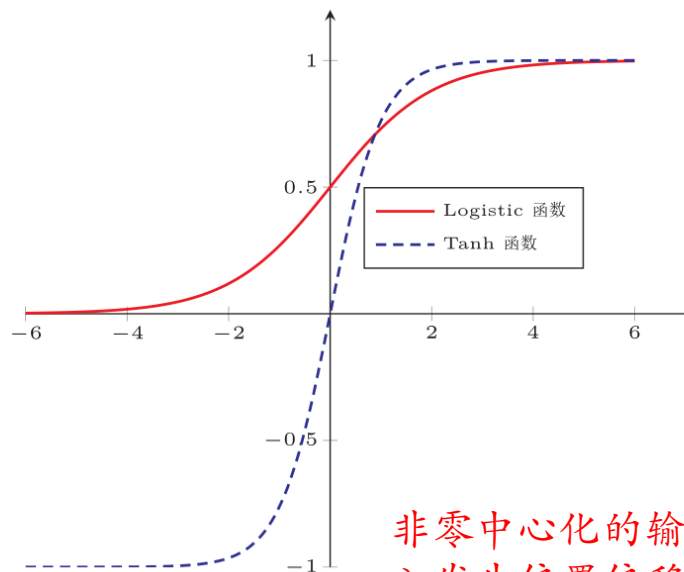
[video: structure of brain](#)



- ▶ 连续并可导（允许少数点上不可导）的非线性函数。
 - ▶ 可导的激活函数可以直接利用数值优化的方法来学习网络参数。
- ▶ 激活函数及其导函数要尽可能的简单
 - ▶ 有利于提高网络计算效率。
- ▶ 激活函数的导函数的值域要在一个合适的区间内
 - ▶ 不能太大也不能太小，否则会影响训练的效率和稳定性。
- ▶ 单调递增
 - ▶ ???

$$\sigma(x) = \frac{1}{1 + \exp(-x)}$$

$$\tanh(x) = \frac{\exp(x) - \exp(-x)}{\exp(x) + \exp(-x)}$$



非零中心化的输出会使得其下一层的神经元的输入发生偏置偏移 (bias shift)，并进一步使得梯度下降的收敛速度变慢。

► 性质：

► 饱和函数

► Tanh函数是零中心化的，而logistic函数的输出恒大于0

$$\text{ReLU}(x) = \begin{cases} x & x \geq 0 \\ 0 & x < 0 \end{cases}$$

$$= \max(0, x).$$

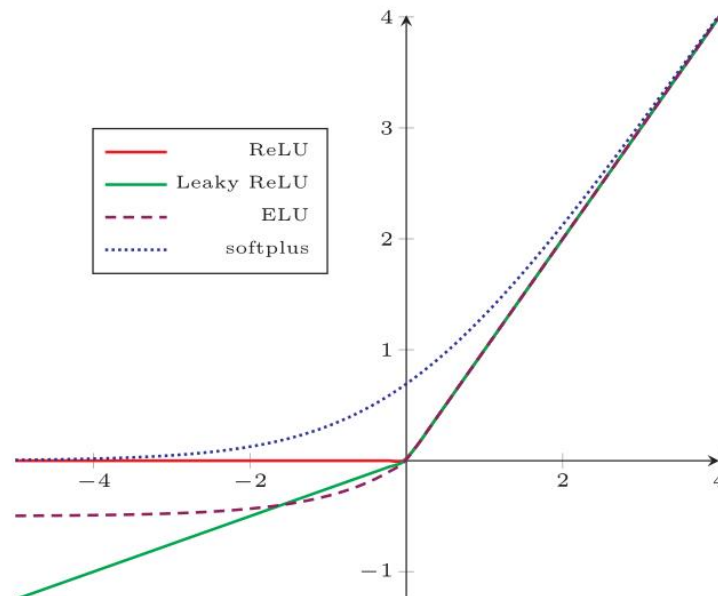
$$\text{LeakyReLU}(x) = \begin{cases} x & \text{if } x > 0 \\ \gamma x & \text{if } x \leq 0 \end{cases}$$

$$\text{PReLU}_i(x) = \begin{cases} x & \text{if } x > 0 \\ \gamma_i x & \text{if } x \leq 0 \end{cases}$$

$$\text{ELU}(x) = \begin{cases} x & \text{if } x > 0 \\ \gamma(\exp(x) - 1) & \text{if } x \leq 0 \end{cases}$$

$$= \max(0, x) + \min(0, \gamma(\exp(x) - 1))$$

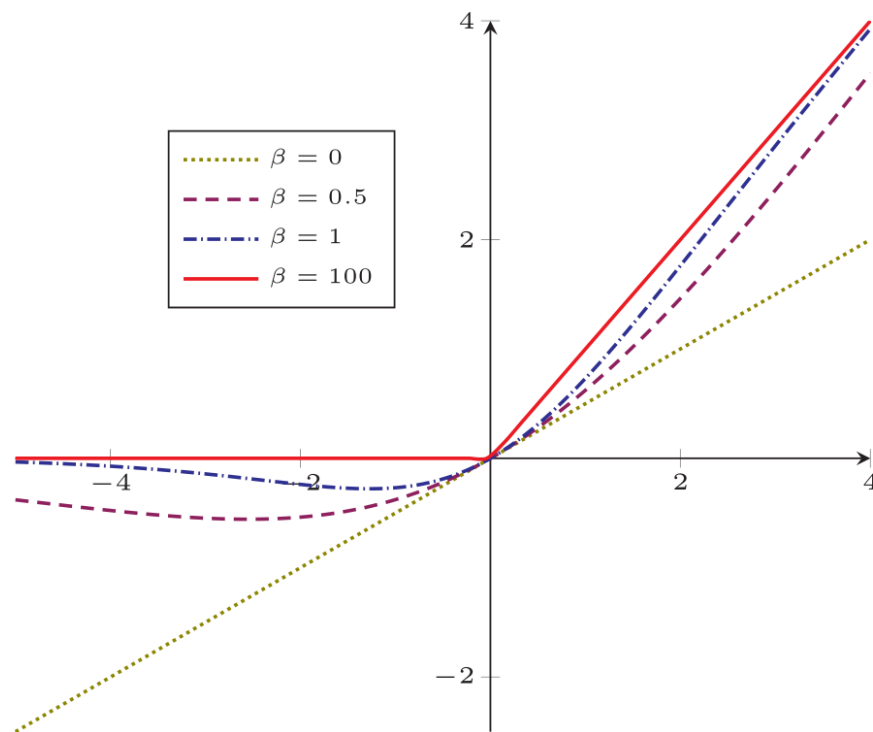
$$\text{softplus}(x) = \log(1 + \exp(x))$$



- ▶ 计算上更加高效
- ▶ 生物学合理性
 - ▶ 单侧抑制、宽兴奋边界
- ▶ 在一定程度上缓解梯度消失问题

死亡ReLU问题 (Dying ReLU Problem)

Swish函数 $\text{swish}(x) = x\sigma(\beta x)$



► 高斯误差线性单元 (Gaussian Error Linear Unit, GELU)

$$\text{GELU}(x) = xP(X \leq x)$$

- 其中 $P(X \leq x)$ 是高斯分布 $N(\mu, \sigma^2)$ 的累积分布函数，其中 μ, σ 为超参数，一般设 $\mu = 0, \sigma = 1$ 即可
- 由于高斯分布的累积分布函数为S型函数，因此GELU可以用Tanh函数或Logistic函数来近似

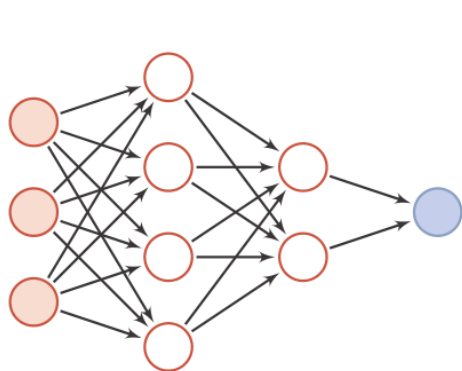
$$\text{GELU}(x) \approx 0.5x \left(1 + \tanh \left(\sqrt{\frac{2}{\pi}} (x + 0.044715x^3) \right) \right)$$

或 $\text{GELU}(x) \approx x\sigma(1.702x).$

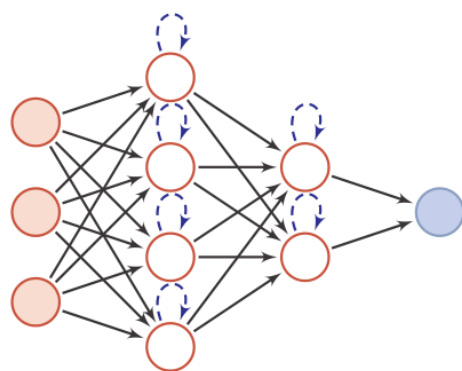
激活函数	函数	导数
Logistic 函数	$f(x) = \frac{1}{1+\exp(-x)}$	$f'(x) = f(x)(1 - f(x))$
Tanh 函数	$f(x) = \frac{\exp(x)-\exp(-x)}{\exp(x)+\exp(-x)}$	$f'(x) = 1 - f(x)^2$
ReLU 函数	$f(x) = \max(0, x)$	$f'(x) = I(x > 0)$
ELU 函数	$f(x) = \max(0, x) + \min(0, \gamma(\exp(x) - 1))$	$f'(x) = I(x > 0) + I(x \leq 0) \cdot \gamma \exp(x)$
SoftPlus 函数	$f(x) = \log(1 + \exp(x))$	$f'(x) = \frac{1}{1+\exp(-x)}$

- ▶ 人工神经网络主要由大量的神经元以及它们之间的有向连接构成。因此考虑三方面：
- ▶ 神经元的激活规则
 - ▶ 主要是指神经元输入到输出之间的映射关系，一般为非线性函数。
- ▶ 网络的拓扑结构
 - ▶ 不同神经元之间的连接关系。
- ▶ 学习算法
 - ▶ 通过训练数据来学习神经网络的参数。

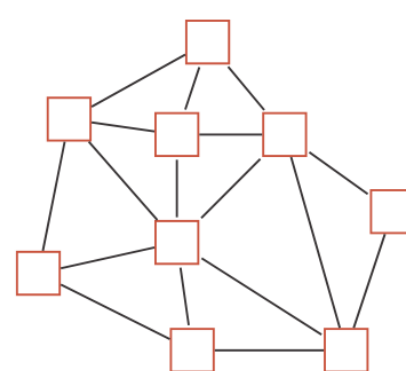
- ▶ 人工神经网络由神经元模型构成，这种由许多神经元组成的信息处理网络具有并行分布结构。



(a) 前馈网络



(b) 记忆网络



(c) 图网络

圆形节点表示一个神经元，方形节点表示一组神经元。

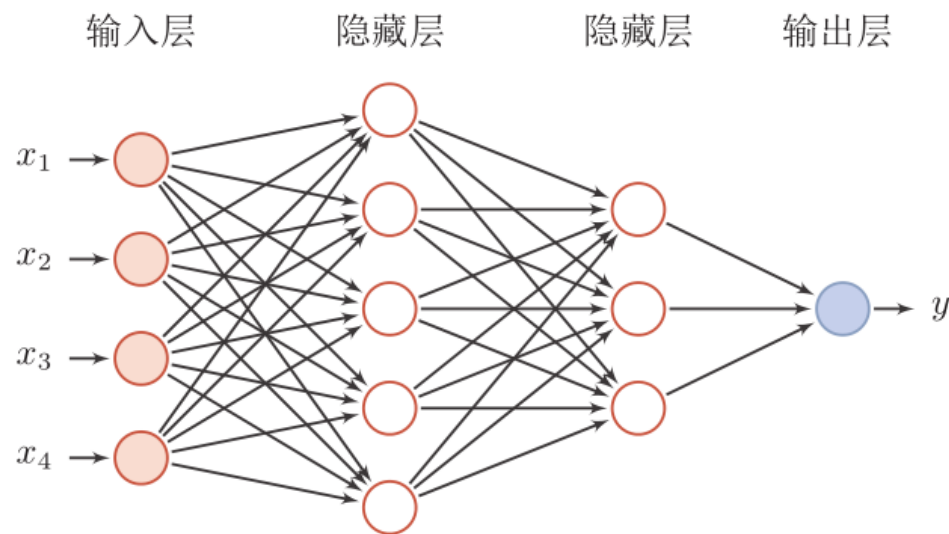


北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY

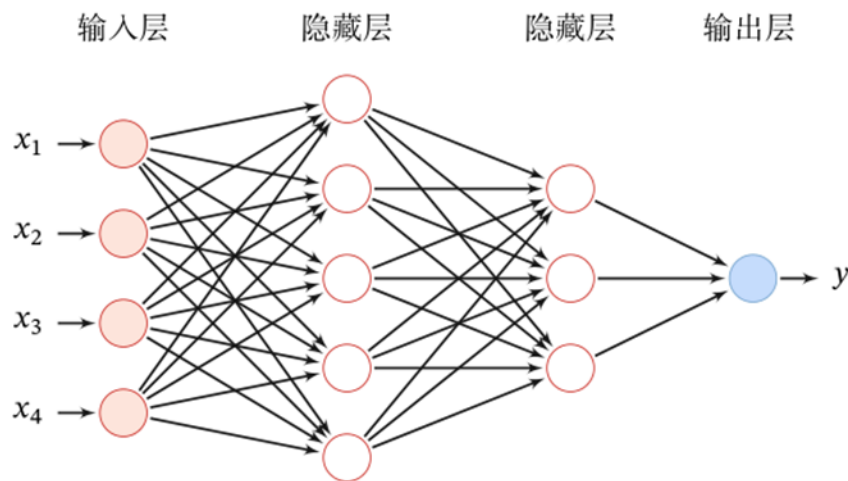
前馈神经网络

▶ 前馈神经网络（全连接神经网络、多层感知器）

- ▶ 各神经元分别属于不同的层，层内无连接。
- ▶ 相邻两层之间的神经元全部两两连接。
- ▶ 整个网络中无反馈，信号从输入层向输出层单向传播，可用一个有向无环图表示。



给定一个前馈神经网络，用下面的记号来描述这样网络：



记号	含义
L	神经网络的层数
M_l	第 l 层神经元的个数
$f_l(\cdot)$	第 l 层神经元的激活函数
$\mathbf{W}^{(l)} \in \mathbb{R}^{M_l \times M_{l-1}}$	第 $l-1$ 层到第 l 层的权重矩阵
$\mathbf{b}^{(l)} \in \mathbb{R}^{M_l}$	第 $l-1$ 层到第 l 层的偏置
$\mathbf{z}^{(l)} \in \mathbb{R}^{M_l}$	第 l 层神经元的净输入（净活性值）
$\mathbf{a}^{(l)} \in \mathbb{R}^{M_l}$	第 l 层神经元的输出（活性值）

► 前馈神经网络通过下面公式进行信息传播。

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)},$$

$$\mathbf{a}^{(l)} = f_l(\mathbf{z}^{(l)}).$$

$$\begin{bmatrix} z_1^{(l)} \\ z_2^{(l)} \\ \vdots \\ z_m^{(l)} \end{bmatrix} = \begin{bmatrix} w_{11}^{(l)} & w_{12}^{(l)} & \cdots & w_{1n}^{(l)} \\ w_{21}^{(l)} & w_{22}^{(l)} & \cdots & w_{2n}^{(l)} \\ \vdots & \vdots & & \vdots \\ w_{m1}^{(l)} & w_{m2}^{(l)} & \cdots & w_{mn}^{(l)} \end{bmatrix} \begin{bmatrix} a_1^{(l-1)} \\ a_2^{(l-1)} \\ \vdots \\ a_n^{(l-1)} \end{bmatrix} + \begin{bmatrix} b_1^{(l)} \\ b_2^{(l)} \\ \vdots \\ b_n^{(l)} \end{bmatrix}$$

► 前馈计算：

$$\mathbf{x} = \mathbf{a}^{(0)} \rightarrow \mathbf{z}^{(1)} \rightarrow \mathbf{a}^{(1)} \rightarrow \mathbf{z}^{(2)} \rightarrow \cdots \rightarrow \mathbf{a}^{(L-1)} \rightarrow \mathbf{z}^{(L)} \rightarrow \mathbf{a}^{(L)} = \phi(\mathbf{x}; \mathbf{W}, \mathbf{b})$$

定理 4.1 – 通用近似定理 (Universal Approximation Theorem)

[Cybenko, 1989, Hornik et al., 1989]: 令 $\varphi(\cdot)$ 是一个非常数、有界、单调递增的连续函数, $\mathcal{I}_d$ 是一个 d 维的单位超立方体 $[0, 1]^d$, $C(\mathcal{I}_d)$ 是定义在 $\mathcal{I}_d$ 上的连续函数集合。对于任何一个函数 $f \in C(\mathcal{I}_d)$, 存在一个整数 m , 和一组实数 $v_i, b_i \in \mathbb{R}$ 以及实数向量 $\mathbf{w}_i \in \mathbb{R}^d$, $i = 1, \dots, m$, 以至于我们可以定义函数

$$F(\mathbf{x}) = \sum_{i=1}^m v_i \varphi(\mathbf{w}_i^T \mathbf{x} + b_i), \quad (4.33)$$

作为函数 f 的近似实现, 即

$$|F(\mathbf{x}) - f(\mathbf{x})| < \epsilon, \forall \mathbf{x} \in \mathcal{I}_d. \quad (4.34)$$

其中 $\epsilon > 0$ 是一个很小的正数。

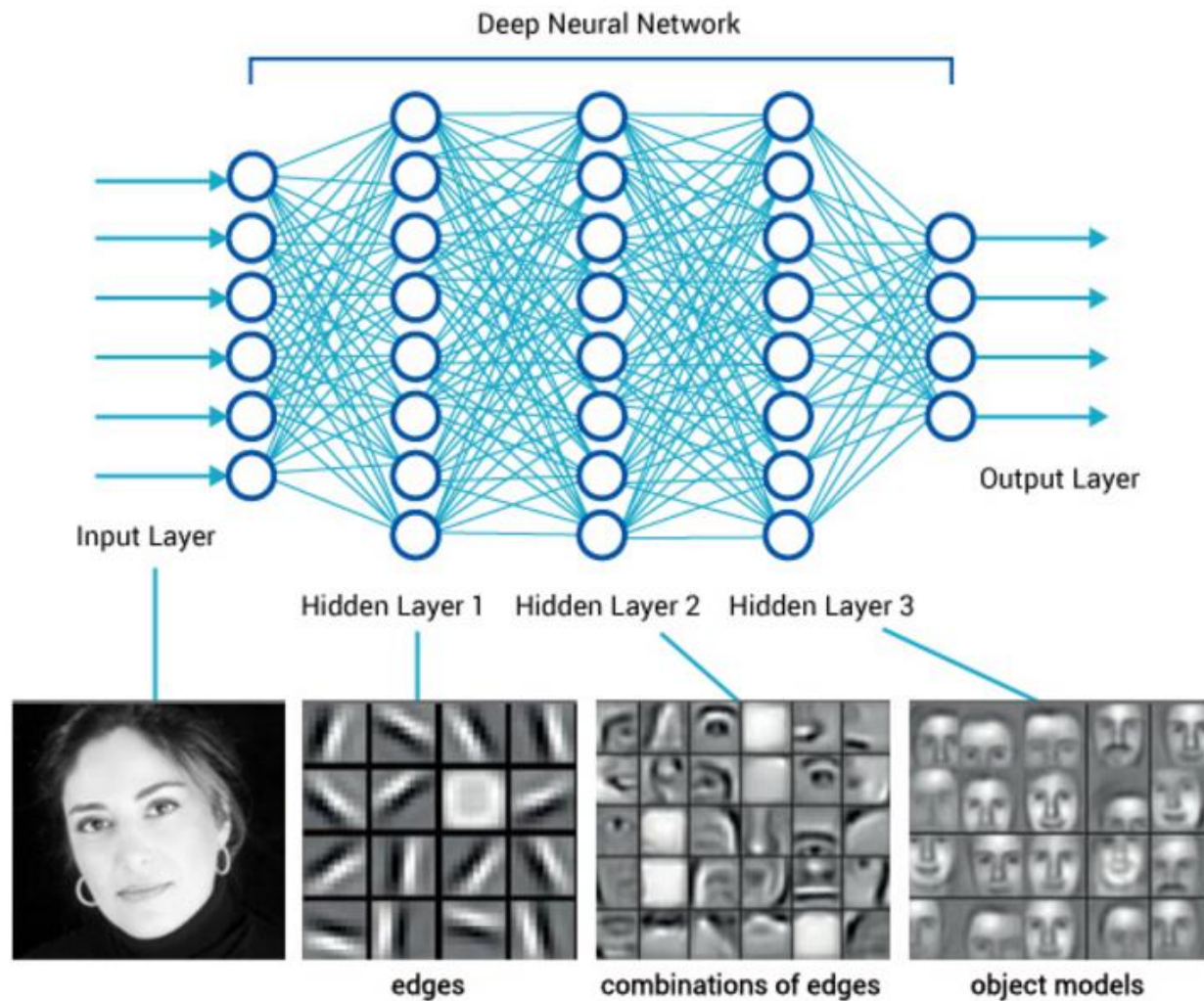
根据通用近似定理, 对于具有线性输出层和至少一个使用“挤压”性质的激活函数的隐藏层组成的前馈神经网络, 只要其隐藏层神经元的数量足够, 它可以以任意的精度来近似任何从一个定义在实数空间中的有界闭集函数。

- ▶ 神经网络可以作为一个“万能”函数来使用，可以用来进行复杂的特征转换，或逼近一个复杂的条件分布。

$$\hat{y} = g(\underbrace{\varphi(\mathbf{x})}_{\text{神经网络}}, \theta)$$

分类器

- ▶ 如果 $g(\cdot)$ 为Logistic回归，那么Logistic回归分类器可以看成神经网络的最后一层。





北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY

参数学习

► 对于多分类问题

- 如果使用Softmax回归分类器，相当于网络最后一层设置C个神经元，其输出经过Softmax函数进行归一化后可以作为每个类的条件概率。

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{z}^{(L)})$$

- 采用交叉熵损失函数，对于样本(x,y)，其损失函数为

$$\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}}) = -\mathbf{y}^T \log \hat{\mathbf{y}}$$

- ▶ 给定训练集为 $D = \{(\mathbf{x}^{(n)}, y^{(n)})\}_{n=1}^N$ ，将每个样本 $\mathbf{x}^{(n)}$ 输入给前馈神经网络，得到网络输出为 $\hat{y}^{(n)}$ ，其在数据集 D 上的结构化风险函数为：

$$\mathcal{R}(W, \mathbf{b}) = \frac{1}{N} \sum_{n=1}^N \mathcal{L}(\mathbf{y}^{(n)}, \hat{\mathbf{y}}^{(n)}) + \frac{1}{2} \lambda \|W\|_F^2$$

- ▶ 梯度下降

$$W^{(l)} \leftarrow W^{(l)} - \alpha \frac{\partial \mathcal{R}(W, \mathbf{b})}{\partial W^{(l)}}$$

$$\mathbf{b}^{(l)} \leftarrow \mathbf{b}^{(l)} - \alpha \frac{\partial \mathcal{R}(W, \mathbf{b})}{\partial \mathbf{b}^{(l)}}$$

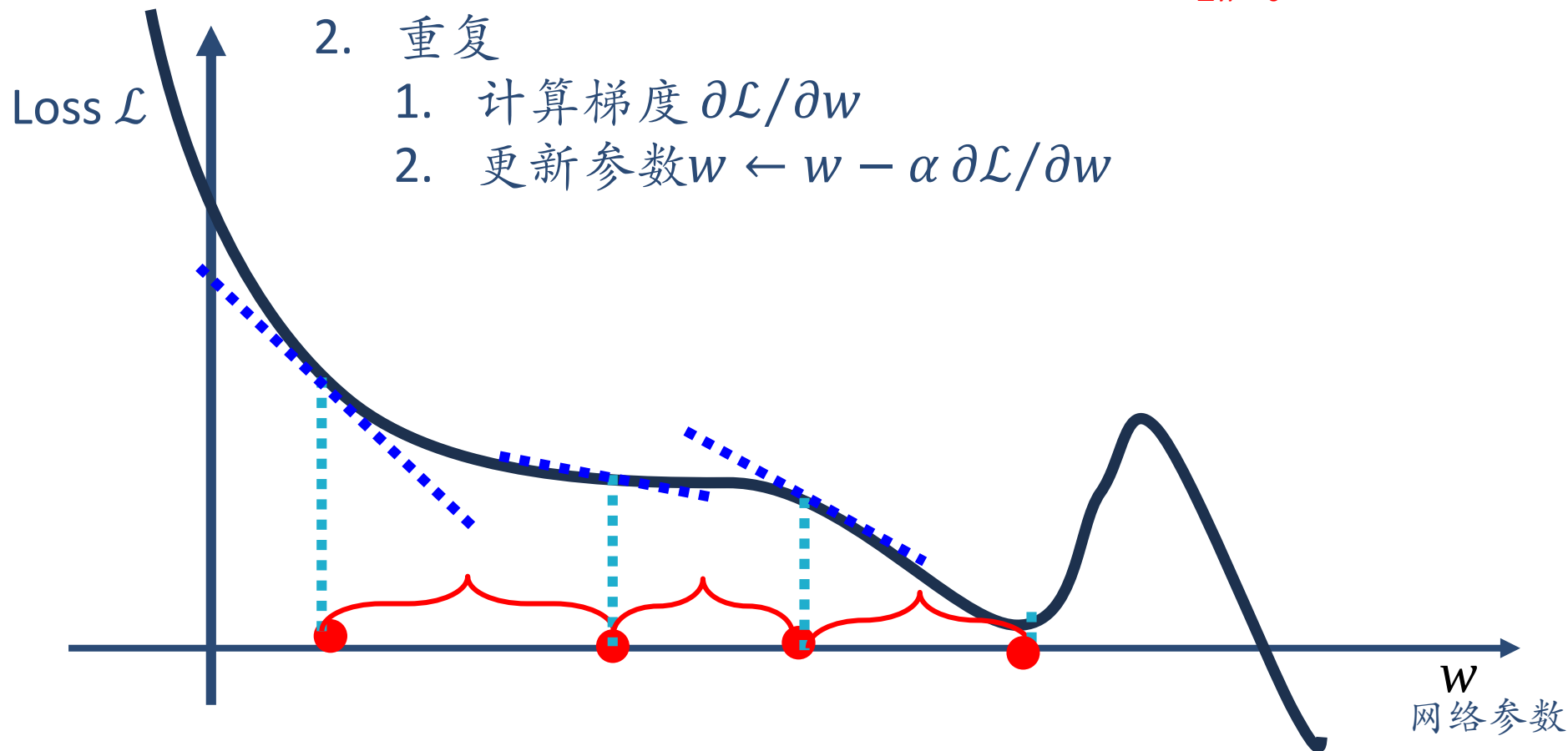
1. 初始化 w

2. 重复

1. 计算梯度 $\partial \mathcal{L} / \partial w$

2. 更新参数 $w \leftarrow w - \alpha \partial \mathcal{L} / \partial w$

$$\text{梯度: } \frac{\partial f(w)}{\partial w} = \lim_{\Delta w \rightarrow 0} \frac{f(w + \Delta w) - f(w)}{\Delta w}$$



▶神经网络为一个复杂的复合函数

▶链式法则

$$y = f^5(f^4(f^3(f^2(f^1(x))))) \rightarrow \frac{\partial y}{\partial x} = \frac{\partial f^1}{\partial x} \frac{\partial f^2}{\partial f^1} \frac{\partial f^3}{\partial f^2} \frac{\partial f^4}{\partial f^3} \frac{\partial f^5}{\partial f^4}$$

▶反向传播算法

▶根据前馈网络的特点而设计的高效方法

▶一个更加通用的计算方法

▶自动微分 (Automatic Differentiation, AD)

- ▶ 矩阵微积分 (Matrix Calculus) 是多元微积分的一种表达方式，即使用矩阵和向量来表示因变量每个成分关于自变量每个成分的偏导数。
- ▶ 分母布局
 - ▶ 标量关于向量的偏导数

$$\frac{\partial y}{\partial \mathbf{x}} = \left[\frac{\partial y}{\partial x_1}, \dots, \frac{\partial y}{\partial x_p} \right]^T$$

- ▶ 向量关于向量的偏导数

$$\frac{\partial f(\mathbf{x})}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \dots & \frac{\partial y_q}{\partial x_1} \\ \vdots & \vdots & \vdots \\ \frac{\partial y_1}{\partial x_p} & \dots & \frac{\partial y_q}{\partial x_p} \end{bmatrix} \in \mathbb{R}^{p \times q}$$

► 链式法则 (Chain Rule) 是在微积分中求复合函数导数的一种常用方法。

(1) 若 $x \in \mathbb{R}$, $\mathbf{u} = u(x) \in \mathbb{R}^s$, $\mathbf{g} = g(\mathbf{u}) \in \mathbb{R}^t$, 则

$$\frac{\partial \mathbf{g}}{\partial x} = \frac{\partial \mathbf{u}}{\partial x} \frac{\partial \mathbf{g}}{\partial \mathbf{u}} \in \mathbb{R}^{1 \times t}.$$

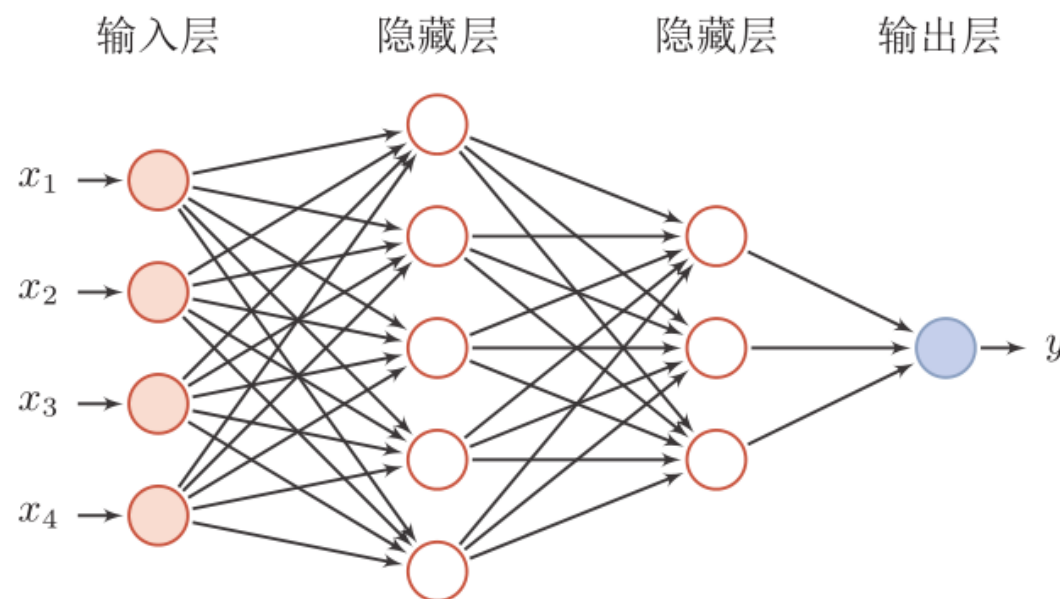
(2) 若 $\mathbf{x} \in \mathbb{R}^p$, $\mathbf{y} = g(\mathbf{x}) \in \mathbb{R}^s$, $z = f(\mathbf{y}) \in \mathbb{R}^t$, 则

$$\frac{\partial z}{\partial \mathbf{x}} = \frac{\partial \mathbf{y}}{\partial \mathbf{x}} \frac{\partial z}{\partial \mathbf{y}} \in \mathbb{R}^{p \times t}.$$

(3) 若 $X \in \mathbb{R}^{p \times q}$ 为矩阵, $\mathbf{y} = g(X) \in \mathbb{R}^s$, $z = f(\mathbf{y}) \in \mathbb{R}$, 则

$$\frac{\partial z}{\partial X_{ij}} = \frac{\partial \mathbf{y}}{\partial X_{ij}} \frac{\partial z}{\partial \mathbf{y}} \in \mathbb{R}.$$

$$\mathcal{R}(W, \mathbf{b}) = \frac{1}{N} \sum_{n=1}^N \mathcal{L}(\mathbf{y}^{(n)}, \hat{\mathbf{y}}^{(n)}) + \frac{1}{2} \lambda \|W\|_F^2$$



$$\frac{\partial \mathcal{R}(w, b)}{\partial w^l}$$

$$\frac{\partial \mathcal{R}(w, b)}{\partial b^l}$$

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$$

$$\begin{aligned} \frac{\partial \mathbf{z}^{(l)}}{\partial w_{ij}^{(l)}} &= \left[\frac{\partial z_1^{(l)}}{\partial w_{ij}^{(l)}}, \dots, \frac{\partial z_i^{(l)}}{\partial w_{ij}^{(l)}}, \dots, \frac{\partial z_m^{(l)}}{\partial w_{ij}^{(l)}} \right] \\ &= \left[0, \dots, \frac{\partial (\mathbf{w}_{i:}^{(l)} \mathbf{a}^{(l-1)} + b_i^{(l)})}{\partial w_{ij}^{(l)}}, \dots, 0 \right] \\ &= \left[0, \dots, a_j^{(l-1)}, \dots, 0 \right] \\ &\triangleq \mathbb{I}_i(a_j^{(l-1)}) \in \mathbb{R}^{m^{(l)}}, \end{aligned}$$

$$\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial w_{ij}^{(l)}} = \frac{\partial \mathbf{z}^{(l)}}{\partial w_{ij}^{(l)}} \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l)}}$$

$$\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{b}^{(l)}} = \frac{\partial \mathbf{z}^{(l)}}{\partial \mathbf{b}^{(l)}} \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l)}}$$

$$\delta^{(l)} = \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l)}} \in \mathbb{R}^{m^{(l)}}$$

误差项

$$\begin{bmatrix} z_1^{(l)} \\ z_2^{(l)} \\ \vdots \\ z_m^{(l)} \end{bmatrix} = \begin{bmatrix} w_{11}^{(l)} & w_{12}^{(l)} & \dots & w_{1n}^{(l)} \\ w_{21}^{(l)} & w_{22}^{(l)} & \dots & w_{2n}^{(l)} \\ \vdots & \vdots & \ddots & \vdots \\ w_{m1}^{(l)} & w_{m2}^{(l)} & \dots & w_{mn}^{(l)} \end{bmatrix} \begin{bmatrix} a_1^{(l-1)} \\ a_2^{(l-1)} \\ \vdots \\ a_n^{(l-1)} \end{bmatrix} + \begin{bmatrix} b_1^{(l)} \\ b_2^{(l)} \\ \vdots \\ b_n^{(l)} \end{bmatrix}$$

$$\frac{\partial \mathbf{z}^{(l)}}{\partial \mathbf{b}^{(l)}} = \mathbf{I}_{m^{(l)}} \in \mathbb{R}^{m^{(l)} \times m^{(l)}}$$

$$\delta^{(l)} = \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l)}} \in \mathbb{R}^{m^{(l)}}$$

$$\mathbf{a}^{(l)} = f_l(\mathbf{z}^{(l)})$$

$$\mathbf{z}^{(l+1)} = W^{(l+1)} \mathbf{a}^{(l)} + \mathbf{b}^{(l+1)}$$

$$\frac{\partial \mathbf{z}^{(l+1)}}{\partial \mathbf{a}^{(l)}} = (W^{(l+1)})^T$$

$$\underline{\delta^{(l)}} \triangleq \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l)}}$$

$$\begin{aligned} \frac{\partial \mathbf{a}^{(l)}}{\partial \mathbf{z}^{(l)}} &= \frac{\partial f_l(\mathbf{z}^{(l)})}{\partial \mathbf{z}^{(l)}} \\ &= \text{diag}(f'_l(\mathbf{z}^{(l)})) \end{aligned}$$

$$\begin{aligned} &= \frac{\partial \mathbf{a}^{(l)}}{\partial \mathbf{z}^{(l)}} \cdot \frac{\partial \mathbf{z}^{(l+1)}}{\partial \mathbf{a}^{(l)}} \cdot \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l+1)}} \\ &= \text{diag}(f'_l(\mathbf{z}^{(l)})) \cdot (W^{(l+1)})^T \cdot \underline{\delta^{(l+1)}} \\ &= f'_l(\mathbf{z}^{(l)}) \odot ((W^{(l+1)})^T \delta^{(l+1)}), \end{aligned}$$

在计算出上面三个偏导数之后, 公式 (4.49) 可以写为

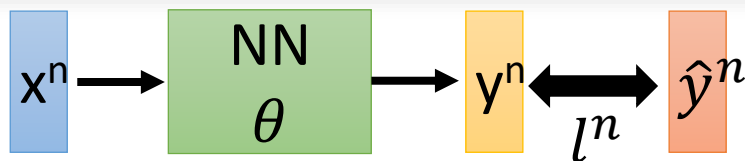
$$\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial w_{ij}^{(l)}} = \mathbb{I}_i(a_j^{(l-1)})\delta^{(l)} = \delta_i^{(l)} a_j^{(l-1)}.$$

进一步, $\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})$ 关于第 l 层权重 $W^{(l)}$ 的梯度为

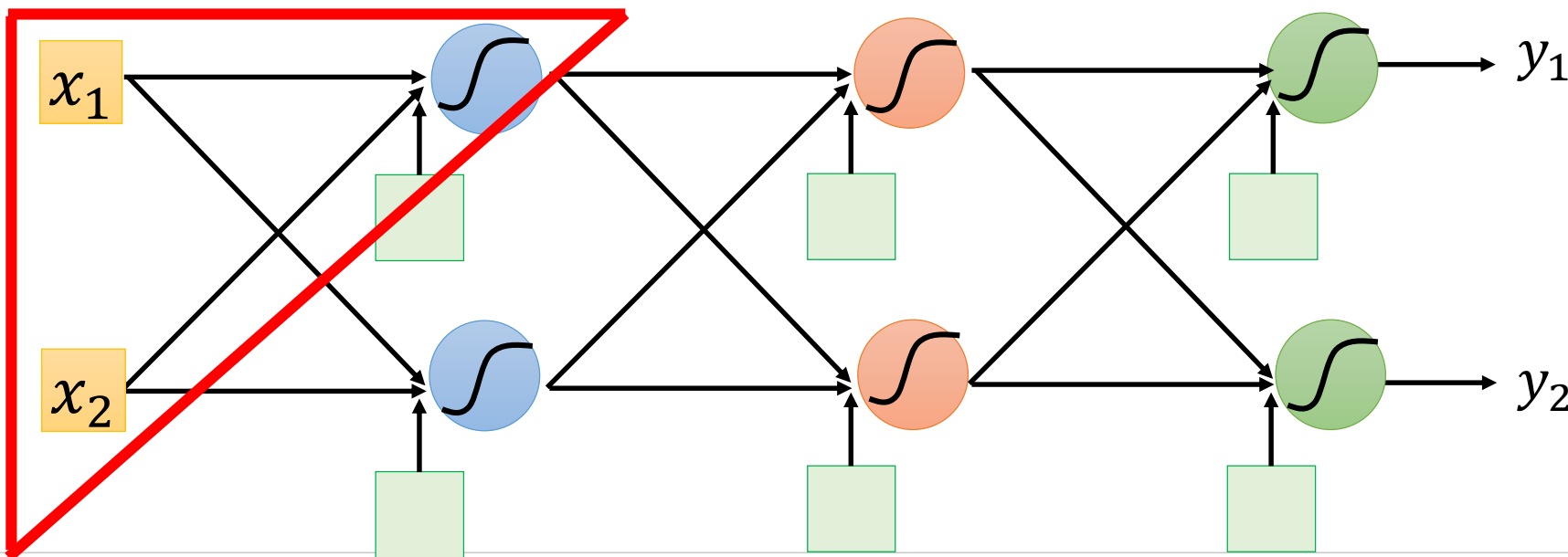
$$\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial W^{(l)}} = \delta^{(l)} (\mathbf{a}^{(l-1)})^\top.$$

同理, $\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})$ 关于第 l 层偏置 $\mathbf{b}^{(l)}$ 的梯度为

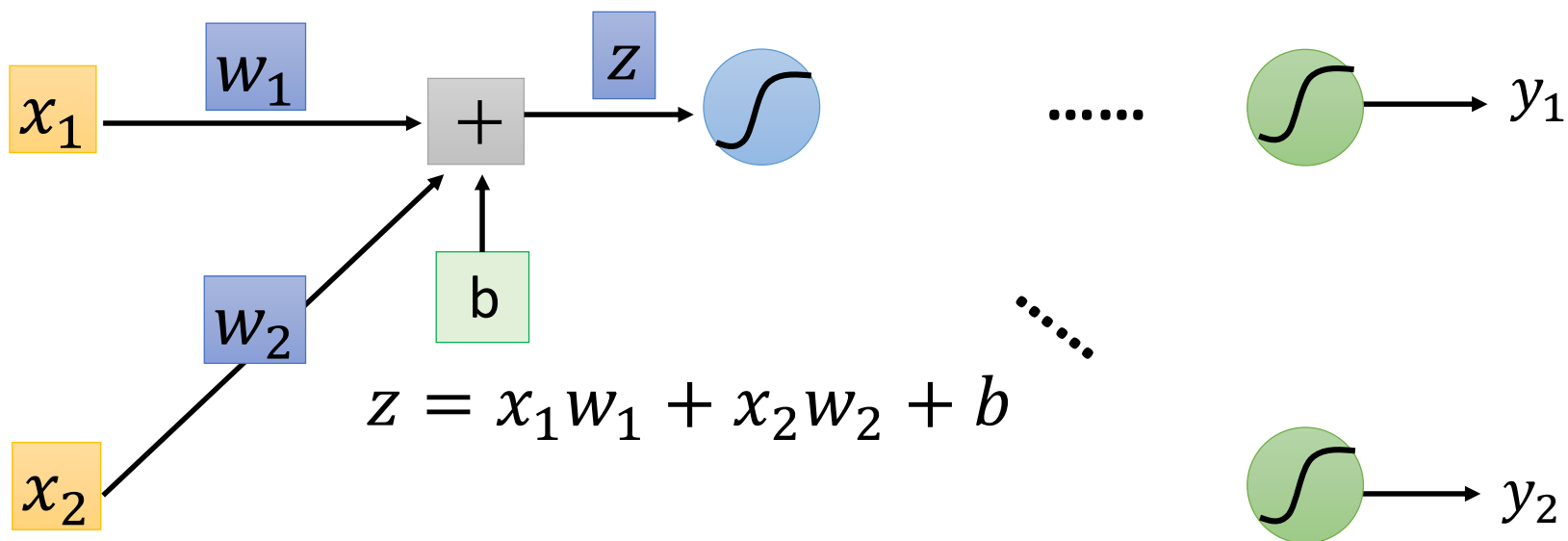
$$\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{b}^{(l)}} = \delta^{(l)}.$$



$$L(\theta) = \sum_{n=1}^N l^n(\theta) \quad \Rightarrow \quad \frac{\partial L(\theta)}{\partial w} = \sum_{n=1}^N \frac{\partial l^n(\theta)}{\partial w}$$

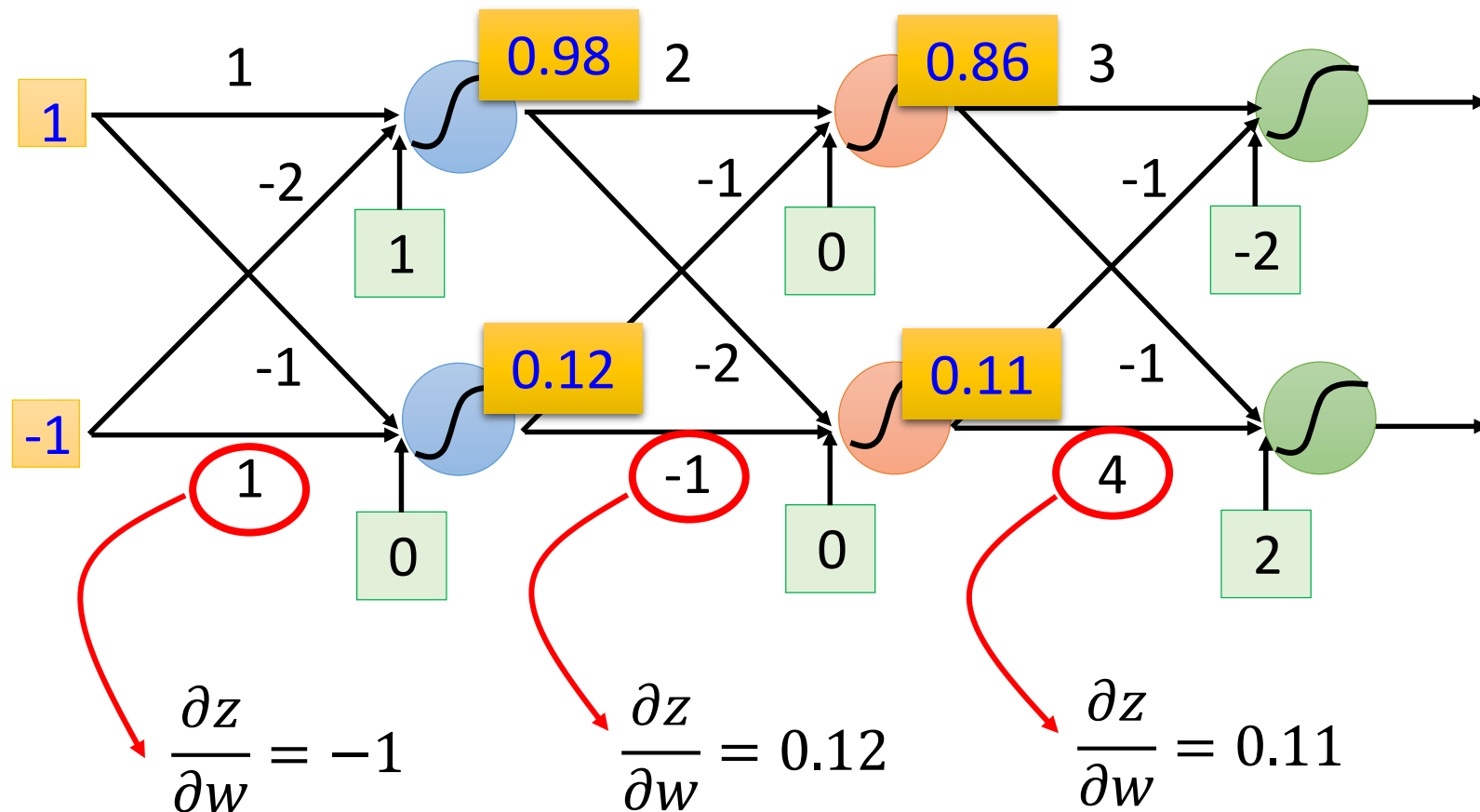


计算 $\partial z / \partial w$

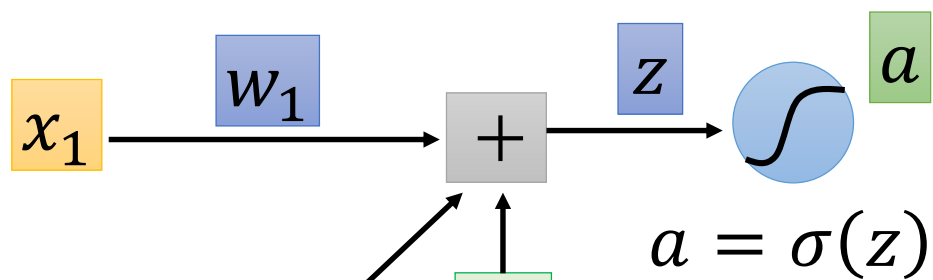


$$\left. \begin{aligned} \partial z / \partial w_1 &=? \quad x_1 \\ \partial z / \partial w_2 &=? \quad x_2 \end{aligned} \right\} \text{结果等于权值对应的输入值}$$

计算 $\partial z / \partial w$

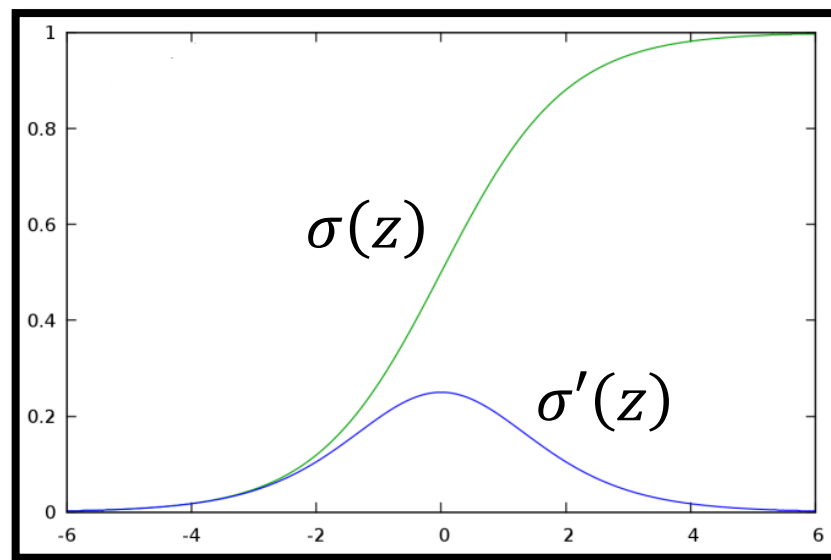


计算 $\partial L / \partial z$

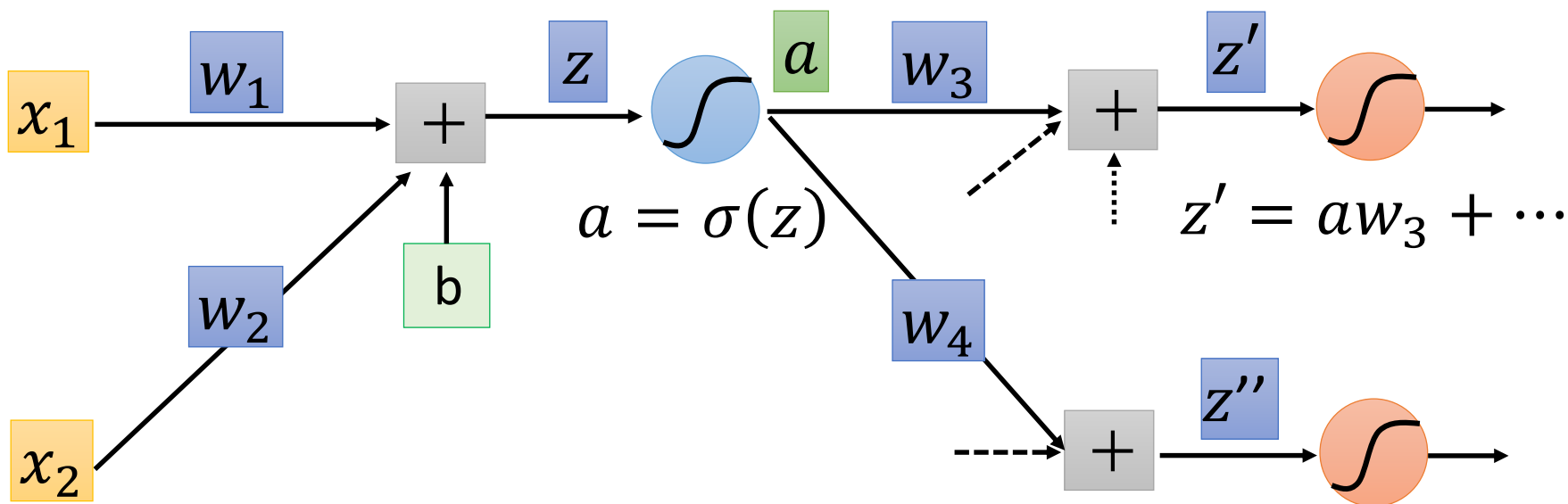


$$\frac{\partial L}{\partial z} = \frac{\partial a}{\partial z} \frac{\partial L}{\partial a}$$

➡ $\sigma'(z)$



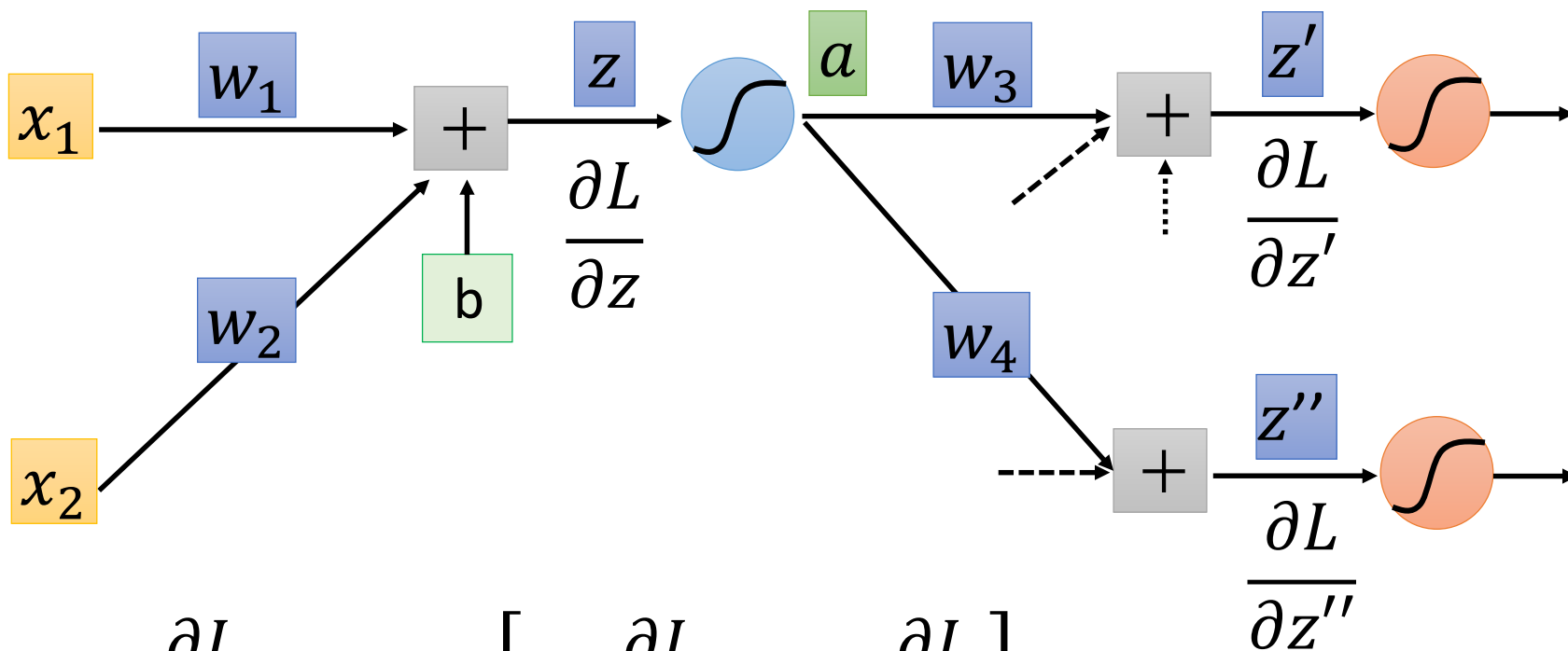
J 计算 $\partial L / \partial z$



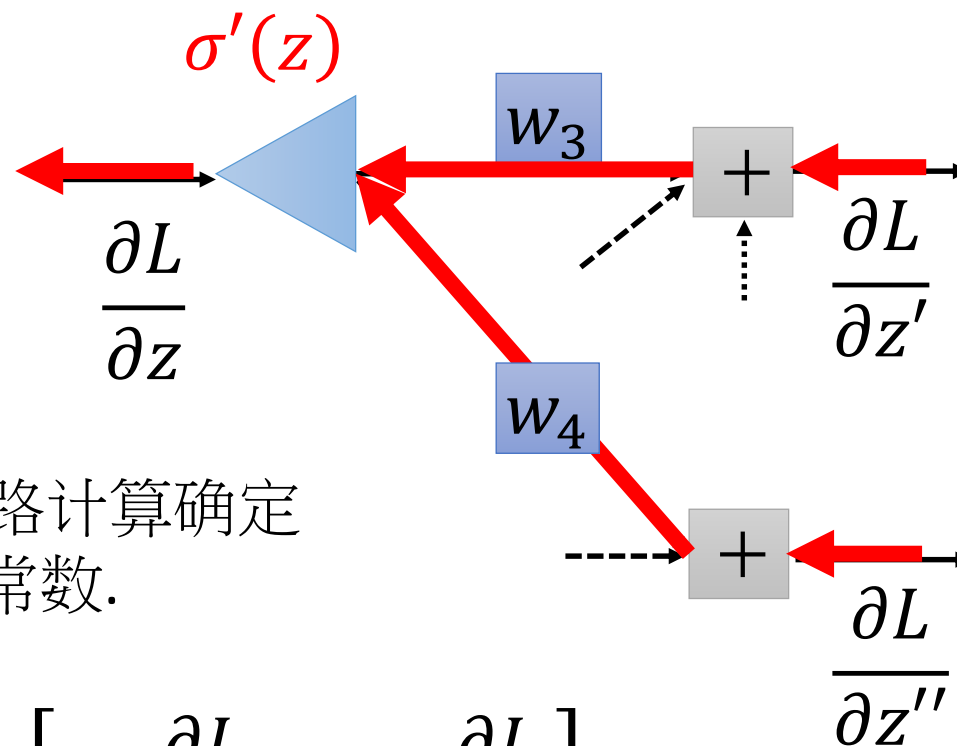
$$\frac{\partial L}{\partial z} = \frac{\partial a}{\partial z} \frac{\partial L}{\partial a}$$

$$\frac{\partial L}{\partial a} = \underbrace{\frac{\partial z'}{\partial a}}_{w_3} \underbrace{\frac{\partial L}{\partial z'}}_{?} + \underbrace{\frac{\partial z''}{\partial a}}_{w_4} \underbrace{\frac{\partial L}{\partial z''}}_{?} \quad (\text{链式法则})$$

计算 $\partial L / \partial z$



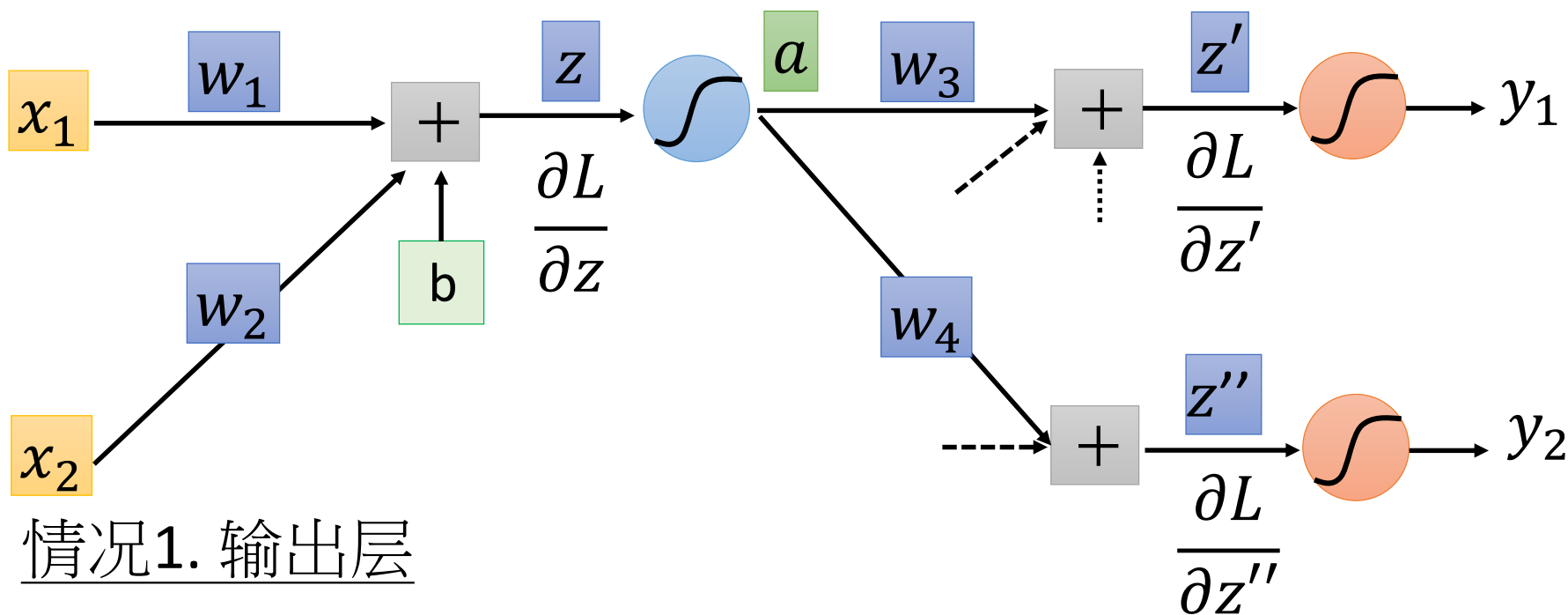
$$\frac{\partial L}{\partial z} = \sigma'(z) \left[w_3 \frac{\partial L}{\partial z'} + w_4 \frac{\partial L}{\partial z''} \right]$$



z 由前向链路计算确定
 $\sigma'(z)$ 是个常数.

$$\frac{\partial L}{\partial z} = \sigma'(z) \left[w_3 \frac{\partial L}{\partial z'} + w_4 \frac{\partial L}{\partial z''} \right]$$

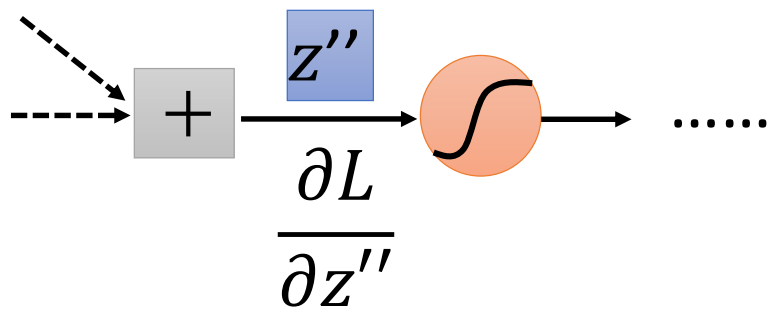
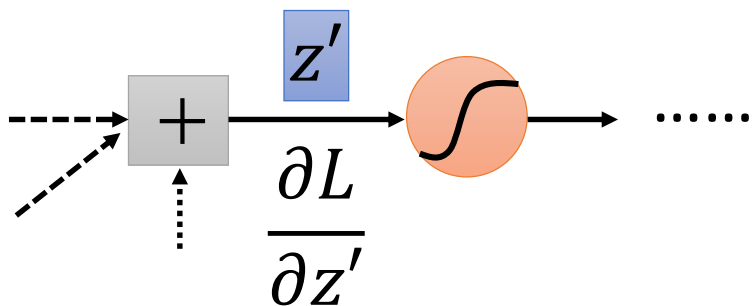
计算 $\partial L / \partial z$



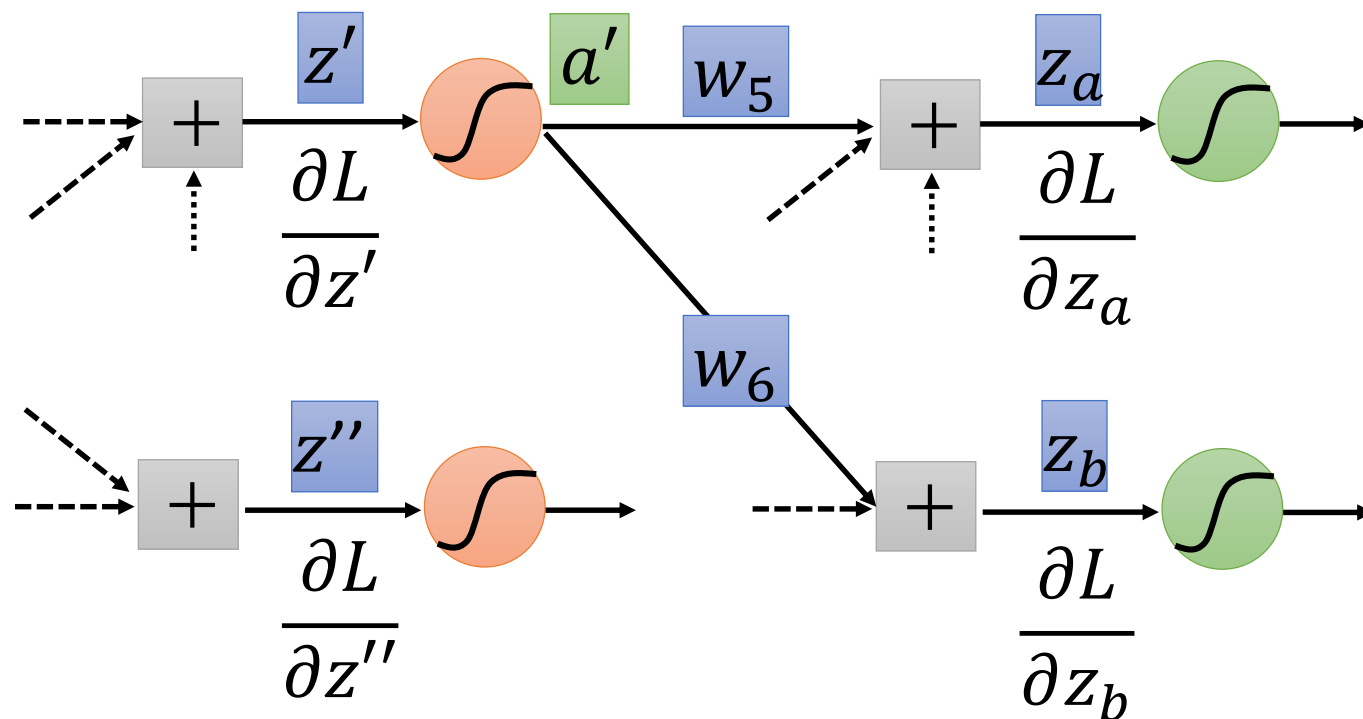
$$\frac{\partial L}{\partial z'} = \frac{\partial y_1}{\partial z'} \frac{\partial L}{\partial y_1} \quad \frac{\partial L}{\partial z''} = \frac{\partial y_2}{\partial z''} \frac{\partial L}{\partial y_2}$$

计算 $\partial L / \partial z$

情况2. 不是输出层

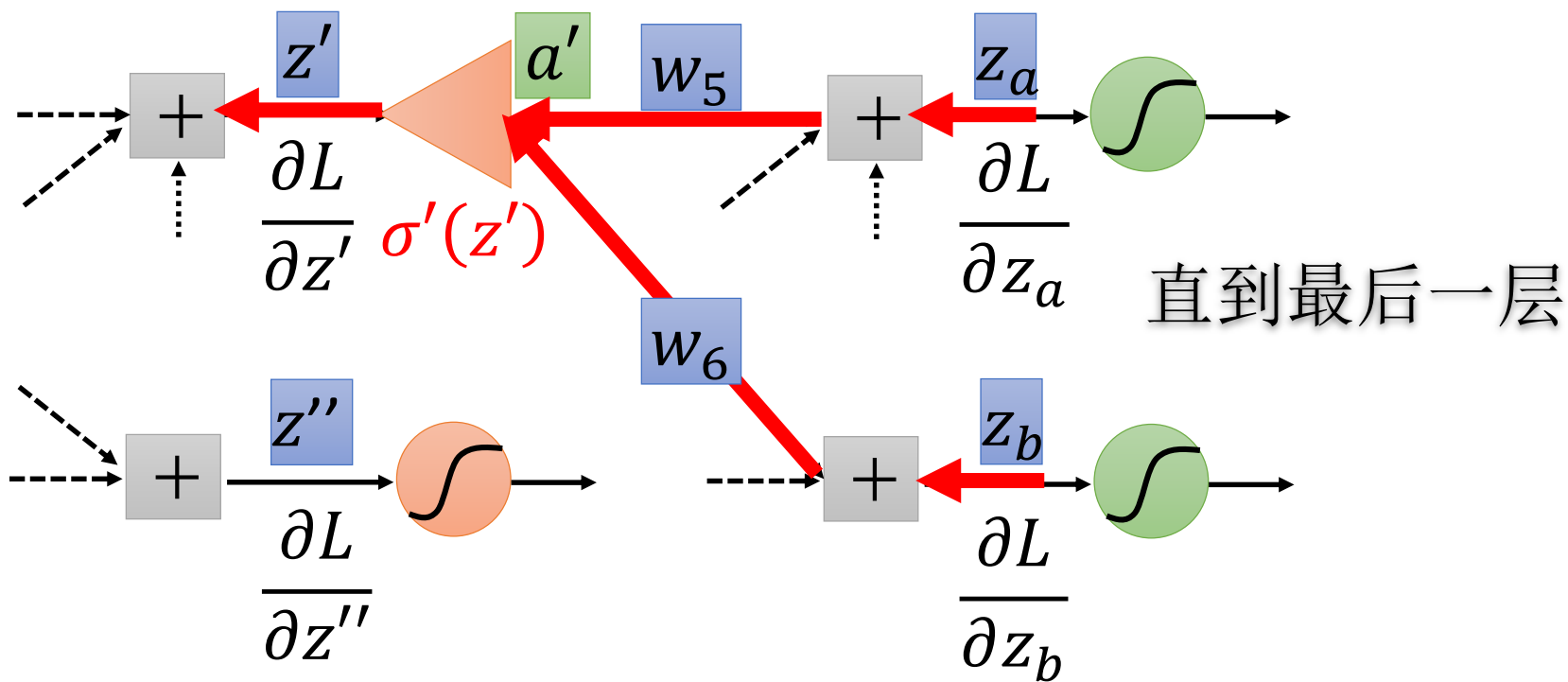


计算 $\partial L / \partial z$



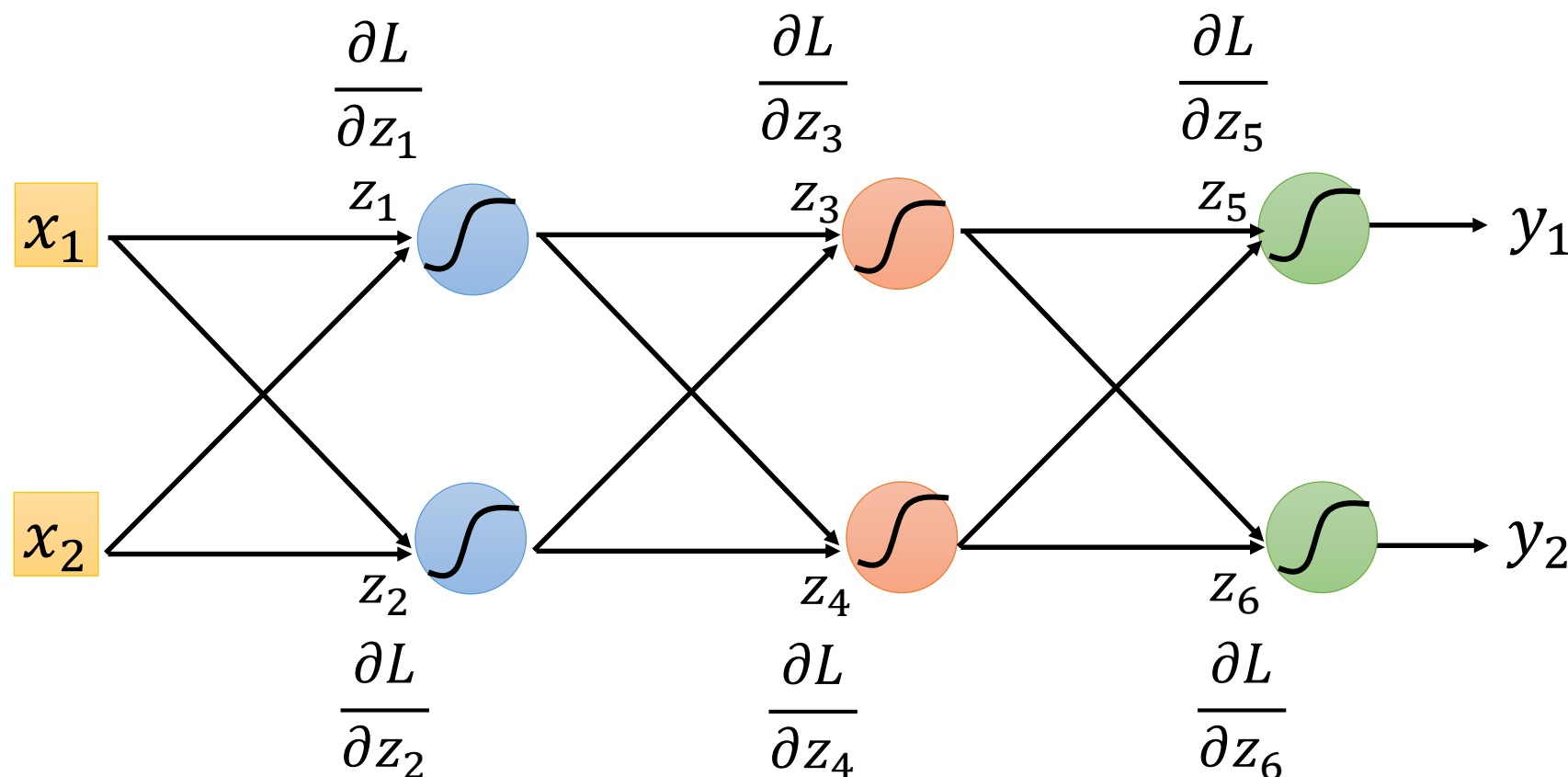
计算 $\partial L / \partial z$

反向计算 $\partial L / \partial z$



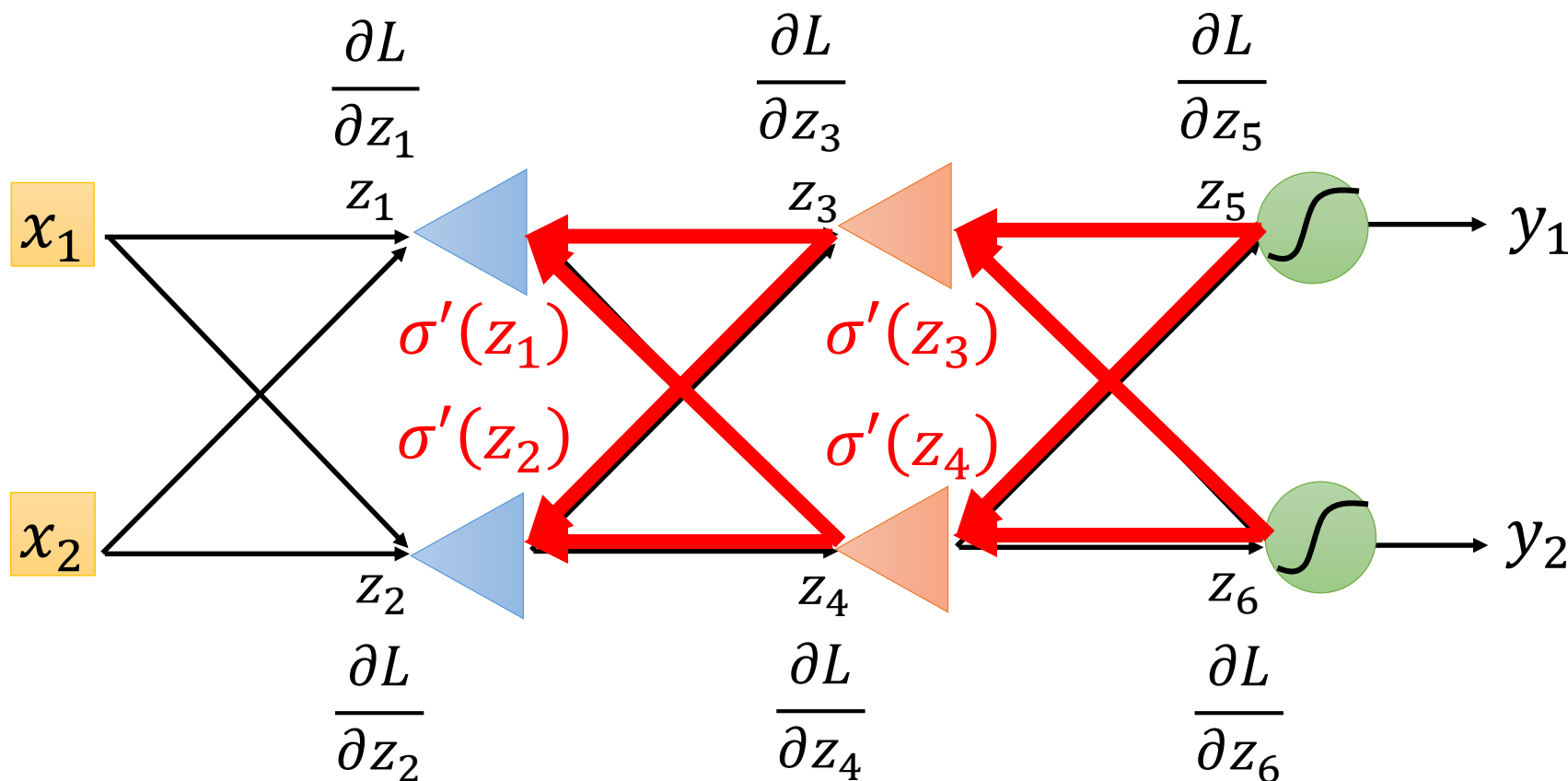
计算所有神经元净输入 z 的偏导数 $\partial L / \partial z$

从最后一层开始计算 $\partial L / \partial z$

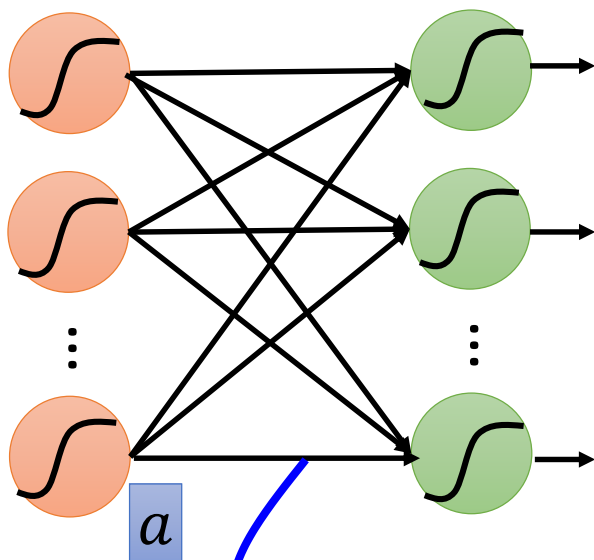


计算所有神经元净输入 z 的偏导数 $\partial L / \partial z$

从最后一层开始计算 $\partial L / \partial z$

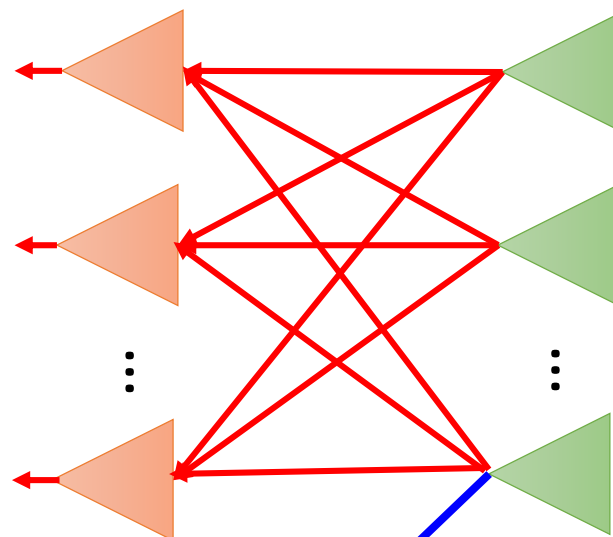


前向传递



$$\frac{\partial z}{\partial w} = a$$

反向传递



X

$$\frac{\partial L}{\partial z}$$

$$= \frac{\partial L}{\partial w}$$

for all w



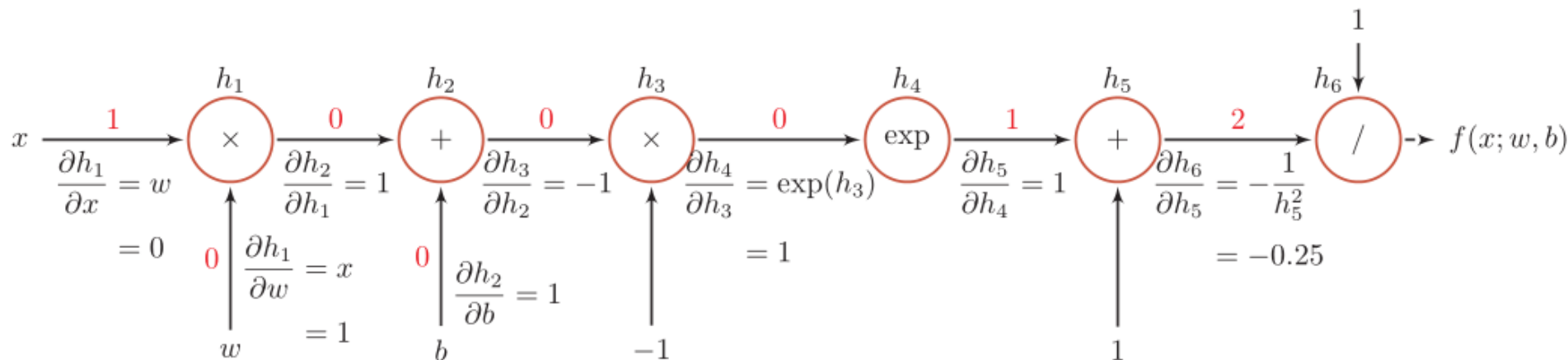
北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY

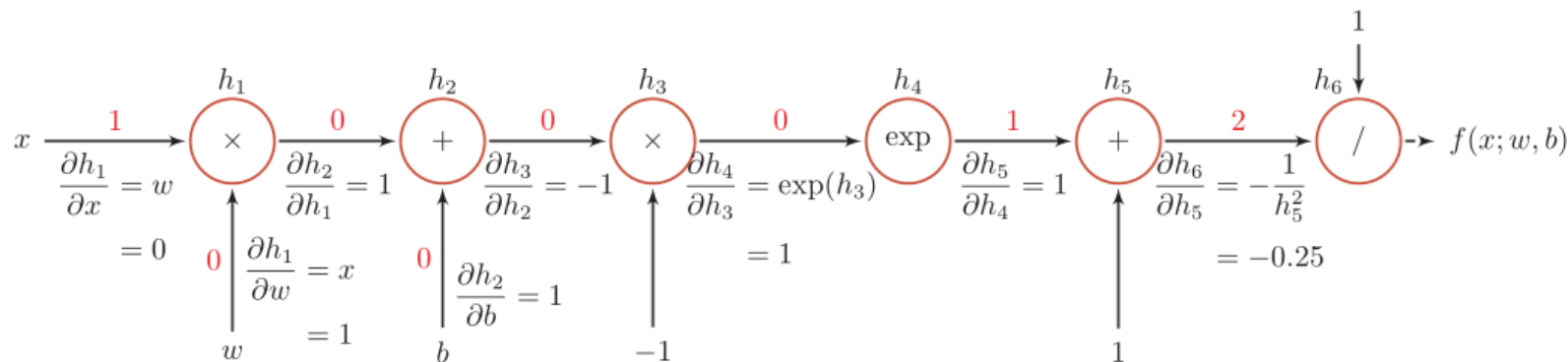
计算图与自动微分

- 自动微分是利用链式法则来自动计算一个复合函数的梯度。

$$f(x; w, b) = \frac{1}{\exp(-(wx + b)) + 1}.$$

- 计算图





函数	导数	
$h_1 = x \times w$	$\frac{\partial h_1}{\partial w} = x$	$\frac{\partial h_1}{\partial x} = w$
$h_2 = h_1 + b$	$\frac{\partial h_2}{\partial h_1} = 1$	$\frac{\partial h_2}{\partial b} = 1$
$h_3 = h_2 \times -1$	$\frac{\partial h_3}{\partial h_2} = -1$	
$h_4 = \exp(h_3)$	$\frac{\partial h_4}{\partial h_3} = \exp(h_3)$	
$h_5 = h_4 + 1$	$\frac{\partial h_5}{\partial h_4} = 1$	
$h_6 = 1/h_5$	$\frac{\partial h_6}{\partial h_5} = -\frac{1}{h_5^2}$	

当 $x = 1, w = 0, b = 0$ 时，可以得到

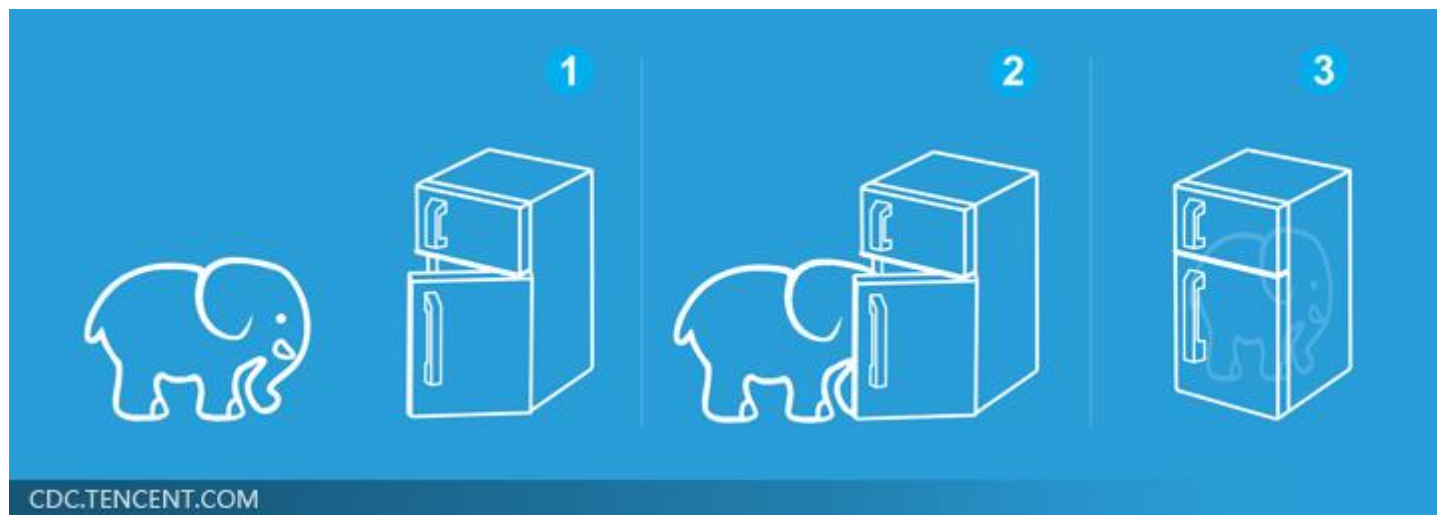
$$\begin{aligned}
 \frac{\partial f(x; w, b)}{\partial w} \Big|_{x=1, w=0, b=0} &= \frac{\partial f(x; w, b)}{\partial h_6} \frac{\partial h_6}{\partial h_5} \frac{\partial h_5}{\partial h_4} \frac{\partial h_4}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial w} \\
 &= 1 \times -0.25 \times 1 \times 1 \times -1 \times 1 \times 1 \\
 &= 0.25.
 \end{aligned}$$

- ▶ 前向模式和反向模式
 - ▶ 反向模式和反向传播的计算梯度的方式相同
- ▶ 如果函数和参数之间有多条路径，可以将这多条路径上的导数再进行相加，得到最终的梯度。

- ▶ 前馈神经网络的训练过程可以分为以下三步
 - ▶ 前向计算每一层的状态和激活值，直到最后一层
 - ▶ 反向计算每一层的参数的偏导数
 - ▶ 更新参数



Deep Learning is so simple





北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY

优化问题

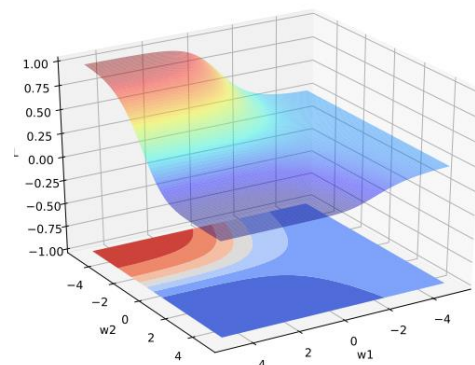
► 难点

- 参数过多，影响训练
- 非凸优化问题：即存在局部最优而非全局最优解，影响迭代
- 梯度消失问题，下层参数比较难调
- 参数解释起来比较困难

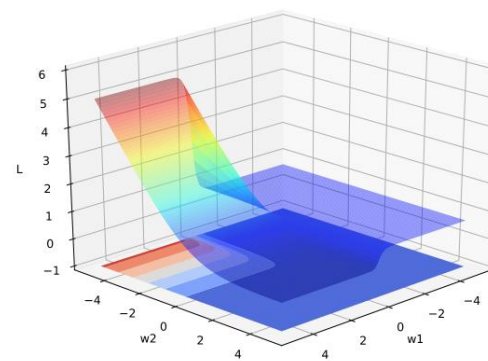
► 需求

- 计算资源要大
- 数据要多
- 算法效率要好：即收敛快

► 非凸优化问题



(a) 平方误差损失



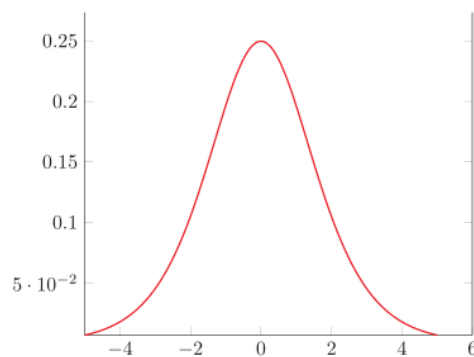
(b) 交叉熵损失

图 4.9 神经网络 $y = \sigma(w_2\sigma(w_1x))$ 的损失函数

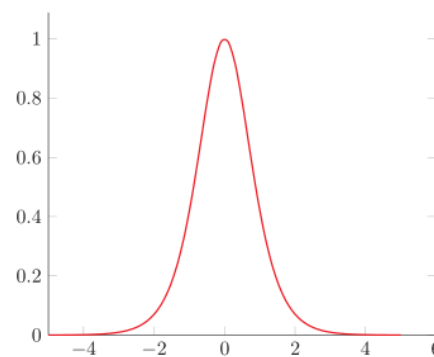
► 梯度消失问题 (Vanishing Gradient Problem)

$$y = f^5(f^4(f^3(f^2(f^1(x)))))$$

$$\frac{\partial y}{\partial x} = \frac{\partial f^1}{\partial x} \frac{\partial f^2}{\partial f^1} \frac{\partial f^3}{\partial f^2} \frac{\partial f^4}{\partial f^3} \frac{\partial f^5}{\partial f^4}$$



(a) logistic 函数的导数



(b) tanh 函数的导数

德以明理 学以精工

谢谢



智能感知

日期：2025年8月

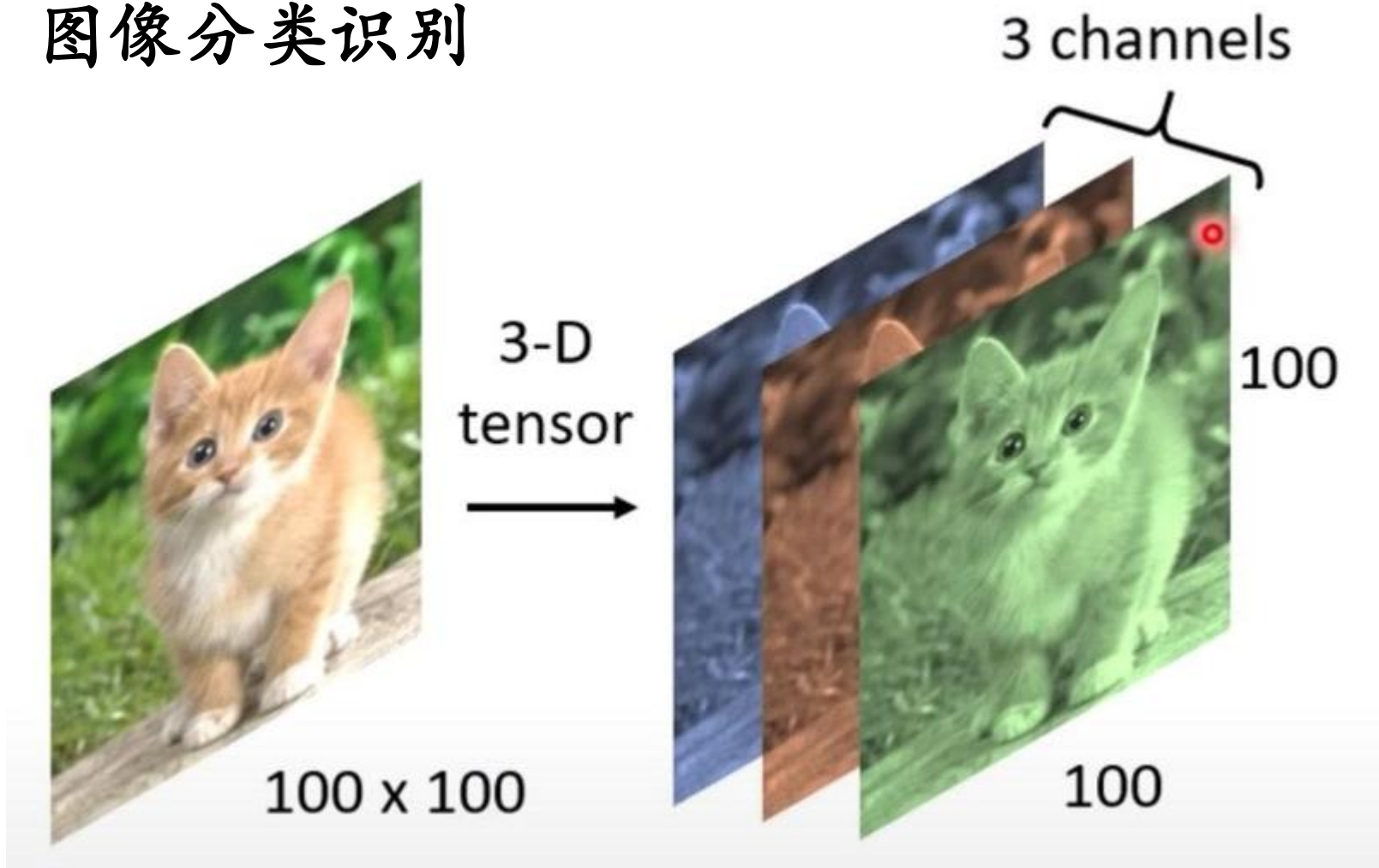


目录

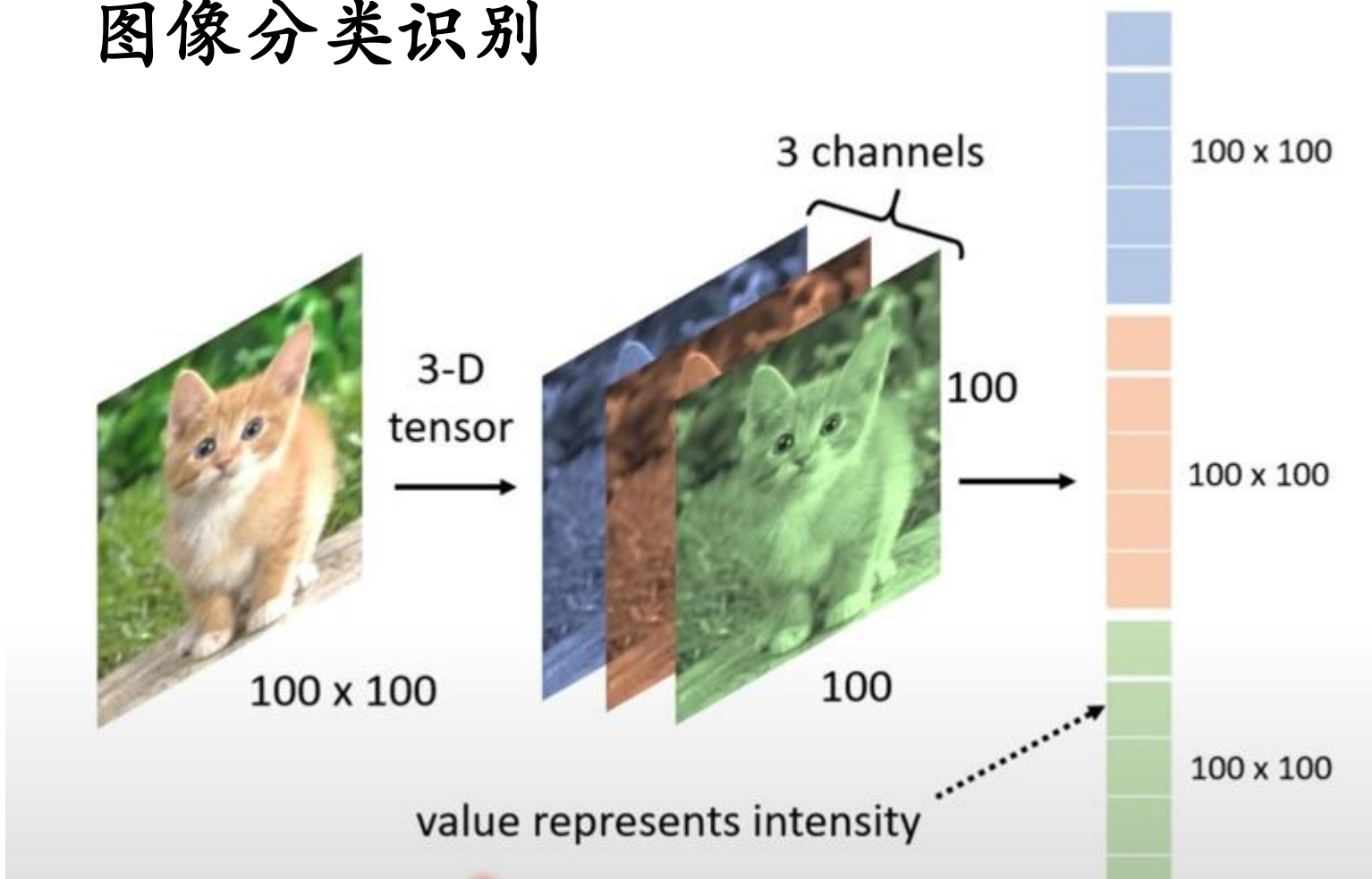
CONTENTS

- 1 绪论
- 2 机器学习基础
- 3 线性模型
- 4 前馈神经网络
- 5 卷积神经网络

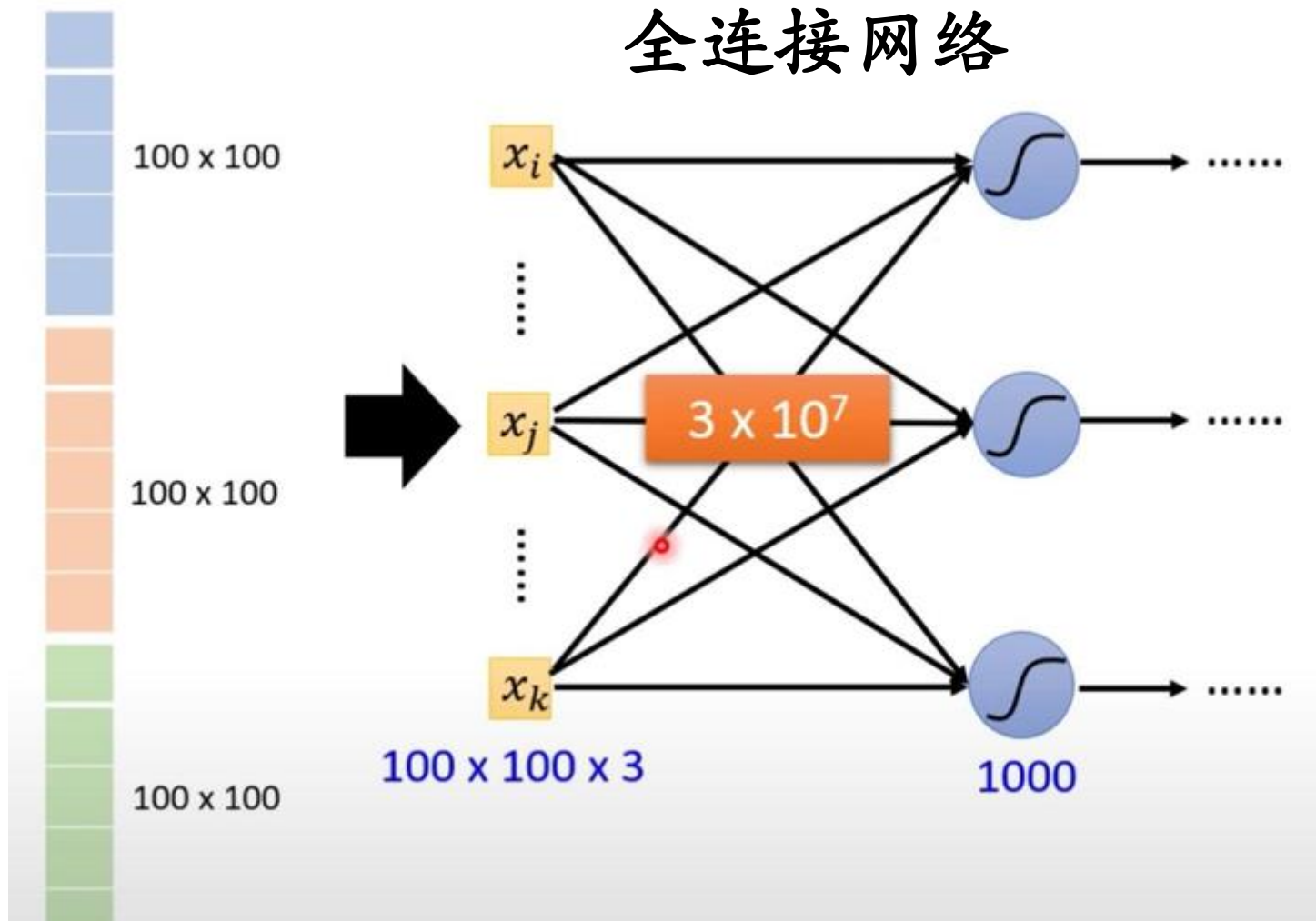
图像分类识别



图像分类识别

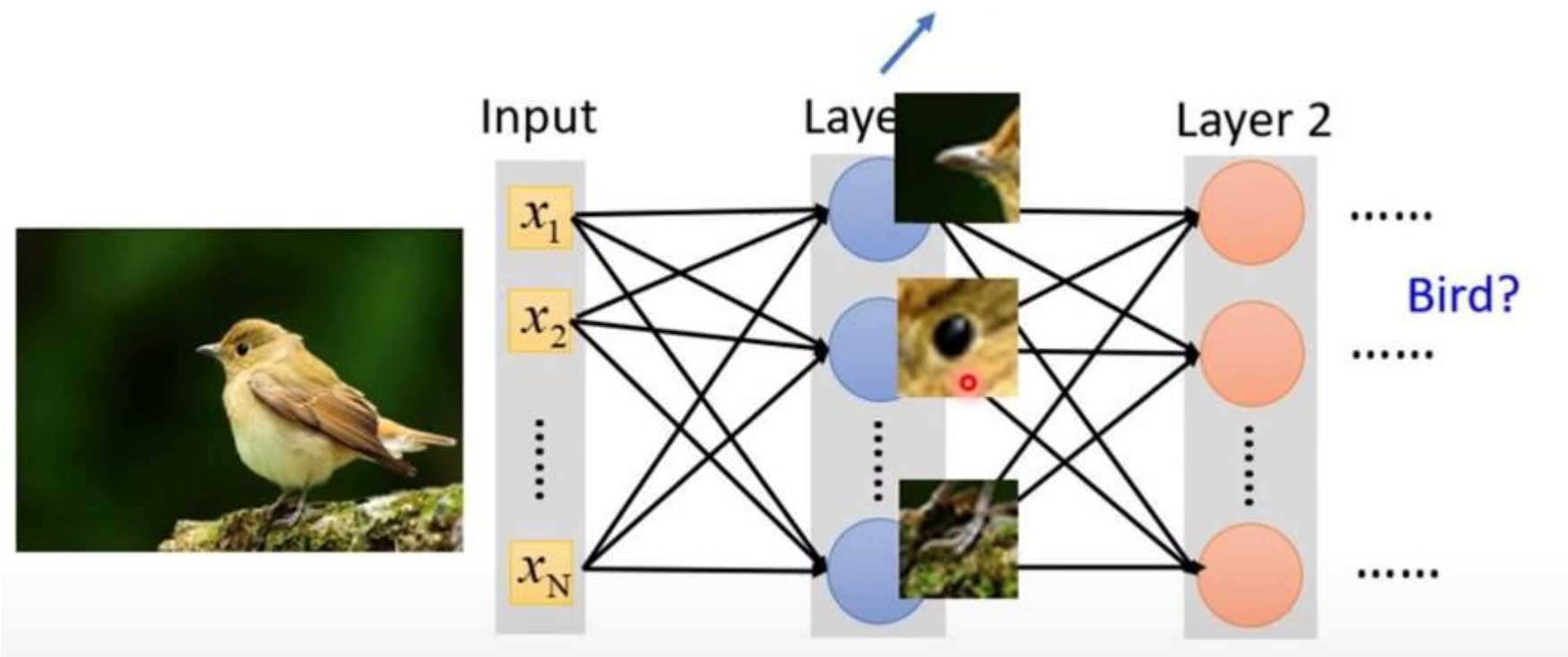


全连接网络



参数量巨大

识别关键特征图案



局部不变特征

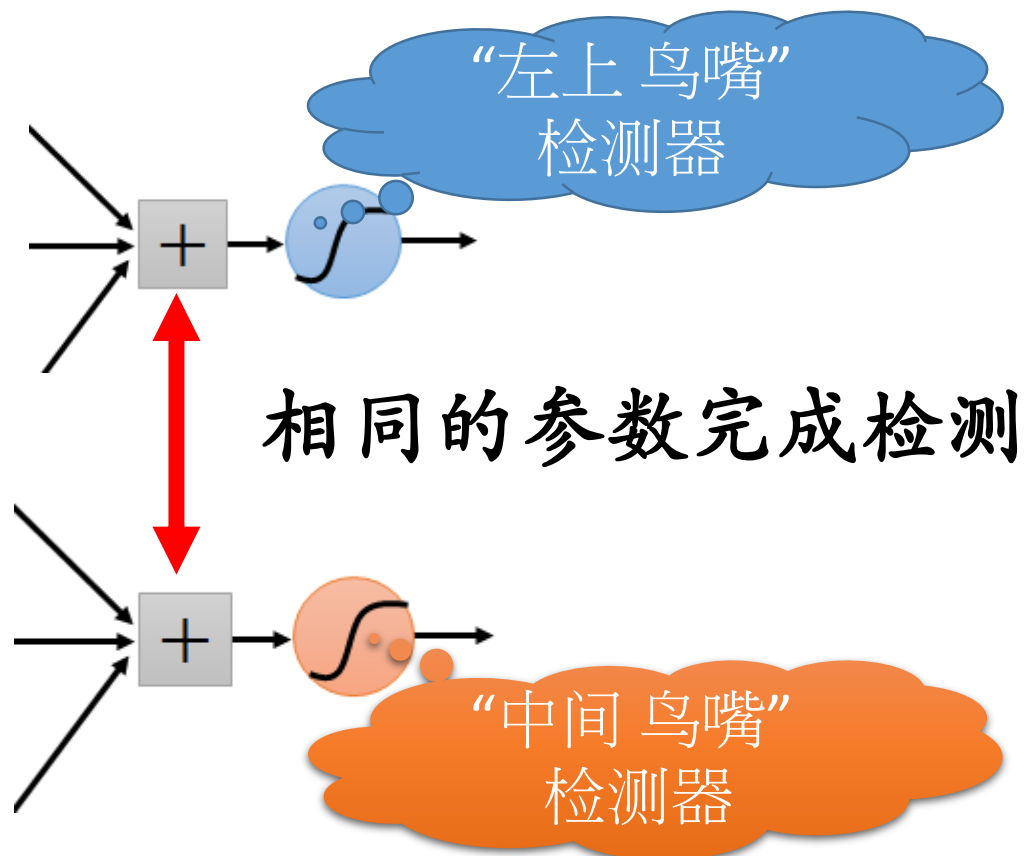
- 目标图像比整个图小得多

神经网络不必看到整个图像来识别图像类别。

连接到较小区域，所需权重参数少



- 同样的图案出现在不同的区域



- 对像素进行抽样不会改变图像类别

bird



抽样

bird



对像素进行抽样以使图像变小



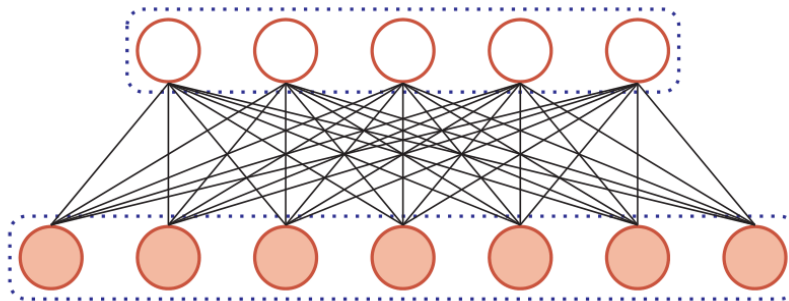
网络处理图像的参数更少



北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY

卷积神经网络

▶ 权重矩阵的参数非常多



▶ 局部不变性特征

- ▶ 自然图像中的物体都具有局部不变性特征
 - ▶ 尺度缩放、平移、旋转等操作不影响其语义信息。
- ▶ 全连接前馈网络很难提取这些局部不变特征

- ▶ **卷积神经网络** (Convolutional Neural Networks, CNN)
 - ▶ 一种前馈神经网络
 - ▶ 受生物学上感受野 (Receptive Field) 的机制而提出的
 - ▶ 在视觉神经系统中，一个神经元的**感受野**是指视网膜上的特定区域，只有这个区域内的刺激才能够激活该神经元。
- ▶ **卷积神经网络有三个结构上的特性：**
 - ▶ 局部连接
 - ▶ 权重共享
 - ▶ 空间或时间上的次采样

- ▶ 卷积经常用在信号处理中，用于计算信号的延迟累积。
- ▶ 假设一个信号发生器每个时刻 t 产生一个信号 x_t ，其信息的衰减率为 w_k ，即在 $k-1$ 个时间步长后，信息为原来的 w_k 倍
- ▶ 假设 $w_1 = 1, w_2 = 1/2, w_3 = 1/4$
- ▶ 时刻 t 收到的信号 y_t 为当前时刻产生的信息和以前时刻延迟信息的叠加

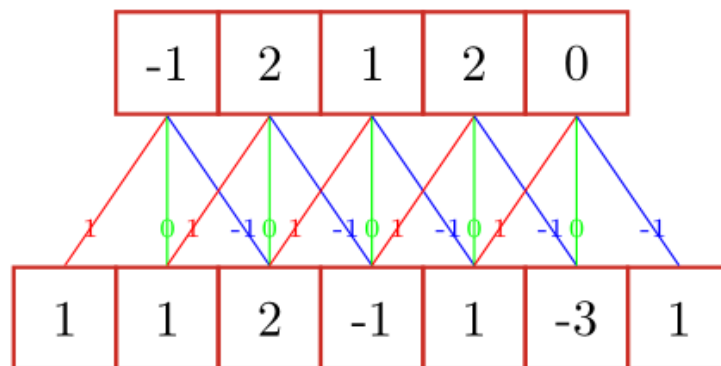
$$\begin{aligned} y_t &= 1 \times x_t + 1/2 \times x_{t-1} + 1/4 \times x_{t-2} \\ &= w_1 \times x_t + w_2 \times x_{t-1} + w_3 \times x_{t-2} \\ &= \sum_{k=1}^3 w_k \cdot x_{t-k+1} \end{aligned}$$

滤波器 (filter) 或卷积核 (convolution kernel)

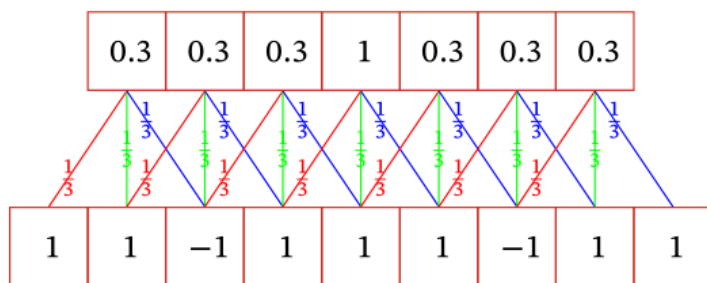
► 给定一个输入信号序列 x 和滤波器 w ,卷积的输出为:

$$y_t = \sum_{k=1}^K w_k x_{t-k+1}$$

Filter: [-1,0,1]

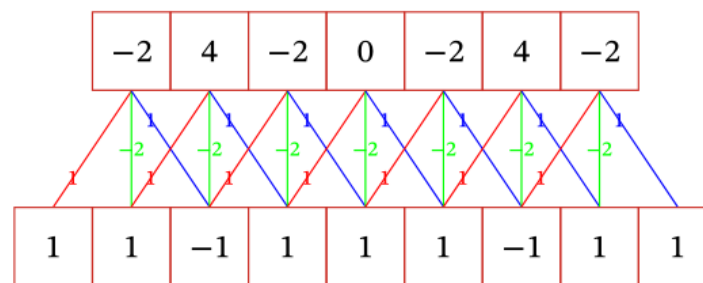


不同的滤波器来提取信号序列中的不同特征



(a) 滤波器 $[1/3, 1/3, 1/3]$

低频信息



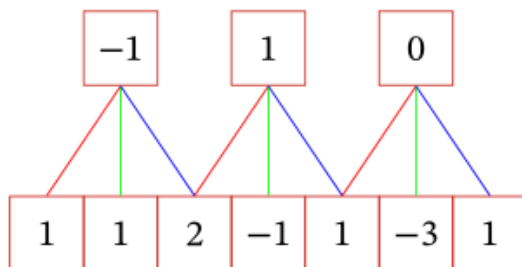
(b) 滤波器 $[1, -2, 1]$

高频信息

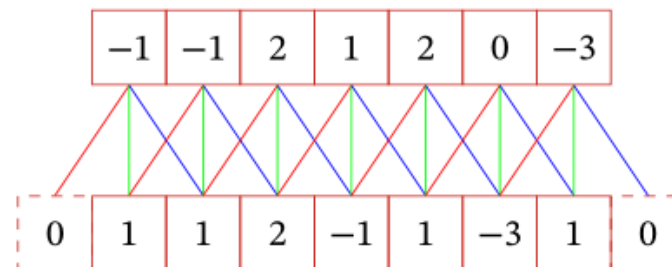
$$y''(u) = y(u + 1) + y(u - 1) - 2y(u)$$

二阶微分

► 引入滤波器的滑动步长 S 和零填充 P



(a) 步长 $S = 2$



(b) 零填充 $P = 1$

► 卷积的结果按输出长度不同可以分为三类：

- **窄卷积**：步长 $T = 1$ ，两端不补零 $P = 0$ ，卷积后输出长度为 $M - K + 1$
 - **宽卷积**：步长 $T = 1$ ，两端补零 $P = K - 1$ ，卷积后输出长度 $M + K - 1$
 - **等宽卷积**：步长 $T = 1$ ，两端补零 $P = (K - 1)/2$ ，卷积后输出长度 M
-
- 在早期的文献中，卷积一般默认为**窄卷积**。
 - 而目前的文献中，卷积一般默认为**等宽卷积**。

- 在图像处理中，图像是以二维矩阵的形式输入到神经网络中，因此我们需要二维卷积。

一个输入信息 $\mathbf{X}$ 和滤波器 $\mathbf{W}$ 的二维卷积定义为

$$\mathbf{Y} = \mathbf{W} * \mathbf{X},$$

$$y_{ij} = \sum_{u=1}^U \sum_{v=1}^V w_{uv} x_{i-u+1, j-v+1}.$$

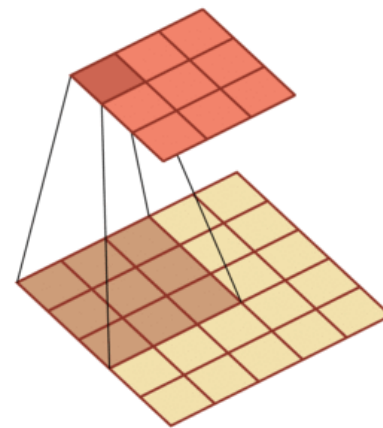
1	1	1	1	1
-1	0	-3	0	1
2	1	1	-1	0
0	-1	1	2	1
1	2	1	1	1

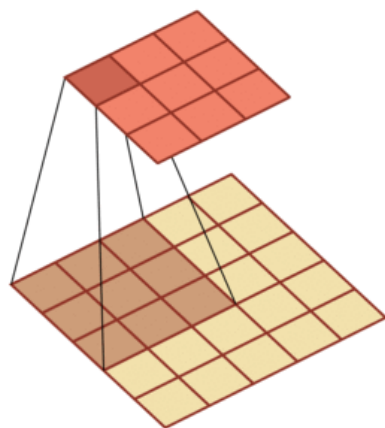
*

1	0	0
0	0	0
0	0	-1

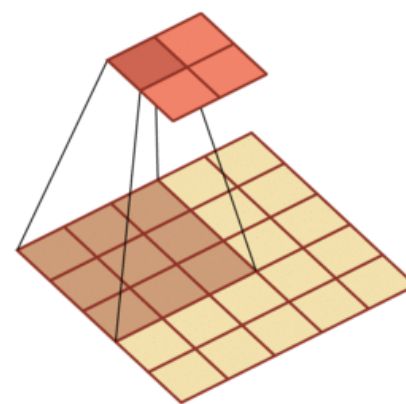
=

0	-2	-1
2	2	4
-1	0	0

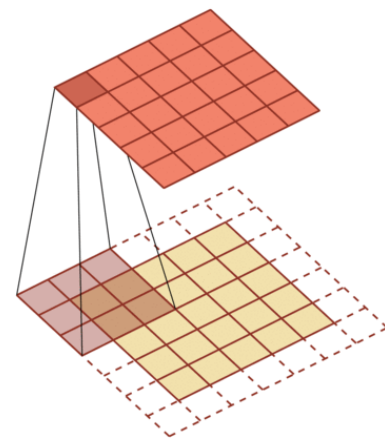




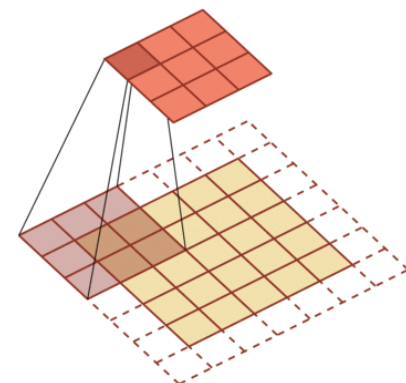
步长1，零填充0



步长2，零填充0

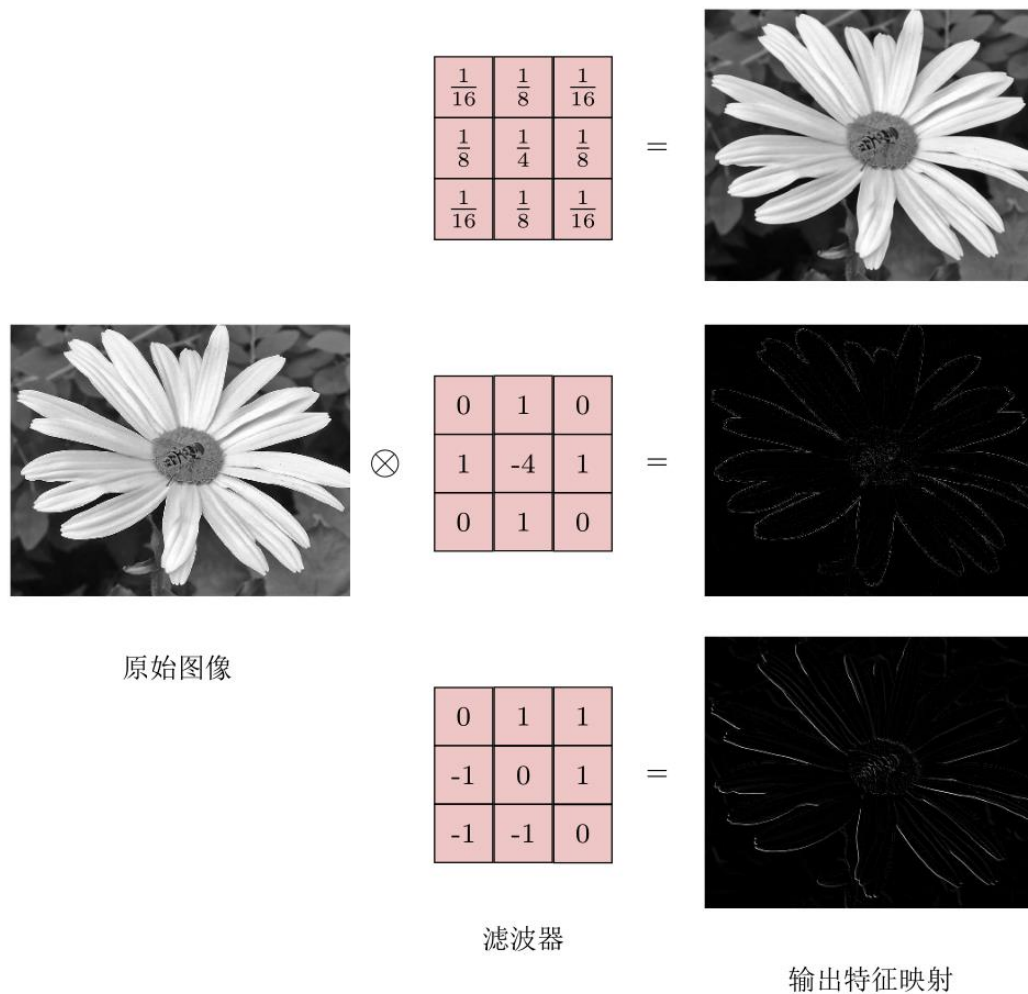


步长1，零填充1

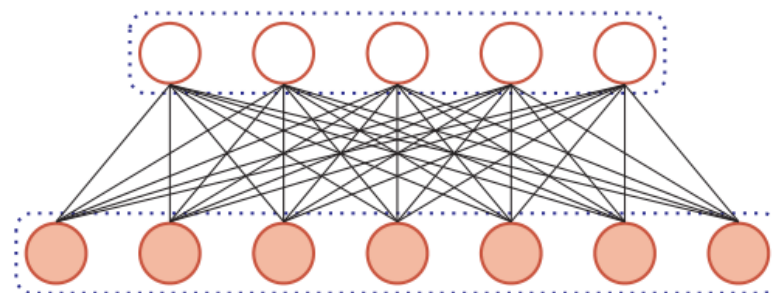


步长2，零填充1

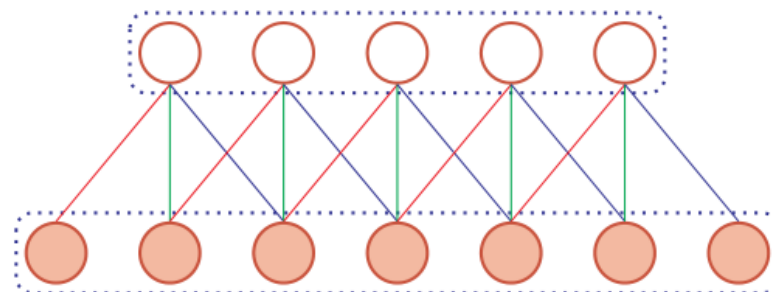
卷积作为特征提取器



► 用卷积层代替全连接层



(a) 全连接层



(b) 卷积层

- ▶ 计算卷积需要进行卷积核翻转。
- ▶ 卷积操作的目标：提取特征。

翻转是不必要的！

▶ 互相关

$$y_{ij} = \sum_{u=1}^m \sum_{v=1}^n w_{uv} \cdot x_{i+u-1, j+v-1}$$

除非特别声明，卷积一般指“互相关”。

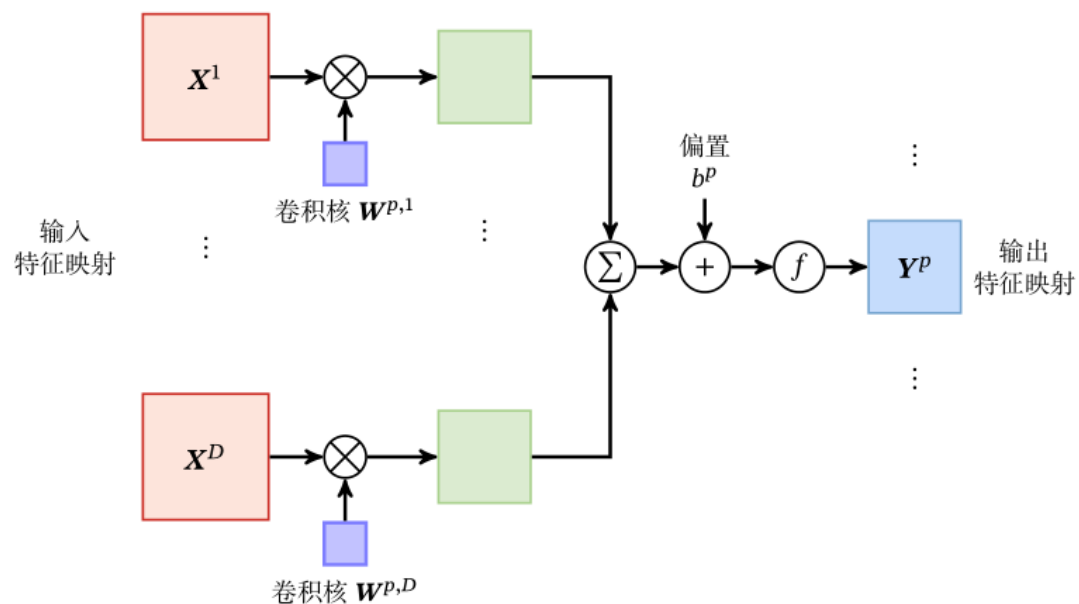
▶ **特征映射 (Feature Map)** : 图像经过卷积后得到的特征。

▶ 卷积核看成一个特征提取器

▶ **卷积层**

▶ 输入: D 个特征映射 $M \times N \times D$

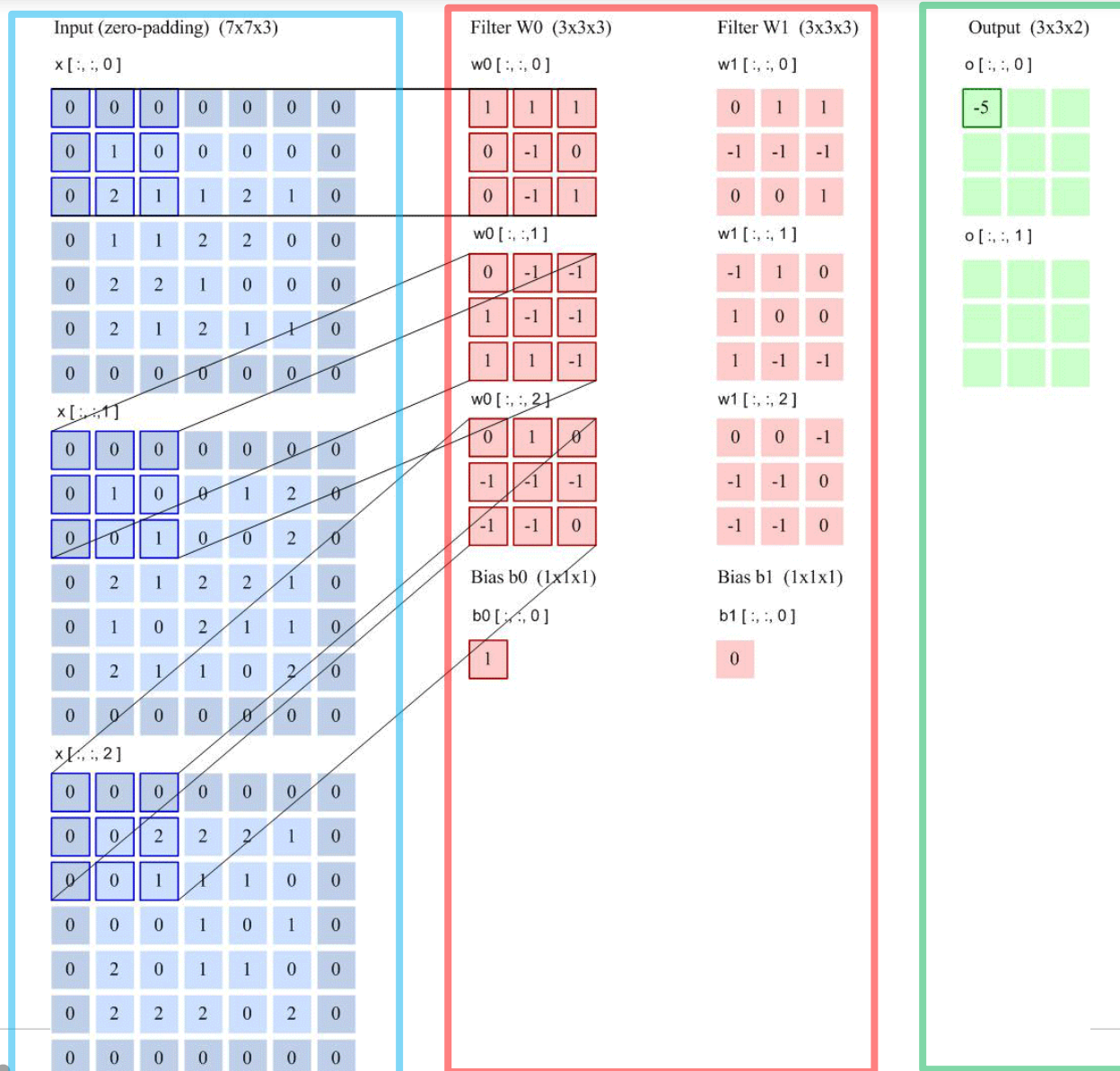
▶ 输出: P 个特征映射 $M' \times N' \times P$



$$Z^p = W^p \otimes X + b^p = \sum_{d=1}^D w^{p,d} \otimes X^d + b^p,$$

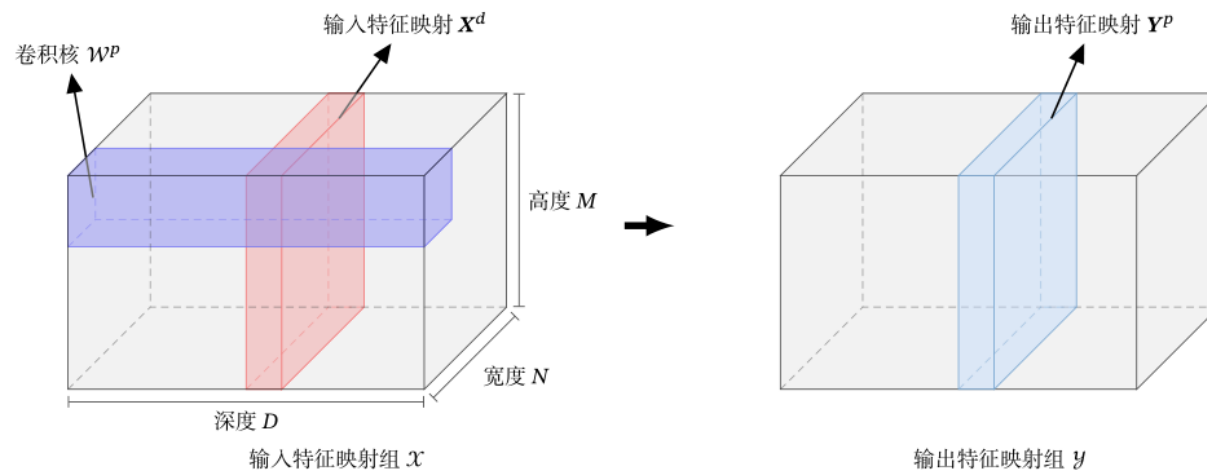
$$Y^p = f(Z^p).$$

卷积层的映射关系



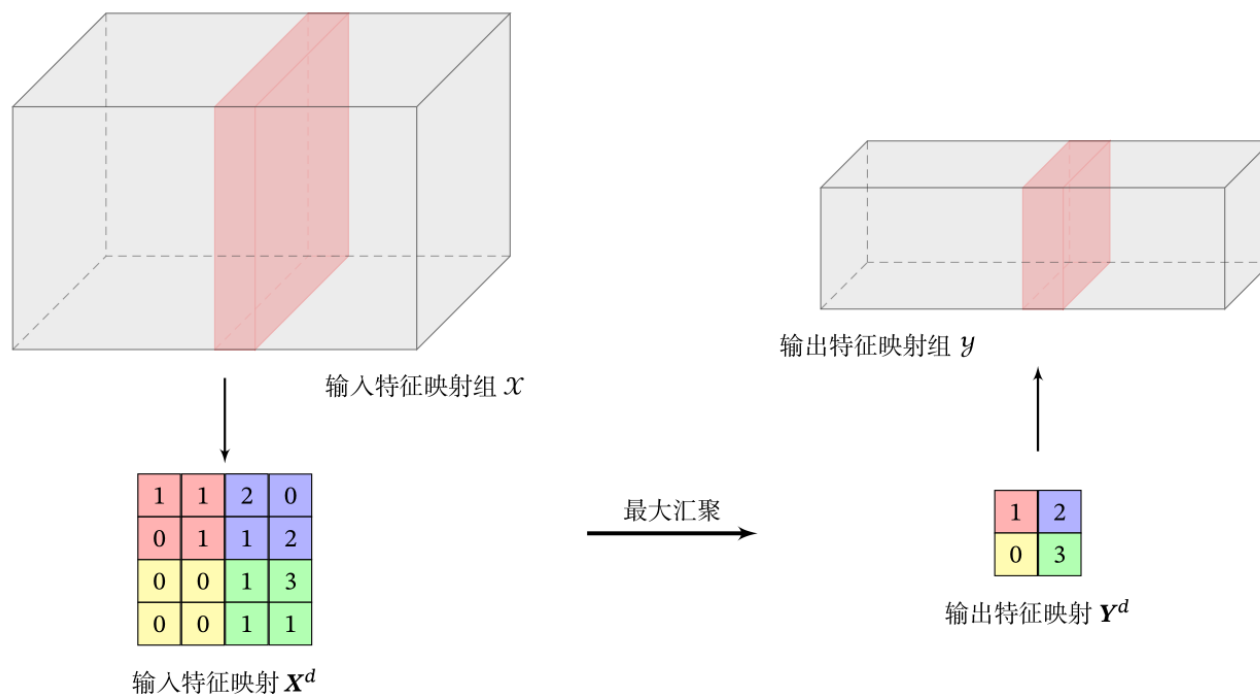
步长2
filter 3*3
filter个数6
零填充1

► 典型的卷积层为3维结构

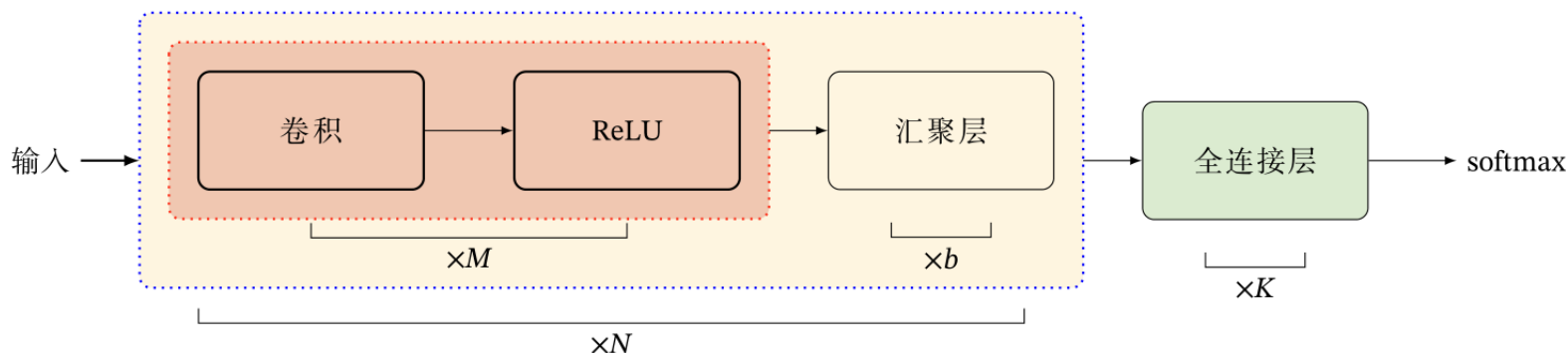


$$Z^p = W^p \otimes X + b^p = \sum_{d=1}^D W^{p,d} \otimes X^d + b^p,$$
$$Y^p = f(Z^p).$$

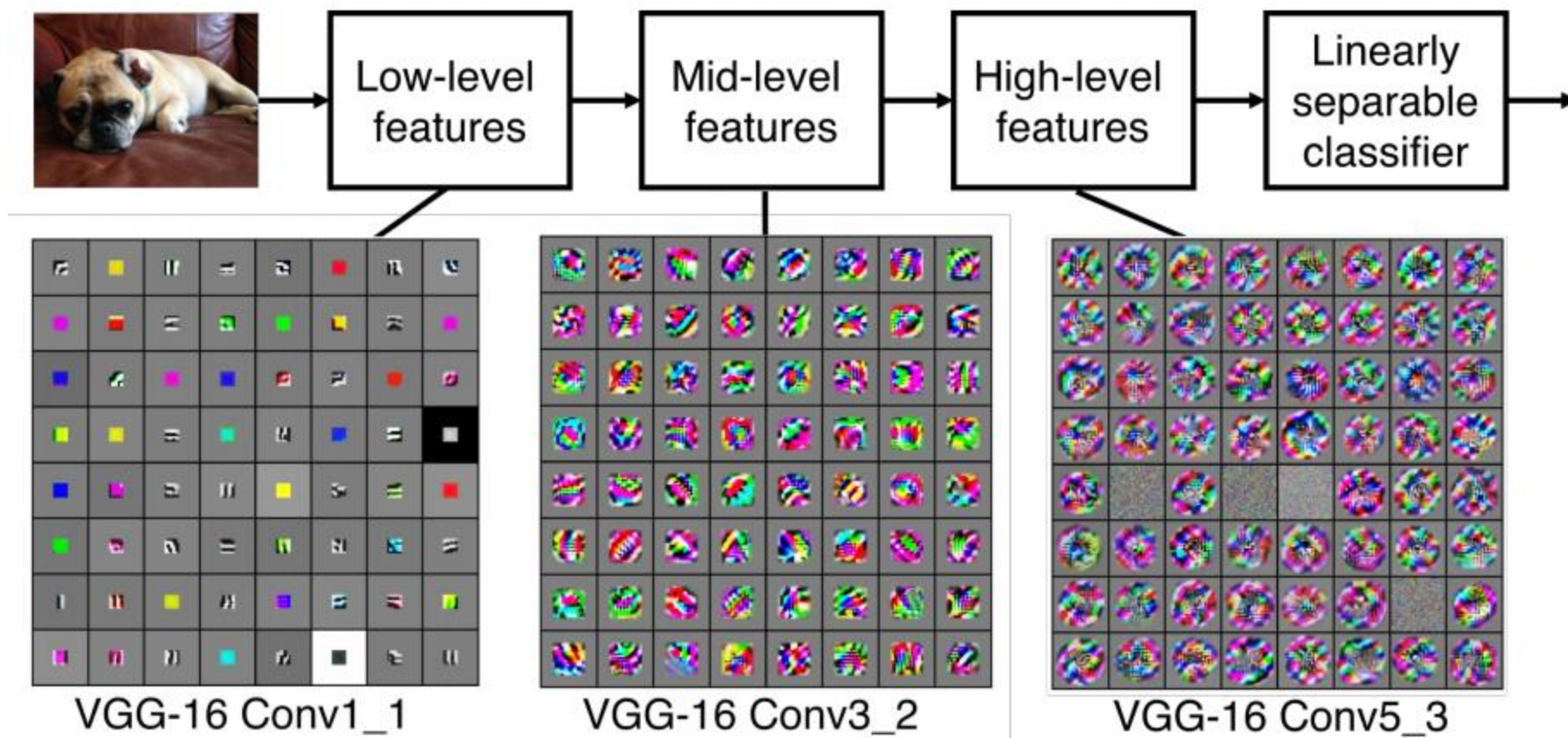
- 卷积层虽然可以显著减少连接的个数，但是每一个特征映射的神经元个数并没有显著减少。



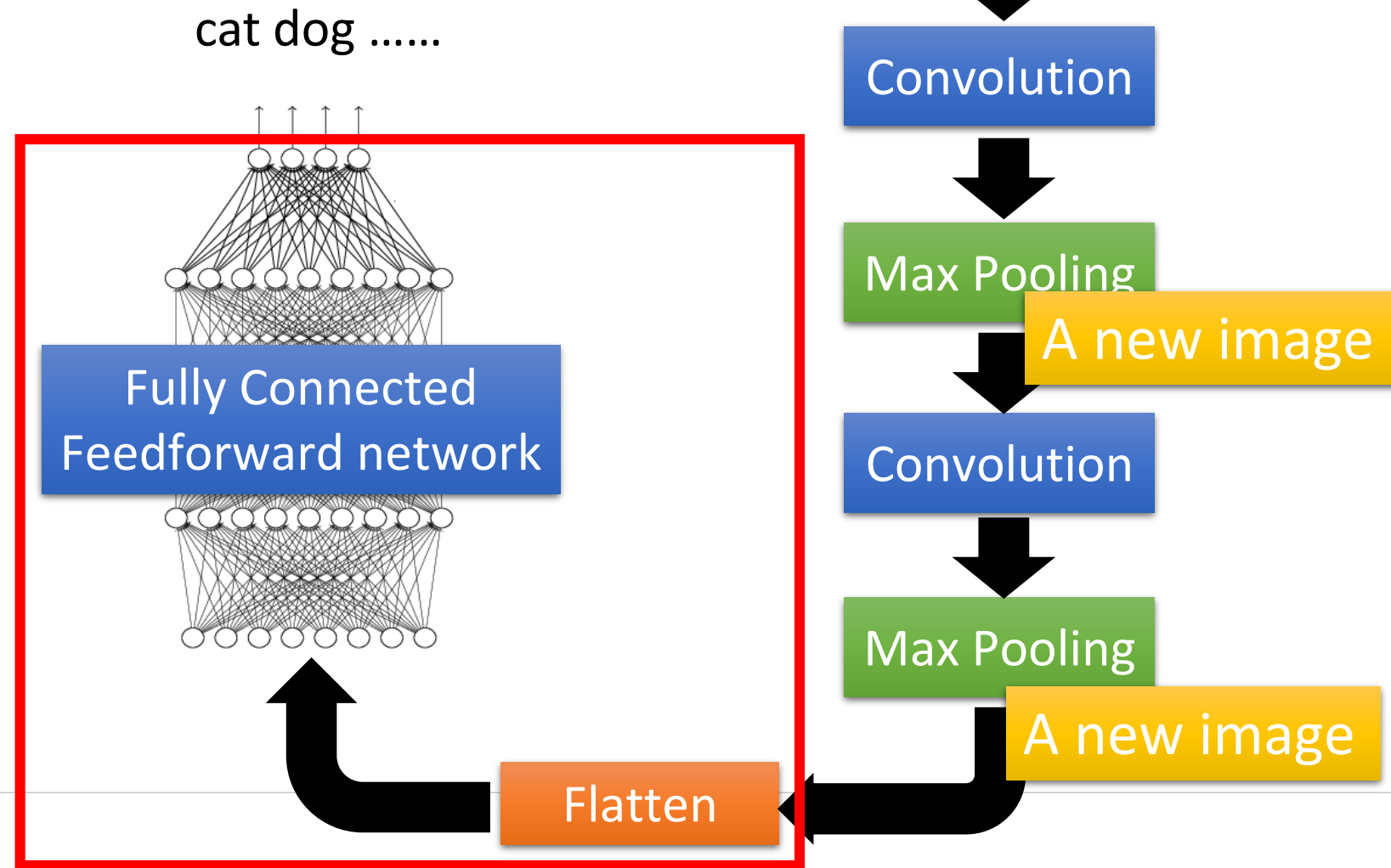
- ▶ 卷积网络是由卷积层、汇聚层、全连接层交叉堆叠而成。
- ▶ 趋向于小卷积、大深度
- ▶ 趋向于全卷积
- ▶ 典型结构



- ▶ 一个卷积块为连续 M 个卷积层和 b 个汇聚层 (M 通常设置为 $2 \sim 5$, b 为0或1)。一个卷积网络中可以堆叠 N 个连续的卷积块, 然后在接着 K 个全连接层 (N 的取值区间比较大, 比如 $1 \sim 100$ 或者更大; K 一般为 $0 \sim 2$)。



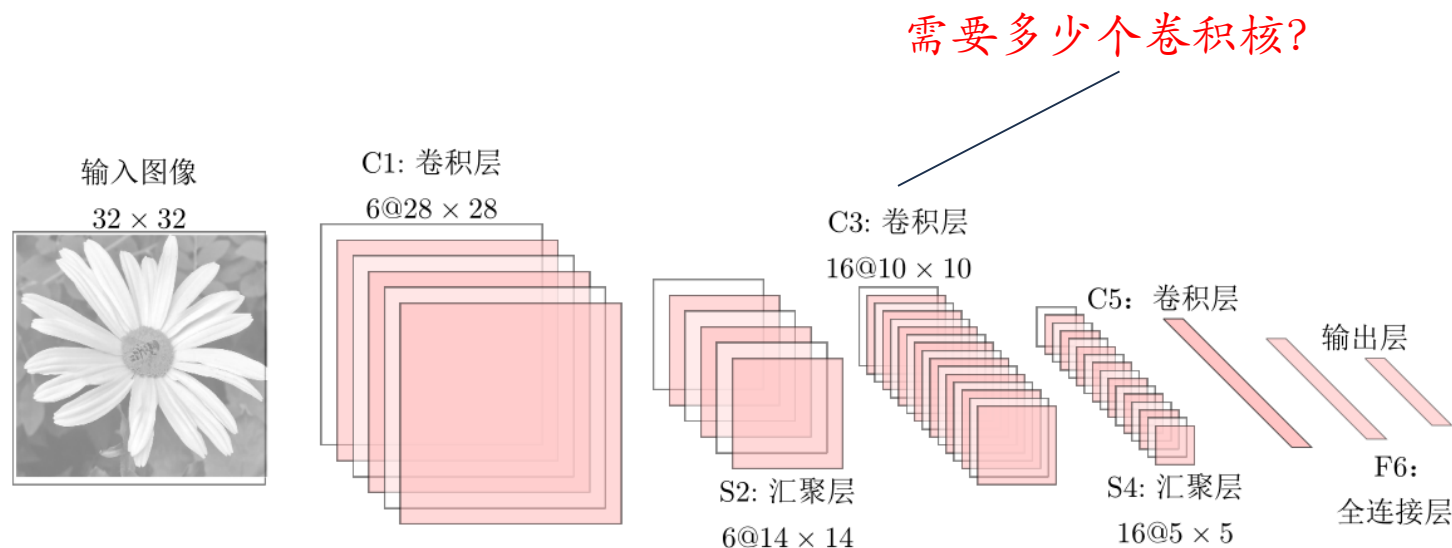
完整的卷积神经网络





其它卷积种类

- ▶ LeNet-5 是一个非常成功的神经网络模型。
- ▶ 基于 LeNet-5 的手写数字识别系统在 90 年代被美国很多银行使用，用来识别支票上面的手写数字。
- ▶ LeNet-5 共有 7 层。



Large Scale Visual Recognition Challenge



2012 Teams	%error	2013 Teams	%error	2014 Teams	%error
Supervision (Toronto)	15.3	Clarifai (NYU spinoff)	11.7	GoogLeNet	6.6
ISI (Tokyo)	26.1	NUS (singapore)	12.9	VGG (Oxford)	7.3
VGG (Oxford)	26.9	Zeiler-Fergus (NYU)	13.5	MSRA	8.0
XRCE/INRIA	27.0	A. Howard	13.5	A. Howard	8.1
UvA (Amsterdam)	29.6	OverFeat (NYU)	14.1	DeeperVision	9.5
INRIA/LEAR	33.4	UvA (Amsterdam)	14.2	NUS-BST	9.7
		Adobe	15.2	TTIC-ECP	10.2
		VGG (Oxford)	15.2	XYZ	11.2
		VGG (Oxford)	23.0	UvA	12.1

► 2012 ILSVRC winner

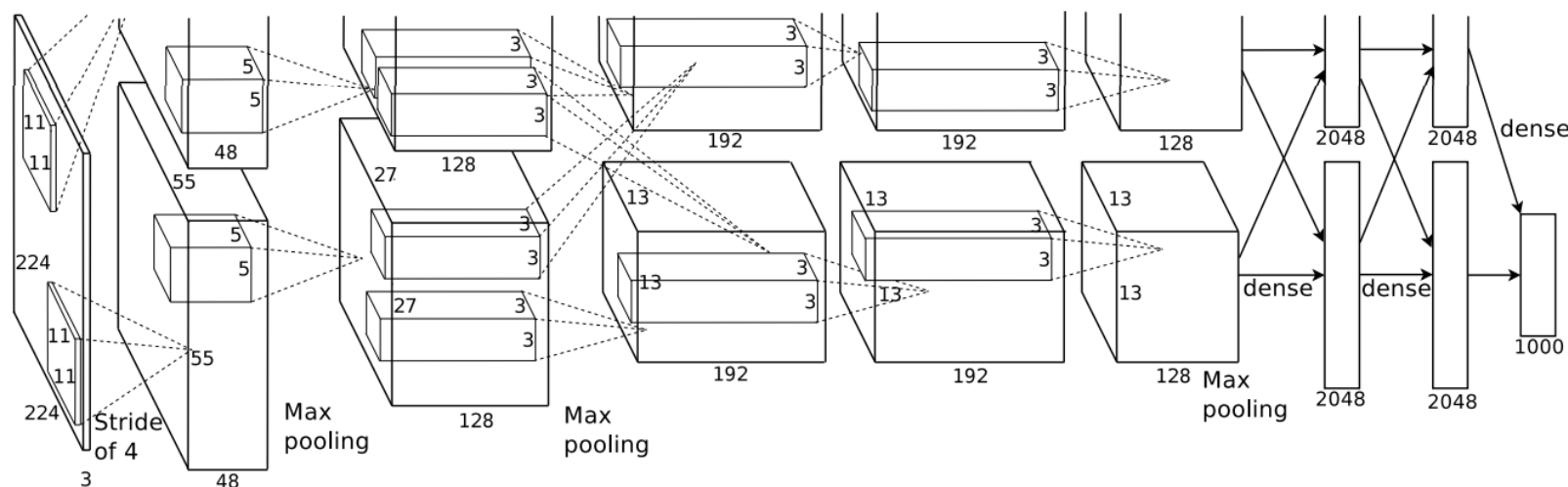
► (top 5 error of 16% compared to runner-up with 26% error)

► 第一个现代深度卷积网络模型

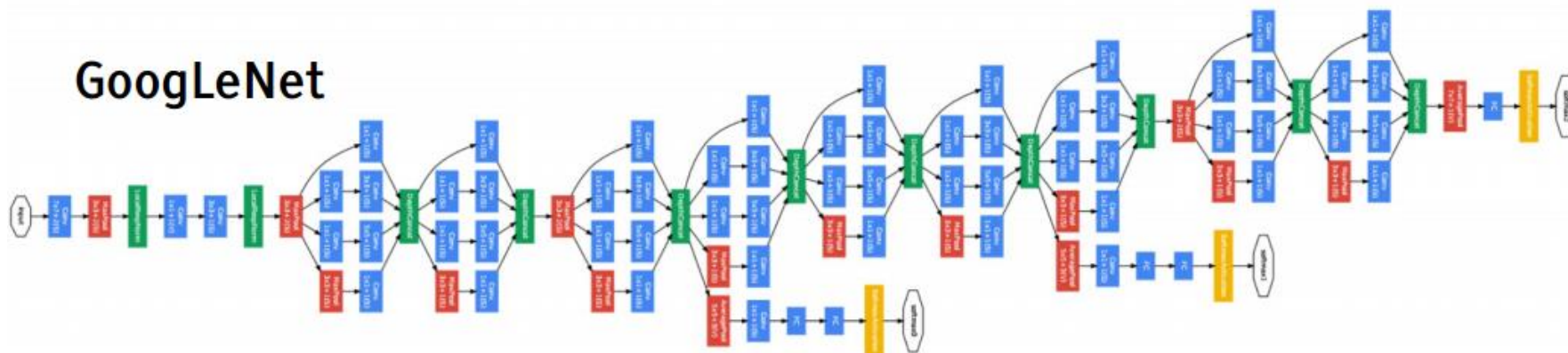
► 首次使用了很多现代深度卷积网络的一些技术方法

□ 使用GPU进行并行训练，采用了ReLU作为非线性激活函数，使用Dropout防止过拟合，使用数据增强

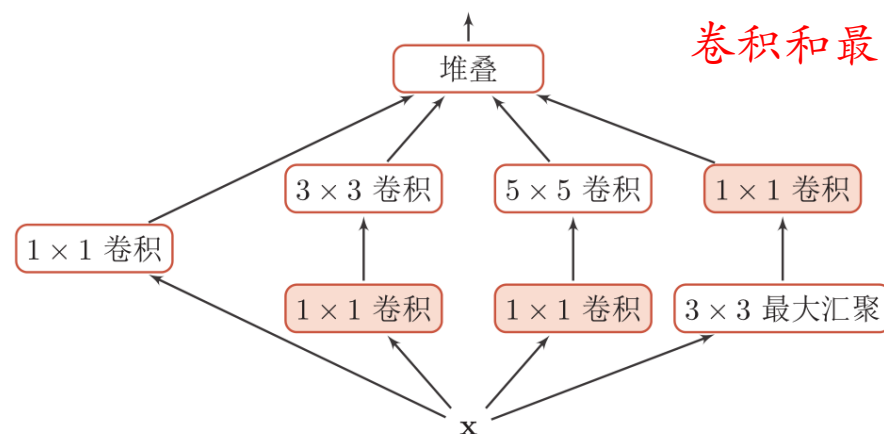
► 5个卷积层、3个汇聚层和3个全连接层



- ▶ 2014 ILSVRC winner (22层)
- ▶ 参数: GoogLeNet: 4M VS AlexNet: 60M
- ▶ 错误率: 6.7%
- ▶ Inception网络是由有多个inception模块和少量的汇聚层堆叠而成。



- ▶ 在卷积网络中，如何设置卷积层的卷积核大小是一个十分关键的问题。
- ▶ 在Inception网络中，一个卷积层包含多个不同大小的卷积操作，称为Inception模块。
- ▶ Inception模块同时使用 1×1 、 3×3 、 5×5 等不同大小的卷积核，并将得到的特征映射在深度上拼接（堆叠）起来作为输出特征映射。



卷积和最大汇聚都是等宽的。

- ▶ 用多层小卷积核替换大卷积核，以减少计算量和参数量。
- ▶ 使用两层 3×3 的卷积来替换v1中的 5×5 的卷积
- ▶ 使用连续的 $n \times 1$ 和 $1 \times n$ 来替换 $n \times n$ 的卷积。

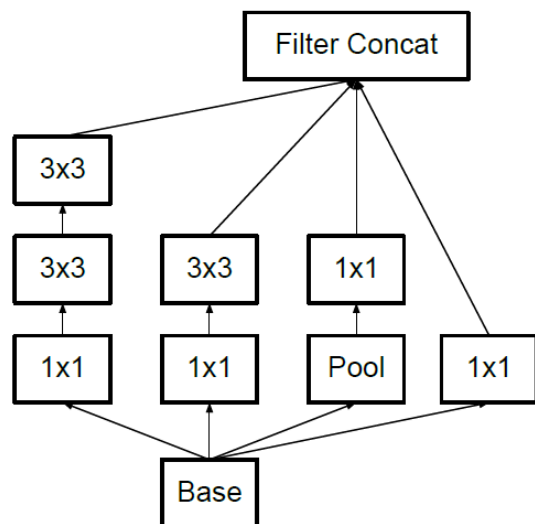


Figure 5. Inception modules where each 5×5 convolution is replaced by two 3×3 convolution, as suggested by principle 3 of Section 2. <http://blog.csdn.net/xbinworld>

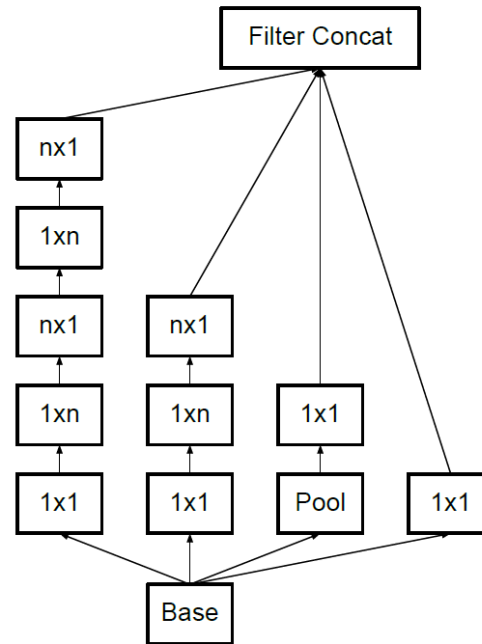
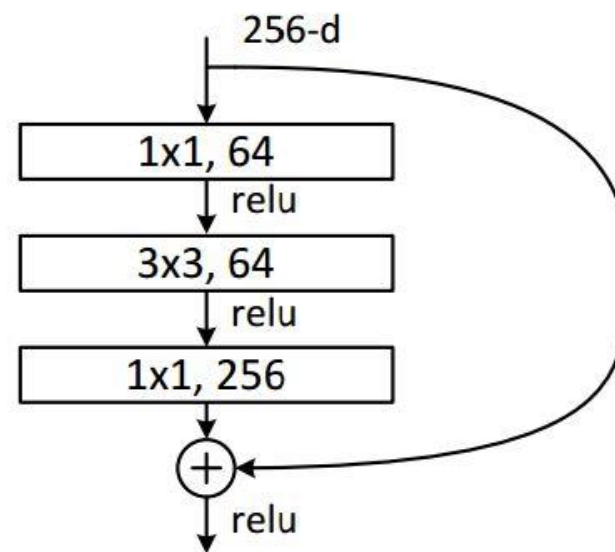
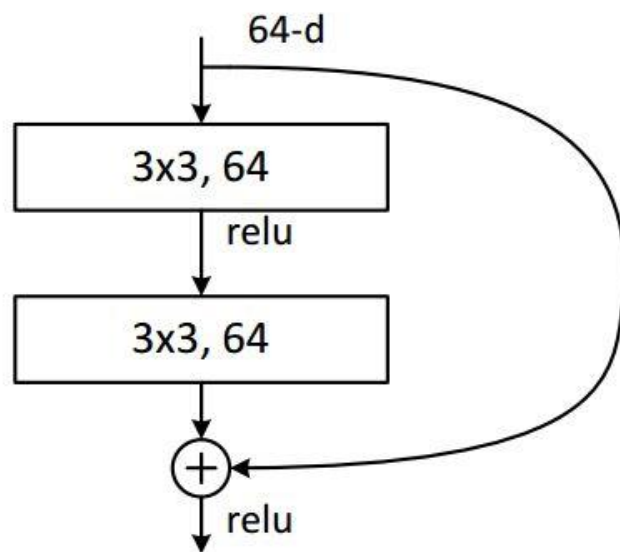
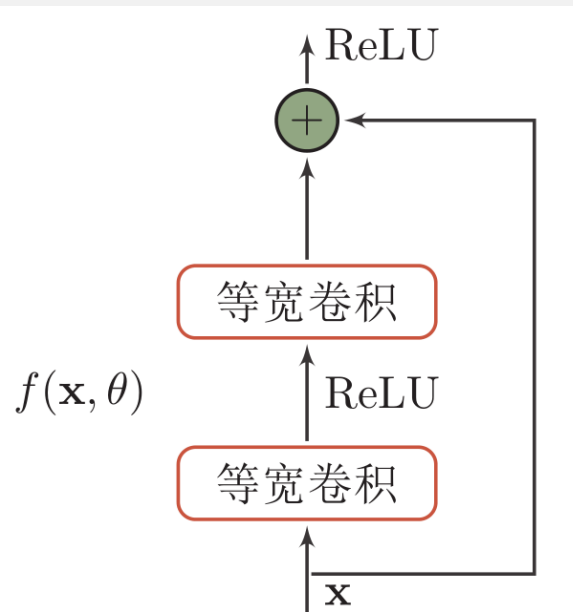


Figure 6. Inception modules after the factorization of the $n \times n$ convolutions. In our proposed architecture, we chose $n = 7$ for the 17×17 grid. (The filter sizes are picked using principle 3) <http://blog.csdn.net/xbinworld>

- ▶ 残差网络 (Residual Network, ResNet) 是通过给非线性的卷积层增加**直连边**的方式来提高信息的传播效率。
- ▶ 假设在一个深度网络中, 我们期望一个非线性单元 (可以为一层或多层的卷积层) $f(\mathbf{x}, \theta)$ 去逼近一个目标函数为 $h(\mathbf{x})$ 。
- ▶ 将目标函数拆分成两部分: **恒等函数** 和 **残差函数**

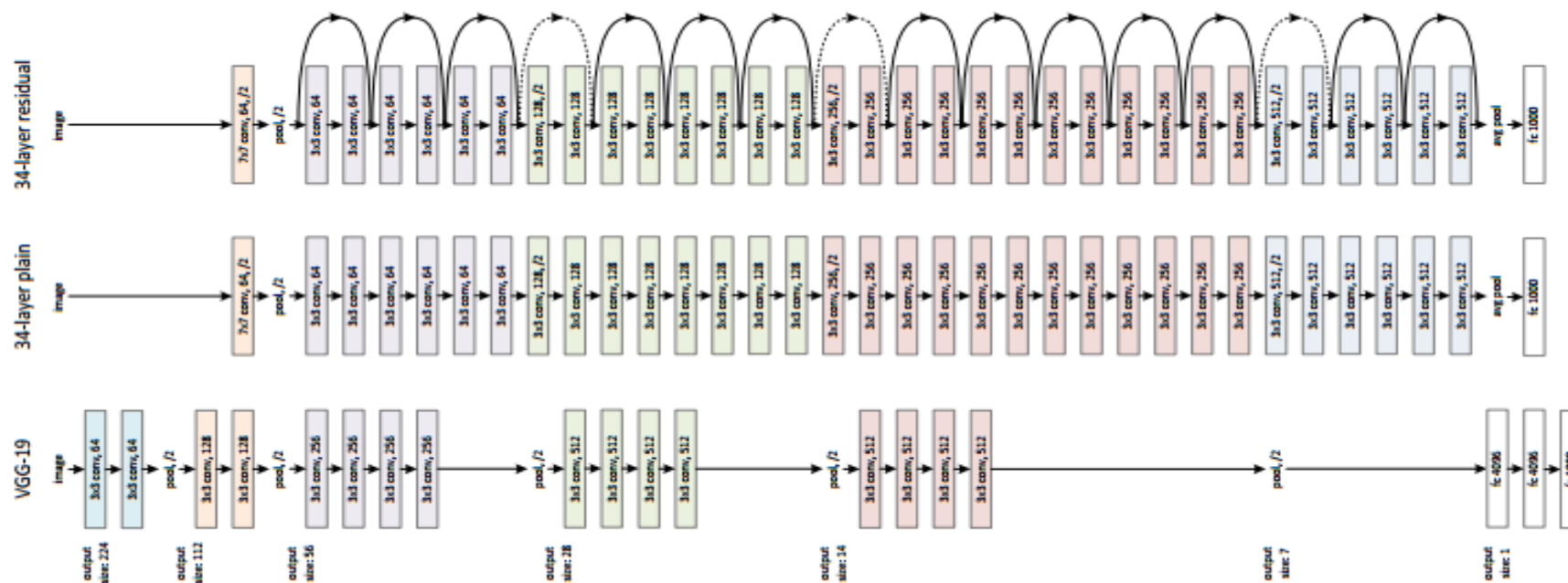
$$h(\mathbf{x}) = \underbrace{\mathbf{x}}_{\text{恒等函数}} + \underbrace{(h(\mathbf{x}) - \mathbf{x})}_{\text{残差函数}} \rightarrow f(\mathbf{x}, \theta)$$

残差单元



► 2015 ILSVRC winner (152层)

► 错误率: 3.57%





北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY

卷积神经网络应用

The input to the policy network is a $19 \times 19 \times 48$ image stack consisting of 48 feature planes. The first hidden layer zero pads the input into a 23×23 image, then convolves k filters of kernel size 5×5 with stride 1 with the input image and applies a rectifier nonlinearity. Each of the subsequent hidden layers 2 to 12 zero pads the respective previous hidden layer into a 21×21 image, then convolves k filters of kernel size 3×3 with stride 1, again followed by a rectifier nonlinearity. The final layer convolves 1 filter of kernel size 1×1 with stride 1, with a different bias for each position, and applies a softmax function. The match version of AlphaGo used $k = 192$ filters; Fig. 2b and Extended Data Table 3 additionally show the results of training with $k = 128, 256$ and 384 filters.

policy network:

[19x19x48] Input

CONV1: 192 5x5 filters , stride 1, pad 2 => [19x19x192]

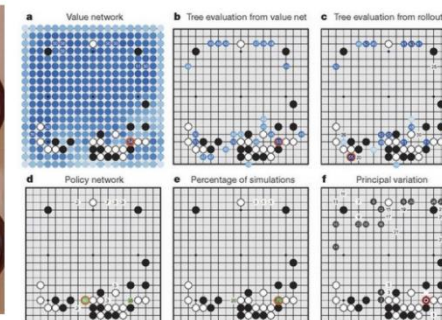
CONV2..12: 192 3x3 filters, stride 1, pad 1 => [19x19x192]

CONV: 1 1x1 filter, stride 1, pad 0 => [19x19] (*probability map of promising moves*)

分布式系统：1202 个CPU 和176 块GPU

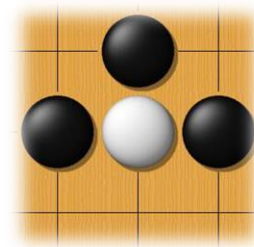
单机版：48 个CPU 和8 块GPU

走子速度：3 毫秒-2 微秒

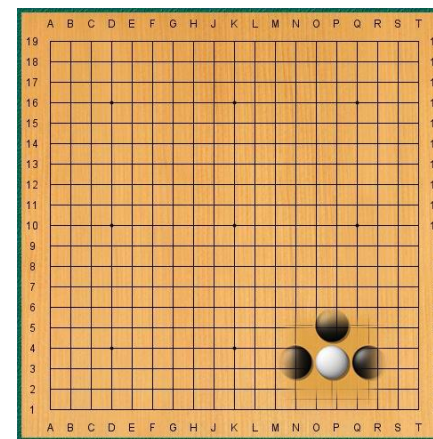
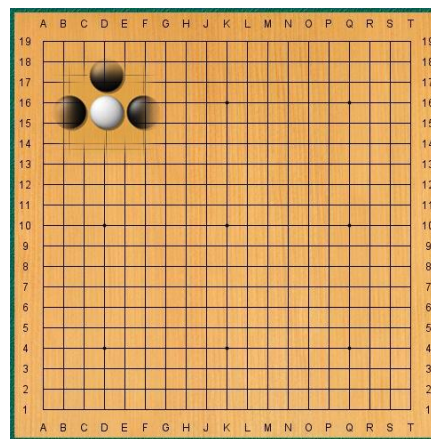


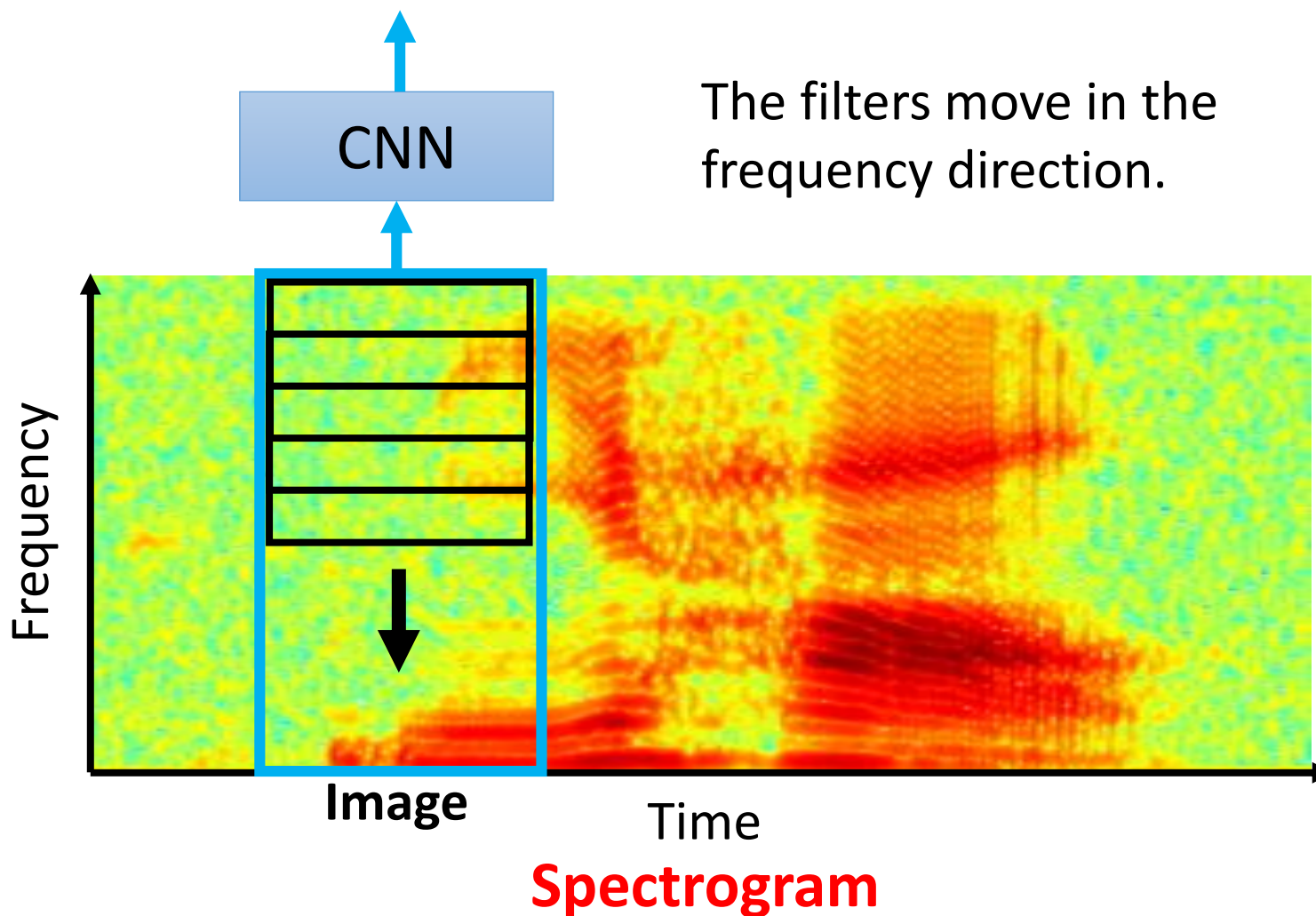
- Some patterns are much smaller than the whole image

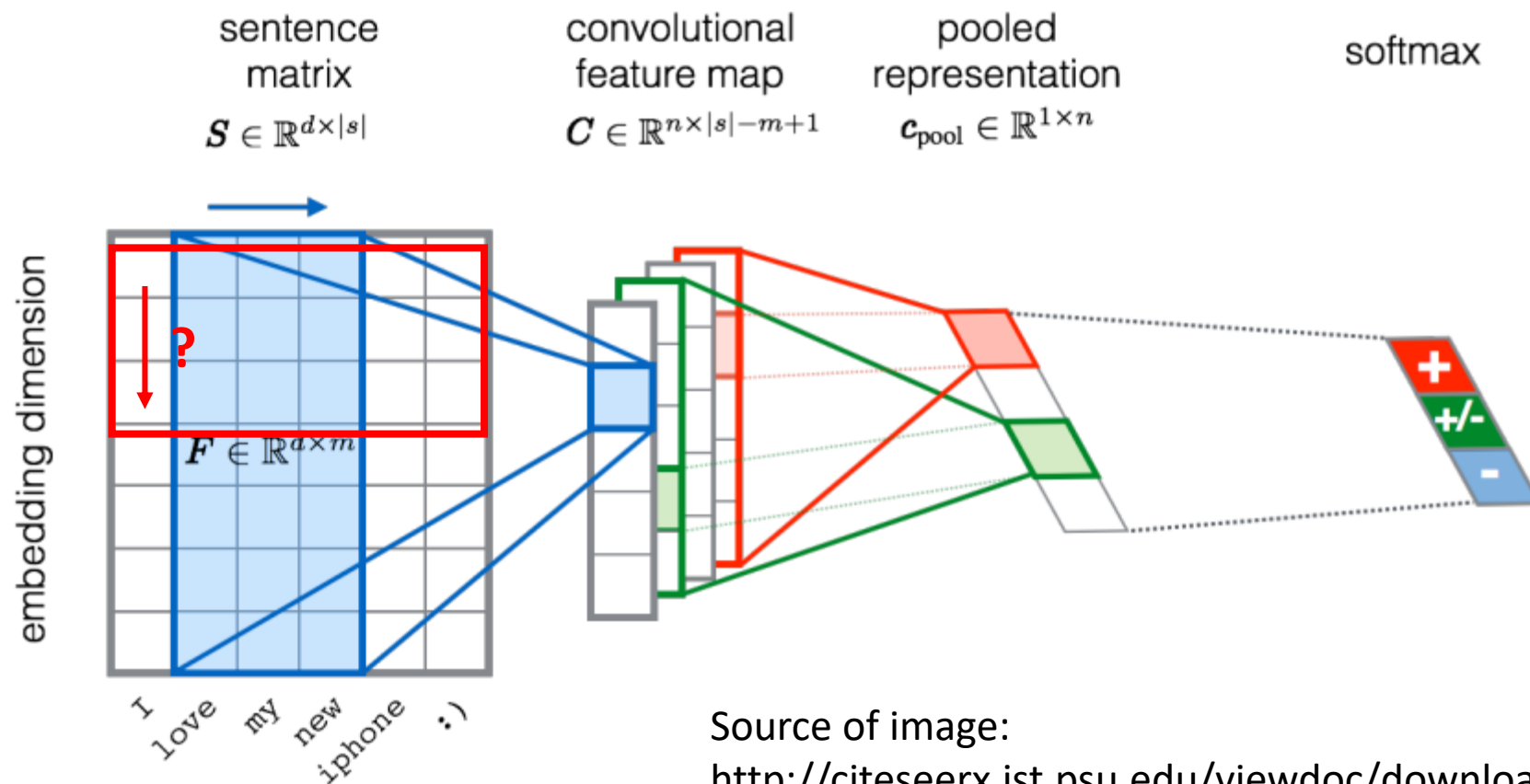
Alpha Go uses 5 x 5 for first layer



- The same patterns appear in different regions.







Source of image:
<http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.703.6858&rep=rep1&type=pdf>



北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY

网络优化与正则化

优化

经验风险最小

正则化

降低模型复杂度



▶ 结构差异大

- ▶ 没有通用的优化算法
- ▶ 超参数多

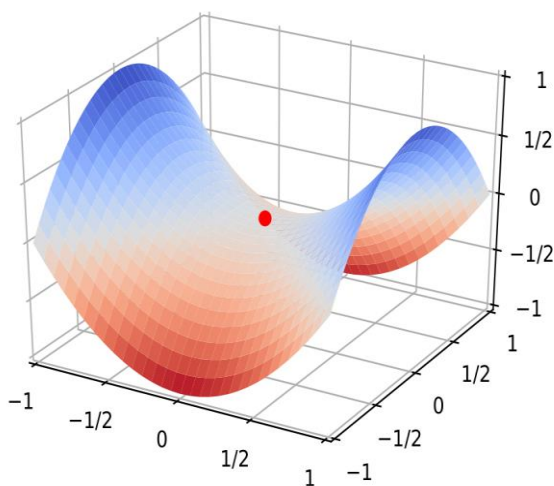
▶ 非凸优化问题

- ▶ 参数初始化
- ▶ 逃离局部最优

▶ 梯度消失（爆炸）问题

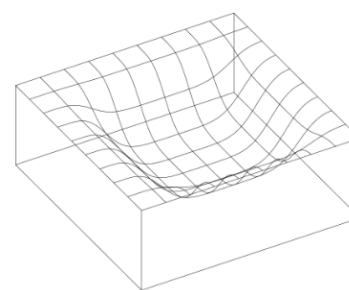
► 鞍点 (Saddle Point)

- 驻点 (Stationary Point) : 梯度为 0 的点。

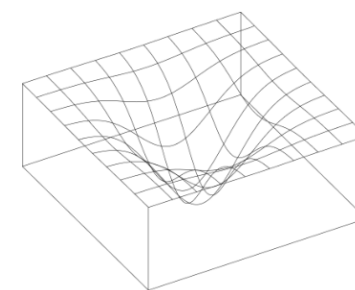


► 平坦最小值 (Flat Minima)

- 一个平坦最小值的邻域内，所有点对应的训练损失都比较接近
- 大部分的局部最小解是等价的
- 局部最小解对应的训练损失都可能非常接近于全局最小解对应的训练损失

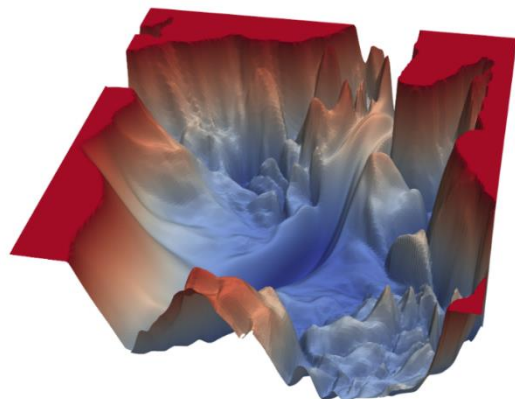


(a) 平坦最小值

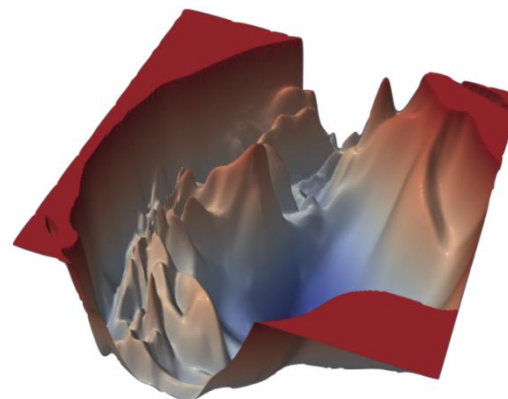


(b) 尖锐最小值

VGG-56

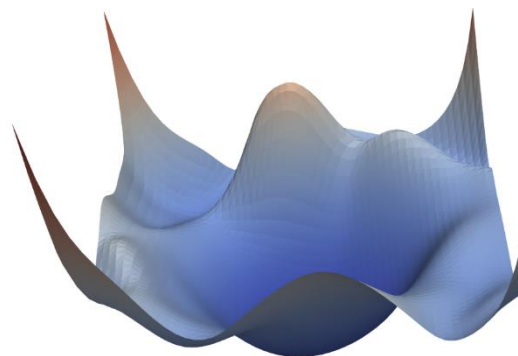


VGG-110

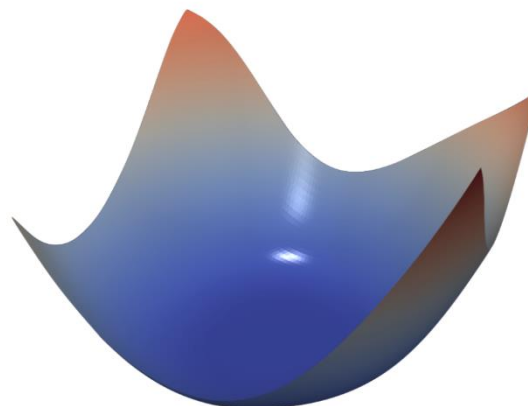


without skip connections

Renset-56



Densenet-121



with skip connections

Li H, Xu Z, Taylor G, et al. Visualizing the loss landscape of neural nets[C]//Advances in Neural Information Processing Systems. 2018: 6389-6399.

- ▶ 更有效的优化算法来提高优化方法的效率和稳定性
 - ▶ 动态学习率调整
 - ▶ 梯度估计修正
- ▶ 更好的参数初始化方法、数据预处理方法来提高优化效率
- ▶ 修改网络结构来得到更好的优化地形
 - ▶ 优化地形（ Optimization Landscape ）指在高维空间中损失函数的曲面形状
 - ▶ 好的优化地形通常比较平滑
 - ▶ 使用 ReLU 激活函数、残差连接、逐层归一化等
- ▶ 使用更好的超参数优化方法

- ▶ 标准的（小批量）梯度下降
- ▶ 学习率调整
 - ▶ 学习率衰减
 - ▶ Adagrad
 - ▶ Adadelat
 - ▶ RMSprop
- ▶ 梯度估计修正
 - ▶ Momentum
 - ▶ 计算负梯度的“加权移动平均”作为参数的更新方向
 - ▶ Nesterov accelerated gradient
 - ▶ 梯度截断

- 大部分优化算法可以使用下面公式来统一描述概括：

$$\Delta\theta_t = -\frac{\alpha_t}{\sqrt{G_t + \epsilon}} M_t,$$

$$G_t = \psi(\mathbf{g}_1, \dots, \mathbf{g}_t),$$

$$M_t = \phi(\mathbf{g}_1, \dots, \mathbf{g}_t),$$

$\mathbf{g}_t$ 为第 t 步的梯度

α_t 为第 t 步的学习率

类别		优化算法
学习率调整	固定衰减学习率	分段常数衰减、逆时衰减、(自然)指数衰减、余弦衰减
	周期性学习率	循环学习率、SGDR
	自适应学习率	AdaGrad、RMSprop、AdaDelta
梯度估计修正		动量法、Nesterov 加速梯度、梯度截断
综合方法		Adam $\approx$ 动量法 + RMSprop

- ▶ 参数不能初始化为0! 为什么?

- ▶ 对称权重问题!

- ▶ 初始化方法

- ▶ 预训练初始化

- ▶ 随机初始化

- ▶ 固定值初始化

- ▶ 偏置 (Bias) 通常用 0 来初始化

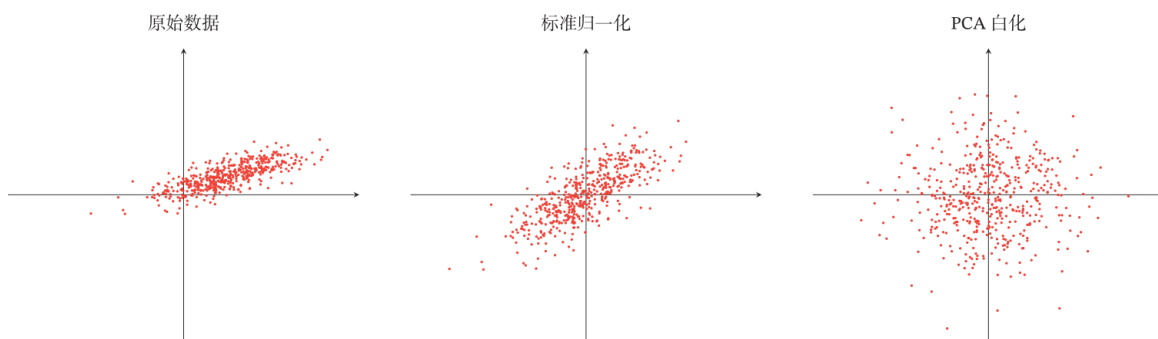
▶ 数据归一化

▶ 最小最大值归一化

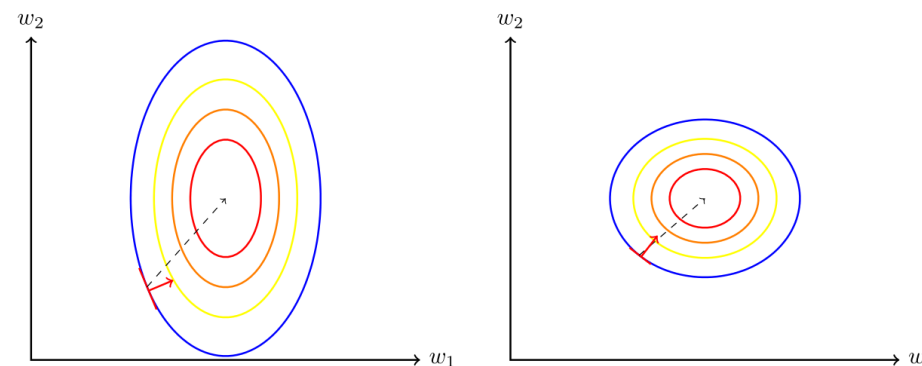
▶ 标准化 $\Longrightarrow$

▶ PCA

$$\mu = \frac{1}{N} \sum_{n=1}^N x^{(n)},$$
$$\sigma^2 = \frac{1}{N} \sum_{n=1}^N (x^{(n)} - \mu)^2.$$



▶ 数据归一化对梯度的影响



(a) 未归一化数据的梯度

(b) 归一化数据的梯度

▶ 目的

- ▶ 更好的尺度不变性
 - ▶ 内部协变量偏移
- ▶ 更平滑的优化地形

▶ 归一化方法

- ▶ 批量归一化 (Batch Normalization, BN)
- ▶ 层归一化 (Layer Normalization)
- ▶ 权重归一化 (Weight Normalization)
- ▶ 局部响应归一化 (Local Response Normalization, LRN)

▶ 超参数

- ▶ 层数
- ▶ 每层神经元个数
- ▶ 激活函数
- ▶ 学习率（以及动态调整算法）
- ▶ 正则化系数
- ▶ mini-batch 大小

▶ 优化方法

- ▶ 网格搜索
- ▶ 随机搜索
- ▶ 贝叶斯优化
- ▶ 动态资源分配
- ▶ 神经架构搜索

►神经网络

►过度参数化

►拟合能力强



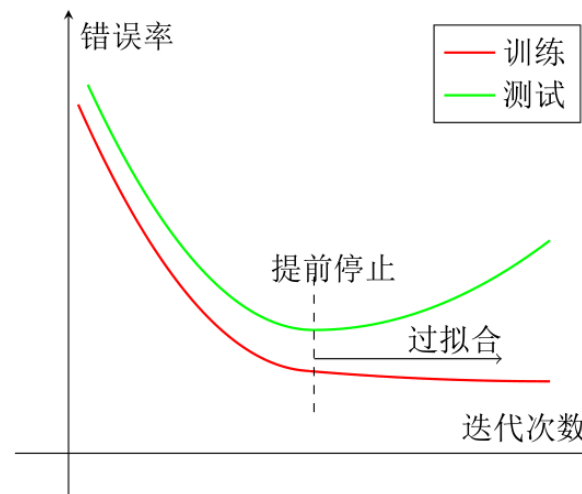
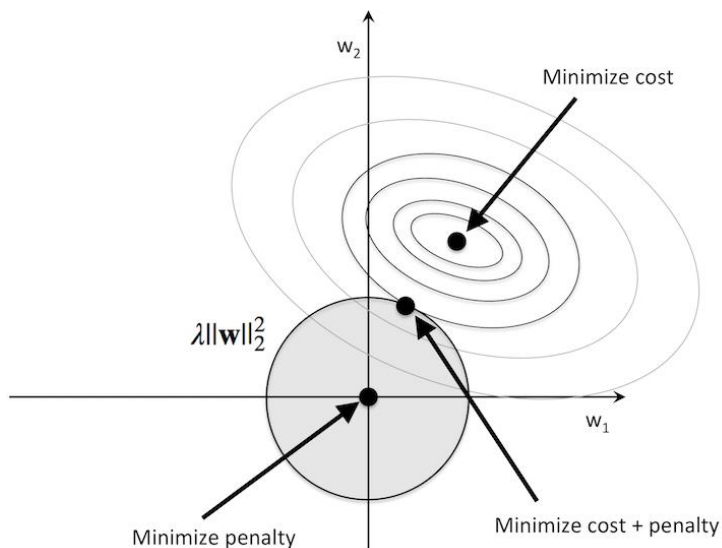
所有损害优化的方法都是正则化。

增加优化约束

干扰优化过程

L1/L2约束、数据增强

权重衰减、随机梯度下降、提前停止



▶ 如何提高神经网络的泛化能力

▶ ℓ_1 和 ℓ_2 正则化

▶ early stop

▶ 权重衰减

▶ SGD

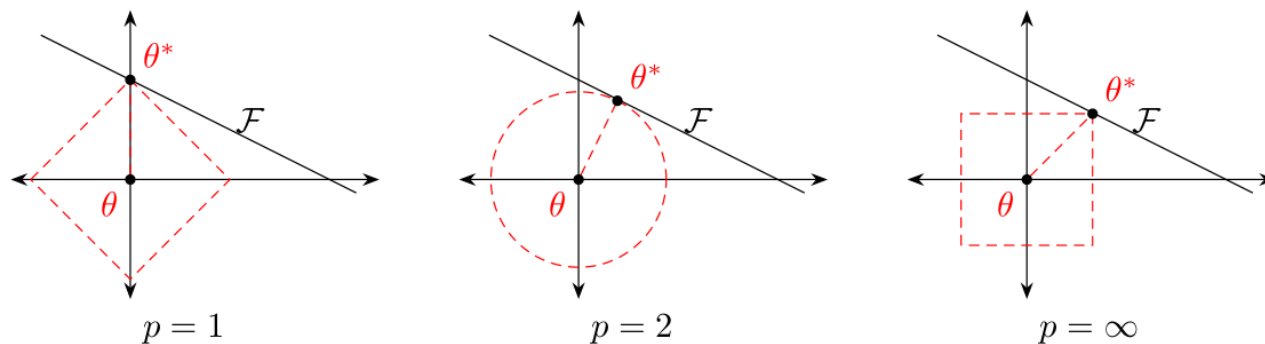
▶ Dropout

▶ 数据增强

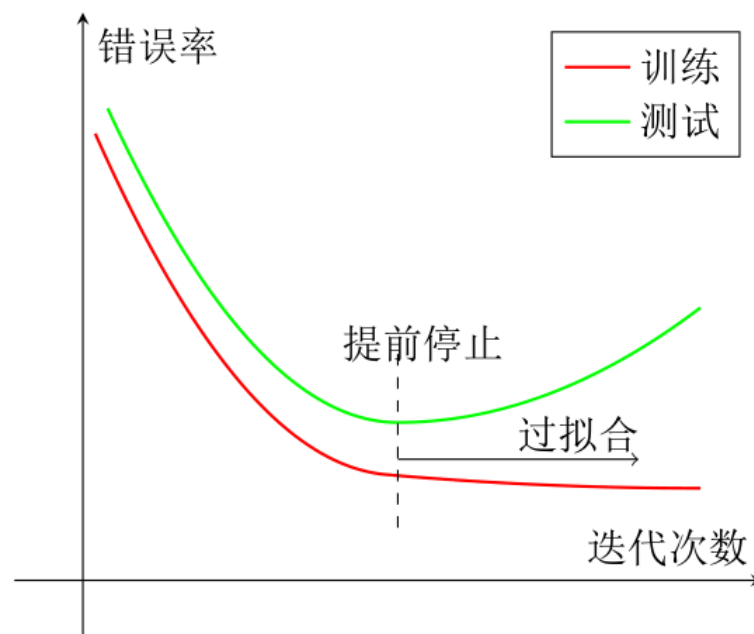
► 优化问题可以写为

$$\theta^* = \arg \min_{\theta} \frac{1}{N} \sum_{n=1}^N \mathcal{L}(y^{(n)}, f(\mathbf{x}^{(n)}, \theta)) + \lambda \ell_p(\theta)$$

► ℓ_p 为范数函数， p 的取值通常为 $\{1, 2\}$ 代表 ℓ_1 和 ℓ_2 范数， λ 为正则化系数。



- ▶ 我们使用一个验证集 (Validation Dataset) 来测试每一次迭代的参数在验证集上是否最优。如果在验证集上的错误率不再下降, 就停止迭代。



- ▶ 在每次参数更新时，引入一个衰减系数 w 。

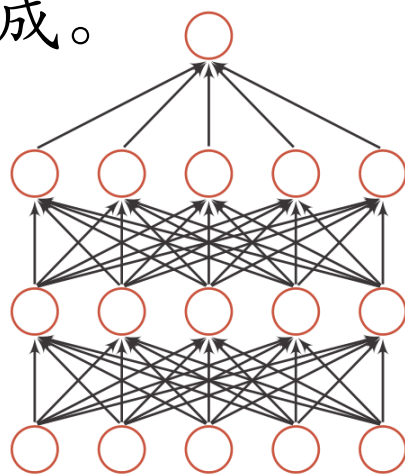
$$\theta_t \leftarrow (1 - w)\theta_{t-1} - \alpha \mathbf{g}_t$$

- ▶ 在标准的随机梯度下降中，权重衰减正则化和 ℓ_2 正则化的效果相同。
- ▶ 在较为复杂的优化方法（比如Adam）中，权重衰减和 ℓ_2 正则化并不等价。

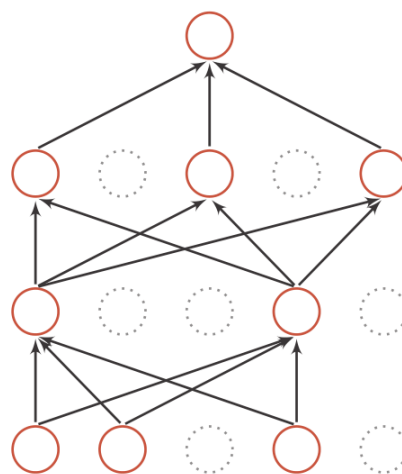
- ▶ 对于一个神经层 $y = f(Wx + b)$, 引入一个丢弃函数 $d(\cdot)$ 使得 $y = f(Wd(x) + b)$ 。

$$d(\mathbf{x}) = \begin{cases} \mathbf{m} \odot \mathbf{x} & \text{当训练阶段时} \\ p\mathbf{x} & \text{当测试阶段时} \end{cases}$$

- ▶ 其中 $m \in \{0,1\}^d$ 是丢弃掩码 (dropout mask), 通过以概率为 p 的贝努力分布随机生成。



(a) 标准网络



(b) Dropout 后的网络

- ▶ 图像数据的增强主要是通过算法对图像进行转变，引入噪声等方法来增加数据的多样性。
- ▶ 图像数据的增强方法：
 - ▶ 旋转 (Rotation)：将图像按顺时针或逆时针方向随机旋转一定角度；
 - ▶ 翻转 (Flip)：将图像沿水平或垂直方法随机翻转一定角度；
 - ▶ 缩放 (Zoom In/Out)：将图像放大或缩小一定比例；
 - ▶ 平移 (Shift)：将图像沿水平或垂直方法平移一定步长；
 - ▶ 加噪声 (Noise)：加入随机噪声。

► 模型

- 用 ReLU 作为激活函数
- 分类时用交叉熵作为损失函数
- 逐层归一化

► 优化

- SGD+mini-batch (动态学习率、Adam算法优先)
- 每次迭代都重新随机排序
- 数据预处理 (标准归一化)
- 参数初始化

► 正则化

- ℓ_1 和 ℓ_2 正则化 (跳过前几轮)
- Dropout
- Early-stop
- 数据增强

德以明理 学以精工

谢谢



智能感知

日期：2025年8月



目录

CONTENTS

1

Python语言基础

2

PyTorch入门

3

手写数字识别实战

4

雷达图像目标识别实战



北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY

Python语言基础

▶ 基础语法

▶ 变量

▶ 数据类型

▶ 运算符

▶ 判断/循环

▶ 函数

变量

- 变量：在程序运行时，能储存计算结果或能表示值的抽象概念。
- 简单的说，变量就是在程序运行时，记录数据用的

变量的定义格式

变量名称 = 变量的值

每一个变量都有自己存储的值（内容），称之为：变量值

赋值，表示将等号右侧的值，赋予左侧的变量

每一个变量都有自己的名称，称之为：变量名，也就是变量本身



变量就像盒子
可以存放内容

数据类型

- 常用的数据类型主要有：

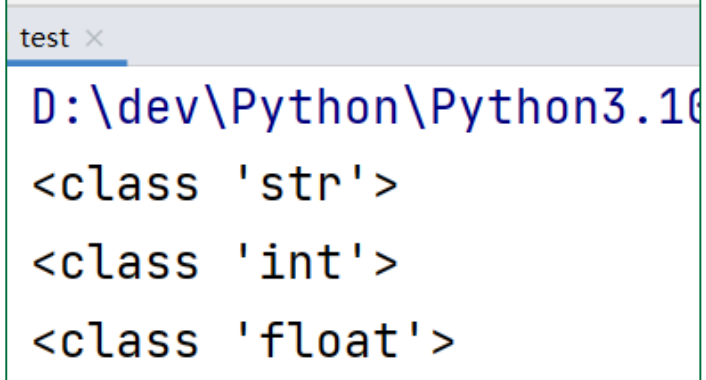
类型	描述	说明
string	字符串类型	用引号引起来的数据都是字符串
int	整型（有符号）	数字类型，存放整数 如 -1,10, 0 等
float	浮点型（有符号）	数字类型，存放小数 如 -3.14, 6.66

- string、int、float这三个英文单词，就是类型的标准名称。
- Type(**被查看类型的数据**)可以查看数据的类型

数据类型

- 1. 在print语句中，直接输出类型信息：

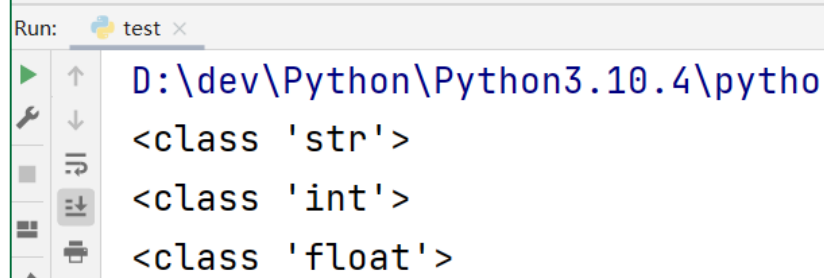
```
print(type("黑马程序员"))  
print(type(666))  
print(type(11.345))
```



str是**string**的缩写

- 2. 用变量存储type()的结果（返回值）：

```
1 string_type = type("黑马程序员")  
2 int_type = type(666)  
3 float_type = type(11.345)  
4 print(string_type)  
5 print(int_type)  
6 print(float_type)
```



运算符 算术（数学）运算符

运算符	描述	实例
+	加	两个对象相加 $a + b$ 输出结果 30
-	减	得到负数或是一个数减去另一个数 $a - b$ 输出结果 -10
*	乘	两个数相乘或是返回一个被重复若干次的字符串 $a * b$ 输出结果 200
/	除	b / a 输出结果 2
//	取整除	返回商的整数部分 $9 // 2$ 输出结果 4 , $9.0 // 2.0$ 输出结果 4.0
%	取余	返回除法的余数 $b \% a$ 输出结果 0
**	指数	$a ** b$ 为10的20次方, 输出结果 100000000000000000000

运算符 算术 (数学) 运算符演示

```
print("1 + 1结果是: %d" % (1 + 1))  
print("2 - 1结果是: %d" % (2 - 1))  
print("1 * 3结果是: %d" % (1 * 3))  
print("9 / 3结果是: %d" % (9 / 3))  
print("9 // 2 (9整除2) 结果是: %d" % (9 // 2))  
print("9 %% 2 (9余2的结果是) 结果是: %d" % (9 % 2))  
print("2 的 6 次方是: 结果是: %d" % (2 ** 6))
```

test (1) ×

D:\dev\Python\Python3.10.4\python.exe D:/pytho

```
1 + 1结果是: 2  
2 - 1结果是: 1  
1 * 3结果是: 3  
9 / 3结果是: 3  
9 // 2 (9整除2) 结果是: 4  
9 % 2 (9余2的结果是) 结果是: 1  
2 的 6 次方是: 结果是: 64
```

运算符 赋值运算符

运算符	描述	实例
=	赋值运算符	把 = 号右边的结果 赋给 左边的变量，如 <code>num = 1 + 2 * 3</code> ，结果num的值为7

复合赋值运算符

运算符	描述	实例
<code>+=</code>	加法赋值运算符	<code>c += a</code> 等效于 <code>c = c + a</code>
<code>-=</code>	减法赋值运算符	<code>c -= a</code> 等效于 <code>c = c - a</code>
<code>*=</code>	乘法赋值运算符	<code>c *= a</code> 等效于 <code>c = c * a</code>
<code>/=</code>	除法赋值运算符	<code>c /= a</code> 等效于 <code>c = c / a</code>
<code>%=</code>	取模赋值运算符	<code>c %= a</code> 等效于 <code>c = c % a</code>
<code>**=</code>	幂赋值运算符	<code>c **= a</code> 等效于 <code>c = c ** a</code>
<code>//=</code>	取整除赋值运算符	<code>c //= a</code> 等效于 <code>c = c // a</code>

判断语句

- 生活中的判断



程序中的判断

`if` 要判断的条件:

条件成立时，要做的事情

```
# 定义变量
```

```
age = 30
```

```
# 进行判断
```

```
if age >= 18:
```

```
    print("我已经成年了")
```



判断语句

```
age = 18

print(f"今年我已经{age}岁了")
if age >= 18:
    print("我已经成年了")
    print("即将步入大学生活")

print("时间过的真快")
```

当if判断条件结果为True时，代码会被执行
为False时，代码不会执行

- 当age为18时，执行结果：

```
D:\dev\python\python
今年我已经30岁了
我已经成年了
即将步入大学生活
时间过的真快
```

- 当age为10时，执行结果：

```
D:\dev\python\python
今年我已经10岁了
时间过的真快
```

判断语句：if;else

```
print("欢迎来到黑马儿童游乐场，儿童免费，成人收费。")
age = int(input("请输入你的年龄："))
if age >= 18:
    print("您已成年，游玩需要补票10元。") 条件成立时执行
else:
    print("您未成年，可以免费游玩。") 条件不成立时执行

print("祝您游玩愉快。")
```


判断语句：if;elif;else

```
print("欢迎来到黑马动物园。")
height = int(input("请输入你的身高（cm）："))
vip_level = int(input("请输入你的vip级别(1~5)："))
if height < 120:
    print("您的身高小于120CM，可以免费游玩。")
elif vip_level > 3:
    print("您的vip级别大于3，可以免费游玩。")
else:
    print("不好意思，所有条件都不满足，需要购票10元。")

print("祝您游玩愉快。")
```

循环语句：while循环语句

- while循环的语法格式

```
while 条件:  
    条件满足时, 做的事情1  
    条件满足时, 做的事情2  
    条件满足时, 做的事情3  
    ... (省略) ...
```

```
i = 0  
while i < 100:  
    print( "第{}次循环 ".format(i))  
    i += 1
```

- while循环的注意事项

- 条件需提供布尔类型结果, True继续, False停止
- 空格缩进不能忘
- 请规划好循环终止条件, 否则将无限循环

循环语句：for循环语句

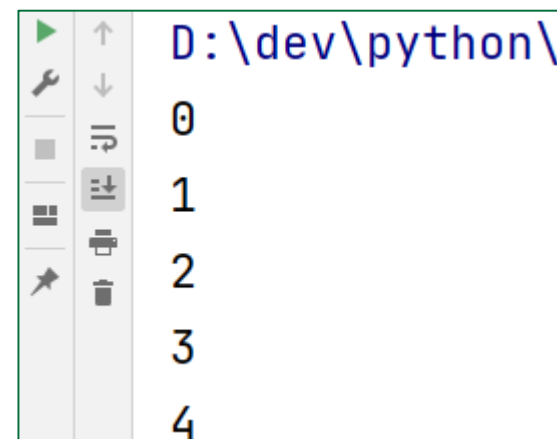
for 临时变量 **in** 待处理数据集(可迭代对象):
循环满足条件时执行的代码

- 语法中的：待处理数据集，严格来说，称之为：**可迭代类型**
- 可迭代类型指，**其内容可以一个个依次取出的一种类型**，包括：
 - **字符串**
 - 列表
 - 元组
 - 等
- 常用的如range(100)代表临时变量从0循环至99结束

循环语句：for循环语句

```
# for循环处理字符串  
for i in range(5):  
    print(i)
```

运行结果如下：



```
D:\dev\python\  
0  
1  
2  
3  
4
```

函数

1. 函数是：

- **组织好的、可重复使用的、用来实现特定功能的代码段**

2. 使用函数的好处是：

- 将功能封装在函数内，可供随时随地重复利用
- 提高代码的复用性，减少重复代码，提高开发效率

函数

函数的定义:

```
def 函数名(传入参数):  
    函数体  
    return 返回值
```

例子：一个实现计算 $1+2$ 功能并打印结果的函数

```
def add():  
    result = 1 + 2  
    print(f"1 + 2的结果是:{result}")
```

函数

函数传入参数:

- 实现了, 每次计算的是 $x + y$, 而非固定的 $1 + 2$
- $x + y$ 的值, 可以在调用函数的时候指定。
- 每次调用`add(1,4)`, 会输出5

```
def add(x, y):  
    result = x + y  
    print(f"{x} + {y}的结果是:{result}")
```

- 函数的传入参数大大扩展了函数使用的可重复性和灵活性

函数

语法解析：

```
# 定义函数
def add(x, y):
    result = x + y
    print(f"{x} + {y}的结果是:{result}")

# 调用函数
add(5, 6)
```

- 函数定义中，提供的x和y，称之为：形式参数（形参），表示函数声明将要使用2个参数
 - 参数之间使用逗号进行分隔
- 函数调用中，提供的5和6，称之为：实际参数（实参），表示函数执行时真正使用的参数值
 - 传入的时候，按照顺序传入数据，使用逗号分隔

函数 函数返回值

```
def add(a, b):  
    result = a + b  
    return result  
  
r = add(1, 2)  
print(r)
```

test (1) ×
D:\dev\python\python3.1
3

如图代码

定义两数相加的函数功能。完成功能后，会将相加的结果返回给函数调用者

所以，变量r接收到了函数的执行结果。

综上所述：

所谓“返回值”，就是程序中函数完成事情后，最后给调用者的结果



北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY

PyTorch入门

- ▶ 概述
- ▶ 安装使用
- ▶ 张量tensor和自动微分autograd
- ▶ torch.nn: Module 类
- ▶ 常用模块



在这个图像中有不同的分类：**猫，骆驼，鹿，大象等。**

卷积神经网络（CNNs） 对于这类图像分类任务十分有效。

从头开始实现一个卷积神经网络模型？

我们可能需要几天（甚至几周）才能得到一个有效的模型，时间成本太高！

深度学习框架让这一切变得更加高效！

什么是深度学习框架？

深度学习框架是一种接口、库或工具，利用预先构建和优化好的组件集合定义模型。

一个良好的深度学习框架应具备的关键特性：

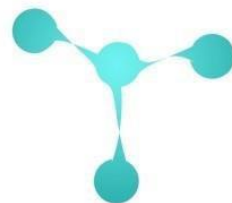
- ✓ 性能优化
- ✓ 易于理解和编码
- ✓ 良好的社区支持
- ✓ 并行处理以减少计算
- ✓ 自动计算梯度



■ 深度学习的主流框架

- ✓ Caffe
- ✓ Theano
- ✓ Torch & PyTorch
- ✓ TensorFlow
- ✓ Keras
- ✓ MXNet

Microsoft
CNTK



Caffe

Caffe2

PYTORCH

Chainer

K Keras

TensorFlow

theano

dy/net

mxnet

GLUON

Torch & PyTorch

- Torch是一个有大量机器学习算法支持的科学计算框架，其诞生已经有十余年，但真正起势得益于Facebook开源了大量Torch的深度学习模块和扩展。Torch的一个特殊之处是采用了Lua编程语言（曾被用来开发视频旅游）。

<https://github.com/torch>

- PyTorch于2016年10月发布，是一款专注于直接处理数组表达式的低级API。前身是Torch。Facebook人工智能研究院对PyTorch提供了强力支持。PyTorch支持动态计算图，为更具数学倾向的用户提供了更低层次的方法和更多的灵活性，目前许多新发表的论文都采用PyTorch作为论文实现的工具，成为学术研究的首选解决方案。

<https://github.com/pytorch>



TensorFlow

- TensorFlow最初由Google Brain团队针对机器学习和深度神经网络进行研究所开发的，目前开源之后可以在几乎各种领域适用。它灵活的架构可以部署在一个或多个CPU、GPU的台式及服务器中，或者使用单一的API应用在移动设备中。TensorFlow可以说是当今十分流行的深度学习框架，Airbnb、DeepMind、Intel、Nvidia、Twitter以及许多其他著名公司都在使用它。TensorFlow提供全面的服务，构建了活跃的社区，完善的文档体系，大大降低了我们的学习成本，另外，TensorFlow有很直观的计算图可视化呈现。模型能够快速的部署在各种硬件机器上，从高性能的计算机到移动设备，再到更小的更轻量的智能终端。
- 但是，TensorFlow相比Pytorch，Caffe等框架，计算速度很慢。而且通过它构建一个深度学习框架需要更复杂的代码，还要忍受重复的多次构建静态图。

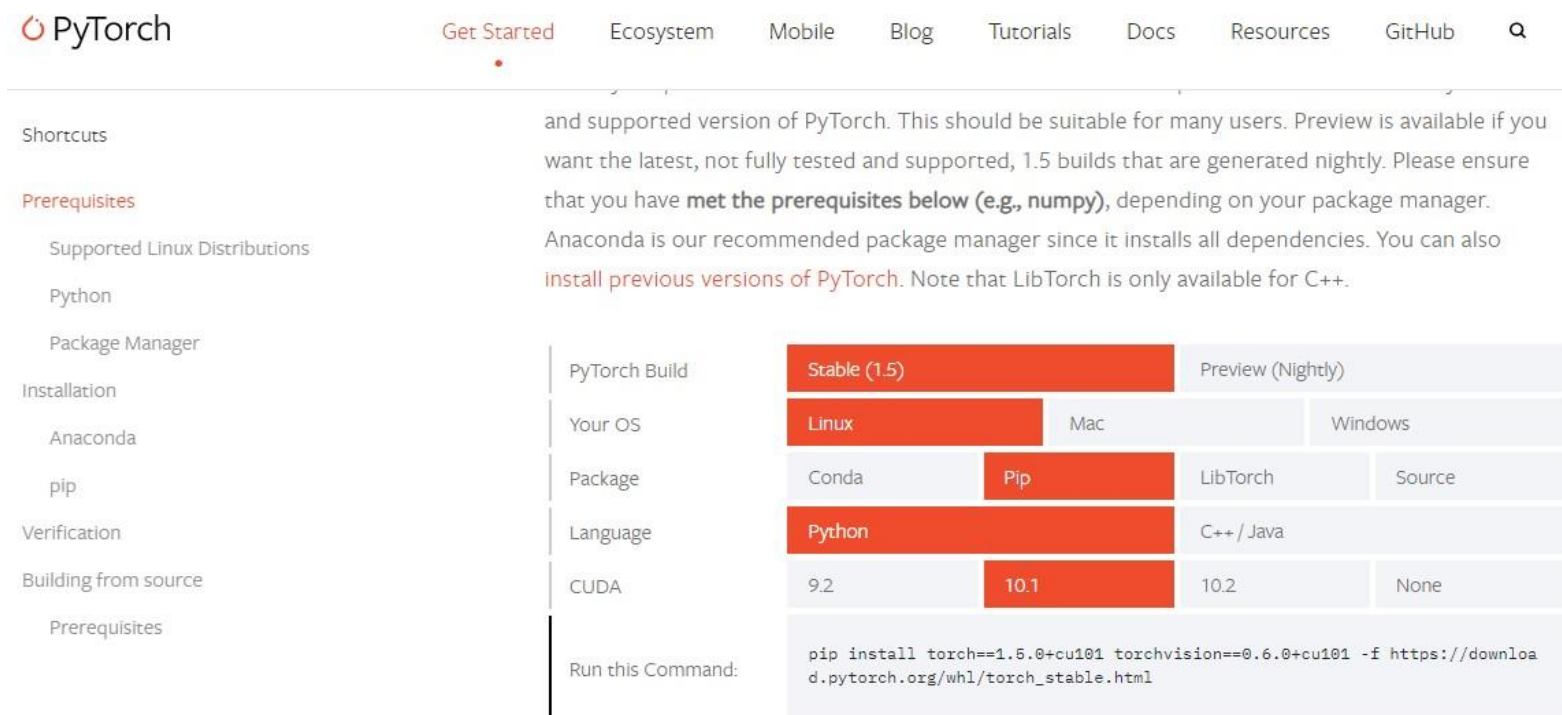
<https://github.com/tensorflow>



- PyTorch是一个基于Python的库，用来提供一个具有灵活性的深度学习开发平台。PyTorch的工作流程非常接近Python的科学计算库——**numpy**。
- PyTorch像Python一样简单，可以顺利地 Python 数据科学栈集成，取代了具有特定功能的预定义图形，PyTorch为我们提供了一个框架，以便可以在运行时构建计算图，甚至在运行时更改它们。PyTorch 的其他一些优点还包括：**多gpu支持**，**自定义数据加载器**和**简化的预处理**。

■ 使用pip安装

目前，使用pip安装PyTorch二进制包是最简单、最不容易出错，同时也是最适合新手的安装方式。从PyTorch官网选择操作系统、包管理器pip、Python版本及CUDA版本，会对应不同的安装命令，如图所示：<https://pytorch.org/get-started/locally/>



PyTorch

Get Started Ecosystem Mobile Blog Tutorials Docs Resources GitHub

Shortcuts

Prerequisites

- Supported Linux Distributions
- Python
- Package Manager

Installation

- Anaconda
- pip

Verification

Building from source

Prerequisites

and supported version of PyTorch. This should be suitable for many users. Preview is available if you want the latest, not fully tested and supported, 1.5 builds that are generated nightly. Please ensure that you have met the prerequisites below (e.g., numpy), depending on your package manager. Anaconda is our recommended package manager since it installs all dependencies. You can also install previous versions of PyTorch. Note that LibTorch is only available for C++.

PyTorch Build	Stable (1.5)	Preview (Nightly)
Your OS	Linux	Mac Windows
Package	Conda	Pip LibTorch Source
Language	Python	C++ / Java
CUDA	9.2	10.1 10.2 None
Run this Command:	<pre>pip install torch==1.5.0+cu101 torchvision==0.6.0+cu101 -f https://download.pytorch.org/whl/torch_stable.html</pre>	

■ Tensor

Tensor，又名张量。可简单地认为它是一个数组，且支持高效的科学计算。它可以是一个数（标量）、一维数组（向量）、二维数组（矩阵）或更高维的数组（高阶数据）。Tensor和numpy的ndarrays类似，但PyTorch的tensor支持GPU加速。如下定义一个简单的一维矩阵：

```
import torch
torch.FloatTensor([2])
```

与numpy一样，科学计算库非常重要的一点是能够实现高效的数学功能。如下实现一个简单的加法操作：

```
a = torch.FloatTensor([2])
b = torch.FloatTensor([3])
a + b
```

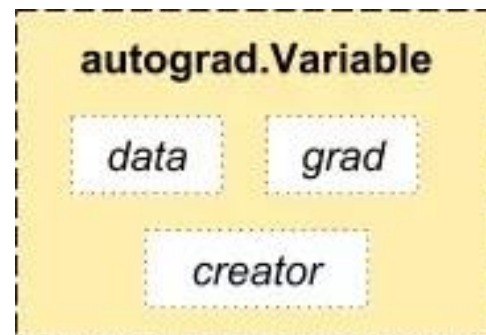
■ autograd

torch.autograd是为了方便用户使用，专门开发的一套**自动求导引擎**，它能够根据**输入**和**前向传播**过程自动构建**计算图**，并执行**反向传播**。

1) Variable

PyTorch在autograd模块中实现了计算图的相关功能，autograd中的核心数据结构是**Variable**。Variable封装了tensor，并记录对tensor的操作记录用来构建计算图。Variable的数据结构如图所示，主要包含如下三个属性：

- data: 保存variable所包含的tensor。
- grad: 保存data对应的梯度，grad也是**variable**，而非tensor。
- grad_fn: 指向一个**Function**，记录variable的操作历史。



代码示例一：

```
import numpy
```

```
as np import
```

```
torch
```

```
from torch.autograd import Variable
```

```
x = Variable(torch.ones(2,2),requires_grad = False) temp
```

```
= Variable(torch.zeros(2,2),requires_grad = True) y = x +
```

```
temp + 2
```

```
y = y.mean()      #求平均数
```

```
y.backward()      #反向传递函数，用于求y对前面的变量（x）的梯度
```

```
print(x.grad)     # d(y)/d(x)
```

```
输出: none      # 因为requires_grad=False
```

代码示例二：

```
import numpy as np
import torch

from torch.autograd import Variable

x = Variable(torch.ones(2,2),requires_grad = False)
temp = Variable(torch.zeros(2,2),requires_grad = True)
y = x + temp + 2

y = y.mean()    #求平均数
y.backward()    #反向传递函数，用于求y对前面的变量（x）的梯度
print(temp.grad)    # d(y)/d(temp)

    输出： tensor([[0.2500, 0.2500],
                  [0.2500, 0.2500]])
```

2 计算图

计算图是一种特殊的有向无环图，用于记录算子与变量之间的关系。一般用矩形表示算子，椭圆形表示变量。

在PyTorch中计算图特点总结如下：

- Autograd根据用户对variable的操作构建计算图。对variable的操作抽象为Function。
- Variable默认是不需要求导的，即requires_grad属性默认为False。
- 多次反向传播时，梯度是累加的。反向传播的中间缓存会被清空。
- PyTorch采用动态图设计，可以很方便地查看中间层的输出，动态地设计计算图结构。

■ nn.Module

torch.nn的核心数据结构是Module，它是一个抽象的概念，既可以表示神经网络中的某个层，也可以表示一个包含很多层的神经网络。最常见的做法就是继承nn.Module，编写自己的网络。

- ✓ 能够自动检测到自己的parameter，并将其作为学习参数。
- ✓ 主Module能够递归查找子Module中的parameter。

为了方便用户使用，PyTorch实现了神经网络中绝大多数的layer，这些layer都继承于nn.Module，封装了可学习参数parameter，并实现了forward函数，且专门针对GPU运算进行了CuDNN优化，其速度和性能都十分优异。

可学习的参数：卷积层和全连接层的权重、bias、BatchNorm的 γ 、 β 等。

不可学习的参数(超参数)：学习率、batch size、weight decay、模型的深度宽度分辨率等。

具体层可参照官方文档 (<http://pytorch.org/docs/nn.html>)，但阅读文档时应主要关注以下几点：

- 构造函数的参数，如`nn.Linear(in_features, out_features, bias)`，需关注这三个参数的作用。
- 属性、可学习参数和子module。如`nn.Linear`中有`weight`和`bias`两个可学习参数，不包含子module。
- 输入输出的形状，如`nn.linear`的输入形状是 $(N, \text{input_features})$ ，输出为 $(N, \text{output_features})$ ， N 是`batch_size`。

- 要实现一个自定义层大致分以下几个主要的步骤：
 - 1 自定义一个类，继承自**Module类**，并且一定要实现两个基本的函数
 - ✓ 构造函数__init__
 - ✓ 前向计算函数forward函数
 - 2 在构造函数**init**中实现层的参数定义。
 - ✓ 例如：Linear层的权重和偏置
 - ✓ Conv2d层的in_channels, out_channels, kernel_size, stride=1, padding=0, dilation=1, groups=1, bias=True, padding_mode='zeros'这一系列参数

- 3 在前向传播**forward函数**里面实现前向运算。
 - ✓ 通过torch.nn.functional函数来实现
 - ✓ 自定义自己的运算方式。如果该层含有权重，那么权重必须是nn.Parameter类型

1. 数据处理

在PyTorch中，数据加载可通过自定义的数据集对象实现。数据集对象被抽象为Dataset类，实现自定义的数据集需要继承Dataset，并实现两个Python方法。

- `__getitem__`：返回一条数据或一个样本。`obj[index]`等价于`obj.__getitem__(index)`。
- `__len__`：返回样本的数量。`len(obj)`等价于`obj.__len__()`。

Dataset只负责数据的抽象，一次调用`__getitem__`只返回一个样本。在训练神经网络时，是对一个batch的数据进行操作，同时还需要对数据进行并行加速等。因此，PyTorch提供了DataLoader帮助实现这些功能。

■ DataLoader函数定义如下:

`DataLoader(dataset, batch_size=1, shuffle=False, sampler=None, num_workers=0, collate_fn=default_collatem, pin_memory=False, drop_last=False)`

➤ 参数解释如下:

- ✓ **dataset**: 加载的数据集 (Dataset对象)
- ✓ **batch_size**: batch size (批大小)
- ✓ **shuffle**: 是否将数据打乱
- ✓ **sampler**: 样本抽样
- ✓ **num_workers**: 使用多进程加载的进程数, 0代表不使用多进程
- ✓ **collate_fn**: 如何将多个样本数据拼接成一个batch, 一般使用默认的拼接方式即可
- ✓ **pin_memory**: 是否将数据保存在pin memory区, pin memory中的数据转到GPU会快一些
- ✓ **drop_last**: dataset中的数据个数可能不是batch_size的整数倍, drop_last为True会将多出来不足一个batch的数据丢弃

2. 计算机视觉工具包：torchvision

视觉工具包torchvision独立于PyTorch，需要通过pip install torchvision安装。torchvision主要包含以下三个部分：

- ✓ models：提供深度学习中各种经典网络的网络结构及预训练好的模型，包括Net、VGG系列、ResNet系列、Inception系列等。
- ✓ datasets：提供常用的数据集加载，设计上都是继承torch.utils.data.Dataset，主要包括MNIST、CIFAR10/100、ImageNet、COCO等。
- ✓ transforms：提供常用的数据预处理操作，主要包括对Tensor以及PIL Image对象的操作。

■ 以MNIST数据集为例：

`torchvision.datasets.MNIST (root, train = True, transform = None, target transform = None, download = False )`

➤ 参数介绍：

- ✓ root (string) - 数据集的根目录在哪里MNIST/processed/training.pt 和 MNIST/processed/test.pt存在
- ✓ train (bool, optional) - 如果为True, 则创建数据集training.pt, 否则创建数据集test.pt。
- ✓ download (bool, optional) - 如果为true, 则从Internet下载数据集并将其放在根目录中。如果已下载数据集, 则不会再次下载
- ✓ transform (callable , optional) - 一个函数/转换, 它接收PIL图像并返回转换后的版本。例如, transforms.RandomCrop
- ✓ target_transform (callable , optional) - 接收目标并对其转换的函数/转换



4. 保存和加载

在PyTorch中，需要将Tensor、Variable等对象保存到硬盘，以便后续能通过相应的方法加载到内存中。

- ✓ 这些信息最终都是保存成Tensor。
- ✓ 使用t.save和t.load即可完成Tensor保存和加载的功能。
- ✓ 在save/load时可指定使用的pickle模块，在load时还可以将GPU tensor映射到CPU或其他GPU上。
- ✓ 使用方式：
 - `t.save(obj, file_name)`: 保存任意可序列化的对象。
 - `obj = t.load(file_name)`: 方法加载保存的数据。



北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY

手写数字识别实战

►PyTorch 实现

►修改参数、网络结构，对比结果（课后作业）

导入所需程序包

```
import torch
import torch.nn as nn
import matplotlib.pyplot as plt
import torchvision

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
device
```

```
device(type='cpu')
```

利用torchvision定义MNIST手写数据集

```
# 设置训练集
train_ds = torchvision.datasets.MNIST('data', train=True, transform=torchvision.transforms.ToTensor(), download=True)

# 设置测试集
test_ds = torchvision.datasets.MNIST('data', train=False, transform=torchvision.transforms.ToTensor(), download=True)
```

定义数据迭代器dataloader

```
batch_size = 32

train_dl = torch.utils.data.DataLoader(train_ds,
                                       batch_size=batch_size,
                                       shuffle=True)

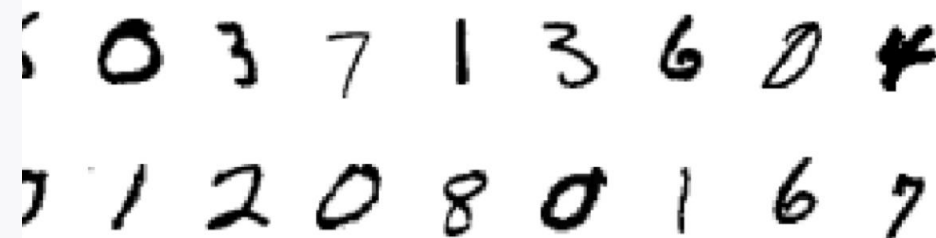
test_dl = torch.utils.data.DataLoader(test_ds,
                                       batch_size=batch_size)

# 取一个批次查看数据格式
# 数据的shape为: [batch_size, channel, height, weight]
# 其中batch_size为自己设定, channel, height和weight分别是图片的通道数, 高度和宽度。
imgs, labels = next(iter(train_dl))
imgs.shape
```

数据可视化

```
import numpy as np

# 指定图片大小, 图像大小为20宽、5高的绘图(单位为英寸inch)
plt.figure(figsize=(20, 5))
for i, imgs in enumerate(imgs[:20]):
    # 维度缩减
    npimg = np.squeeze(imgs.numpy())
    # 将整个figure分成2行10列, 绘制第i+1个子图。
    plt.subplot(2, 10, i+1)
    plt.imshow(npimg, cmap=plt.cm.binary)
    plt.axis('off')
```



A visualization of handwritten digits from the MNIST dataset, displayed in a 2x10 grid. The digits are rendered in black on a white background, showing various styles of handwriting. The first row contains the digits 0, 3, 7, 1, 3, 6, 0, and 4. The second row contains the digits 1, 2, 0, 8, 0, 1, 6, and 7.

定义网络 (可自己设置自定义)

```
import torch.nn.functional as F

num_classes = 10 # 图片的类别数

class Model(torch.nn.Module):
    def __init__(self):
        super().__init__()
        # 特征提取网络
        self.conv1 = nn.Conv2d(1, 32, kernel_size=3) # 第一层卷积, 卷积核大小为3*3
        self.pool1 = nn.MaxPool2d(2) # 设置池化层, 池化核大小为2*2
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3) # 第二层卷积, 卷积核大小为3*3
        self.pool2 = nn.MaxPool2d(2)

        # 分类网络
        self.fc1 = nn.Linear(1600, 64)
        self.fc2 = nn.Linear(64, num_classes)
    # 前向传播
    def forward(self, x):
        x = self.pool1(F.relu(self.conv1(x)))
        x = self.pool2(F.relu(self.conv2(x)))

        x = torch.flatten(x, start_dim=1)

        x = F.relu(self.fc1(x))
        x = self.fc2(x)

        return x
```

打印模型

```
from torchinfo import summary
model = Model().to(device)
summary(model)
```

```
=====
Layer (type:depth-idx)                               Param #
=====
Model                                                  --
|─Conv2d: 1-1                                           320
|─MaxPool2d: 1-2                                         --
|─Conv2d: 1-3                                           18,496
|─MaxPool2d: 1-4                                         --
|─Linear: 1-5                                             102,464
|─Linear: 1-6                                             650
=====
Total params: 121,930
Trainable params: 121,930
Non-trainable params: 0
=====
```

训练模型

损失函数、优化器设置

```
# 创建损失函数
loss_fn = nn.CrossEntropyLoss()
# 设置学习率
learn_rate = 1e-2
# 设置优化器
opt = torch.optim.SGD(model.parameters(), lr=learn_rate)
```


训练模型

训练循环函数

训练循环

```
def train(dataloader, model, loss_fn, optimizer):
    size = len(dataloader.dataset)  # 训练集的大小, 一共60000张图片
    num_batches = len(dataloader)  # 批次数目, 1875 (60000/32)

    train_loss, train_acc = 0, 0  # 初始化训练损失和正确率

    for X, y in dataloader:  # 获取图片及其标签
        X, y = X.to(device), y.to(device)

        # 计算预测误差
        pred = model(X)  # 网络输出
        loss = loss_fn(pred, y)  # 计算网络输出和真实值之间的差距, targets为真实值, 计算二者差值即为损失

        # 反向传播
        optimizer.zero_grad()  # grad属性归零
        loss.backward()  # 反向传播
        optimizer.step()  # 每一步自动更新

        # 记录acc与loss
        train_acc += (pred.argmax(1) == y).type(torch.float).sum().item()
        train_loss += loss.item()

    train_acc /= size
    train_loss /= num_batches

    return train_acc, train_loss
```

测试函数

```
def test (dataloader, model, loss_fn):
    size          = len(dataloader.dataset) # 测试集的大小, 一共10000张图片
    num_batches    = len(dataloader)         # 批次数目, 313 (10000/32=312.5, 向上取整)
    test_loss, test_acc = 0, 0

    # 当不进行训练时, 停止梯度更新, 节省计算内存消耗
    with torch.no_grad():
        for imgs, target in dataloader:
            imgs, target = imgs.to(device), target.to(device)

            # 计算loss
            target_pred = model(imgs)
            loss         = loss_fn(target_pred, target)

            test_loss += loss.item()
            test_acc  += (target_pred.argmax(1) == target).type(torch.float).sum().item()

    test_acc /= size
    test_loss /= num_batches

    return test_acc, test_loss
```

训练主循环

```
epochs = 5
train_loss = []
train_acc = []
test_loss = []
test_acc = []

for epoch in range(epochs):
    model.train()
    epoch_train_acc, epoch_train_loss = train(train_dl, model, loss_fn, opt)

    model.eval()
    epoch_test_acc, epoch_test_loss = test(test_dl, model, loss_fn)

    train_acc.append(epoch_train_acc)
    train_loss.append(epoch_train_loss)
    test_acc.append(epoch_test_acc)
    test_loss.append(epoch_test_loss)

    template = ('Epoch:{:2d}, Train_acc:{:.1f}%, Train_loss:{:.3f}, Test_acc:{:.1f}%, Test_loss:{:.3f}')
    print(template.format(epoch+1, epoch_train_acc*100, epoch_train_loss, epoch_test_acc*100, epoch_test_loss))
print('Done')
```

```
Epoch: 1, Train_acc:76.9%, Train_loss:0.762, Test_acc:91.8%, Test_loss:0.271
Epoch: 2, Train_acc:94.0%, Train_loss:0.198, Test_acc:95.0%, Test_loss:0.159
Epoch: 3, Train_acc:96.3%, Train_loss:0.123, Test_acc:97.3%, Test_loss:0.087
Epoch: 4, Train_acc:97.2%, Train_loss:0.093, Test_acc:97.6%, Test_loss:0.073
Epoch: 5, Train_acc:97.5%, Train_loss:0.078, Test_acc:98.0%, Test_loss:0.064
```

Done

结果可视化

```
import matplotlib.pyplot as plt
#隐藏警告
import warnings
warnings.filterwarnings("ignore") #忽略警告信息
plt.rcParams['font.sans-serif'] = ['SimHei'] # 用来正常显示中文标签
plt.rcParams['axes.unicode_minus'] = False # 用来正常显示负号
plt.rcParams['figure.dpi'] = 100 #分辨率

epochs_range = range(epochs)

plt.figure(figsize=(12, 3))
plt.subplot(1, 2, 1)

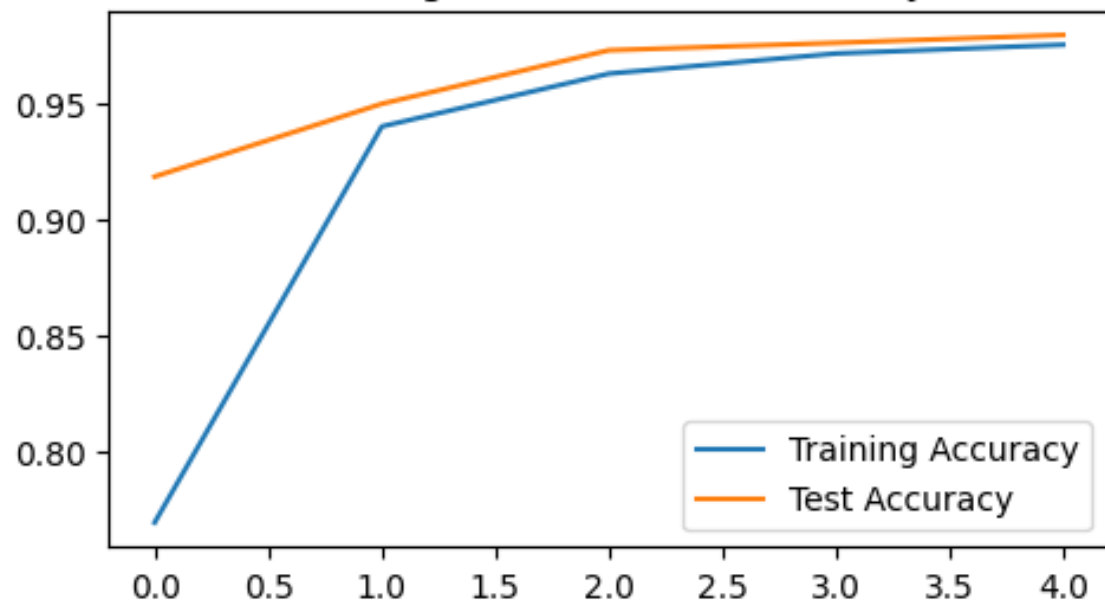
plt.plot(epochs_range, train_acc, label='Training Accuracy')
plt.plot(epochs_range, test_acc, label='Test Accuracy')
plt.legend(loc='lower right')
plt.title('Training and Validation Accuracy')

plt.subplot(1, 2, 2)
plt.plot(epochs_range, train_loss, label='Training Loss')
plt.plot(epochs_range, test_loss, label='Test Loss')
plt.legend(loc='upper right')
plt.title('Training and Validation Loss')
plt.show()
```

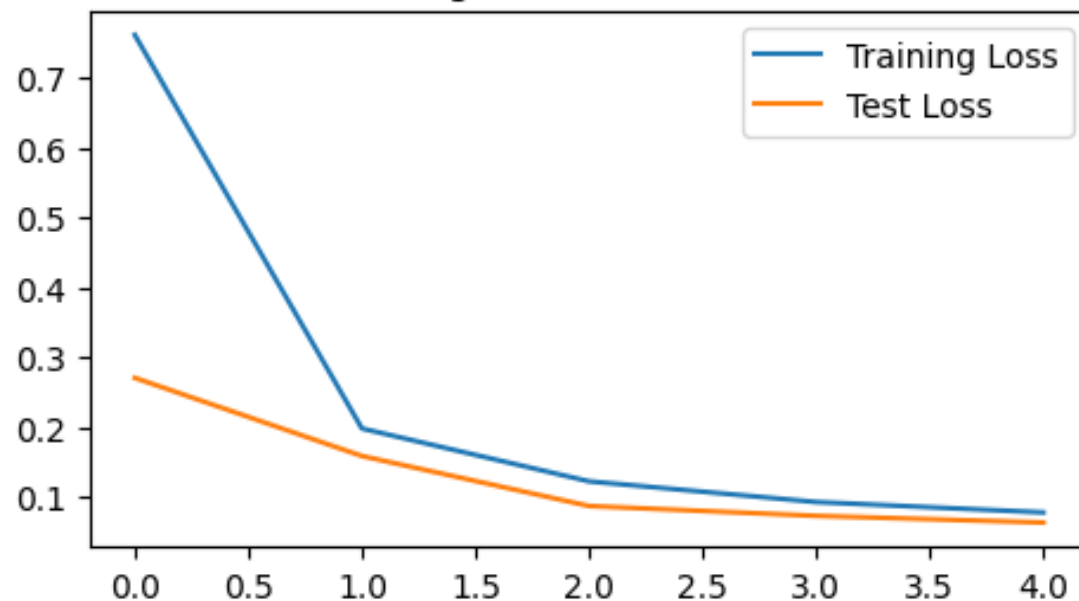
结果可视化

[Download](#)

Training and Validation Accuracy



Training and Validation Loss



- ▶ PyTorch 实现
- ▶ 修改参数、网络结构，对比结果（课后作业）
- ▶ CIFAR10 数据集识别（课后作业）

[手写数字识别 \(Pytorch\) – Datalore Report \(jetbrains.com\)](#)



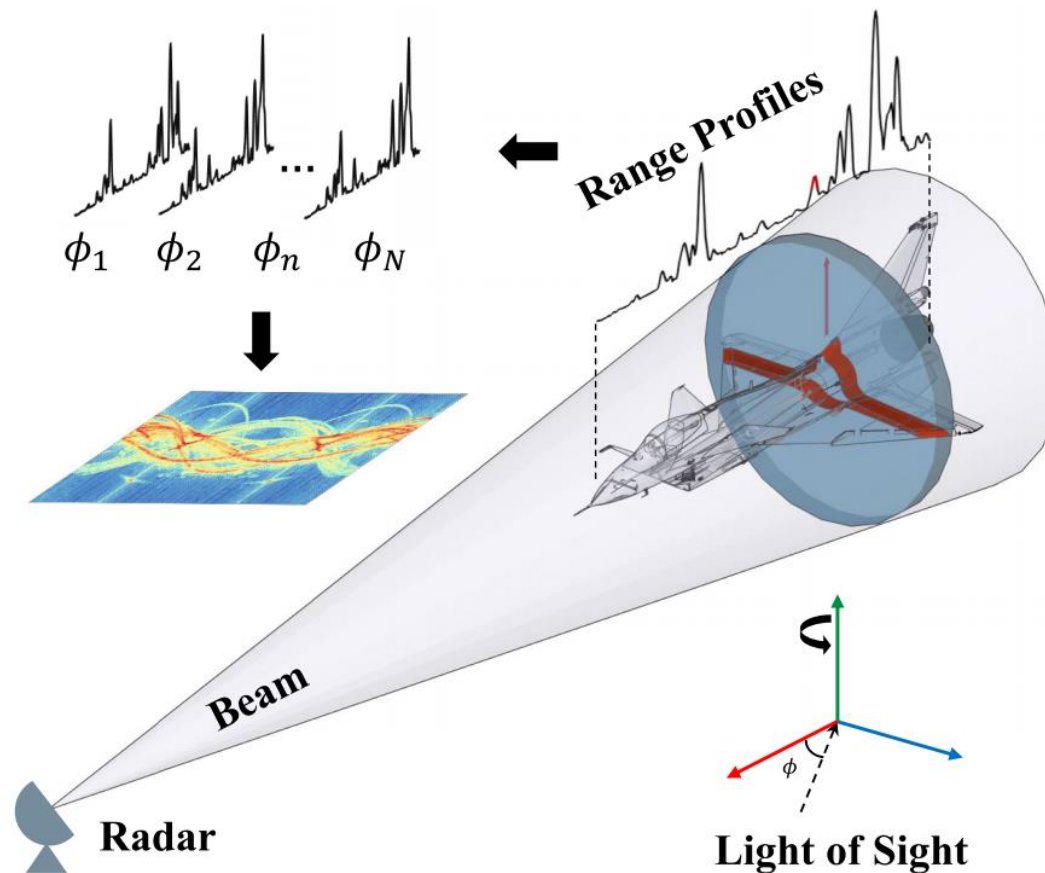
北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY

雷达目标识别实战

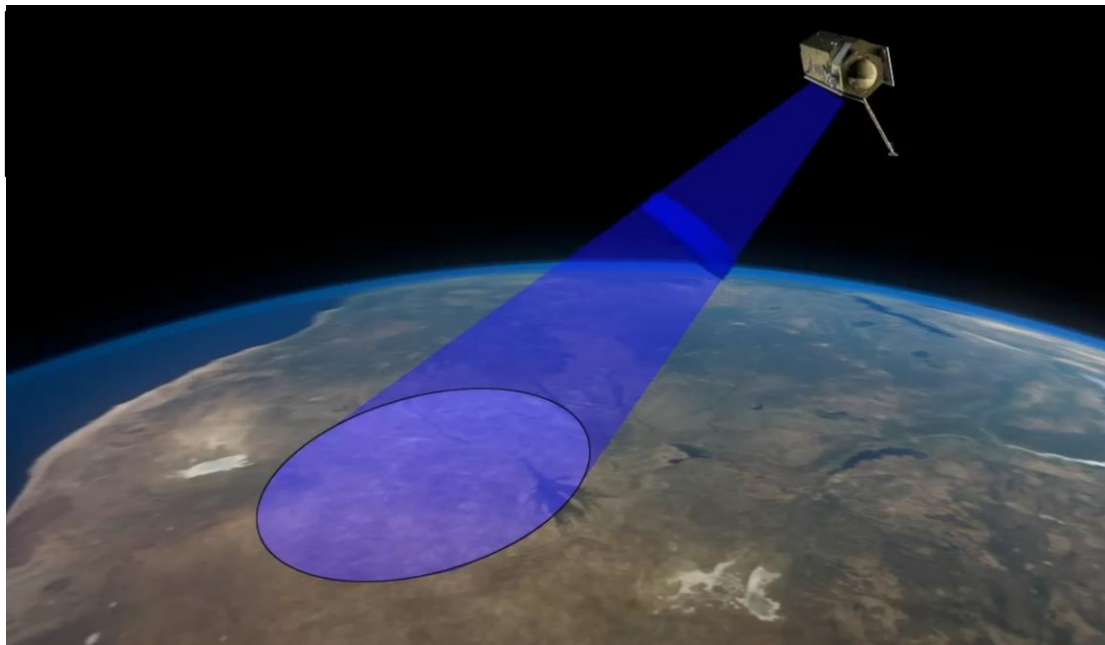
- ▶ 合成孔径雷达成像
- ▶ 基于深度学习的雷达目标识别（课后作业）

[MSTAR数据集在全卷积网络的识别分类\(pytorch版本\)-CSDN博客](#)

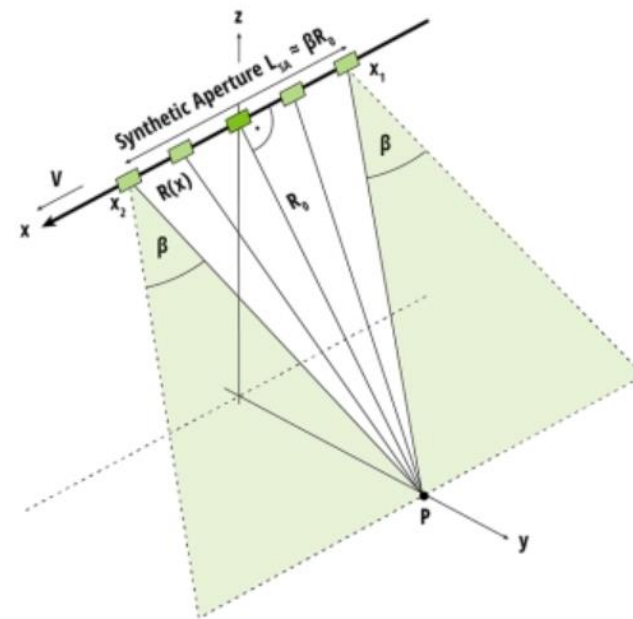
静止的雷达和静止的目标——一维成像（一维距离像）



合成孔径雷达通过平台运动，形成虚拟孔径来实现对观测目标的二维分辨率

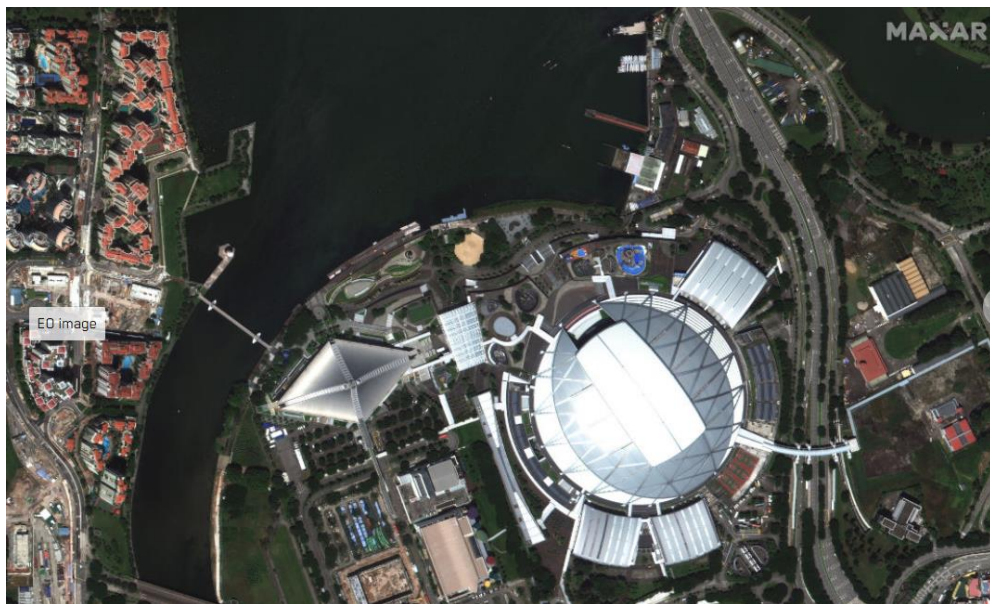


合成孔径雷达观测概念图

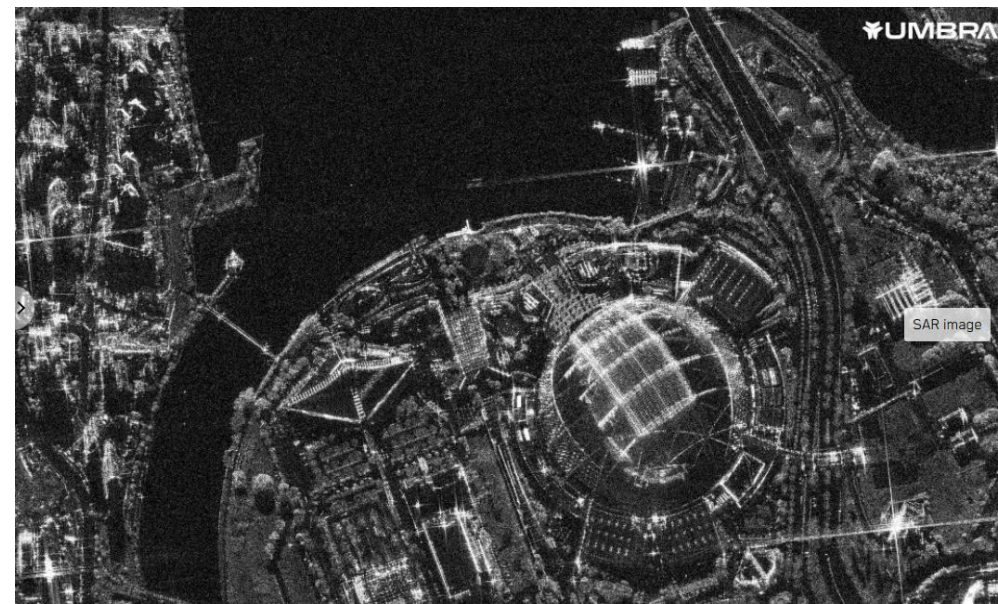


合成孔径雷达观测模型

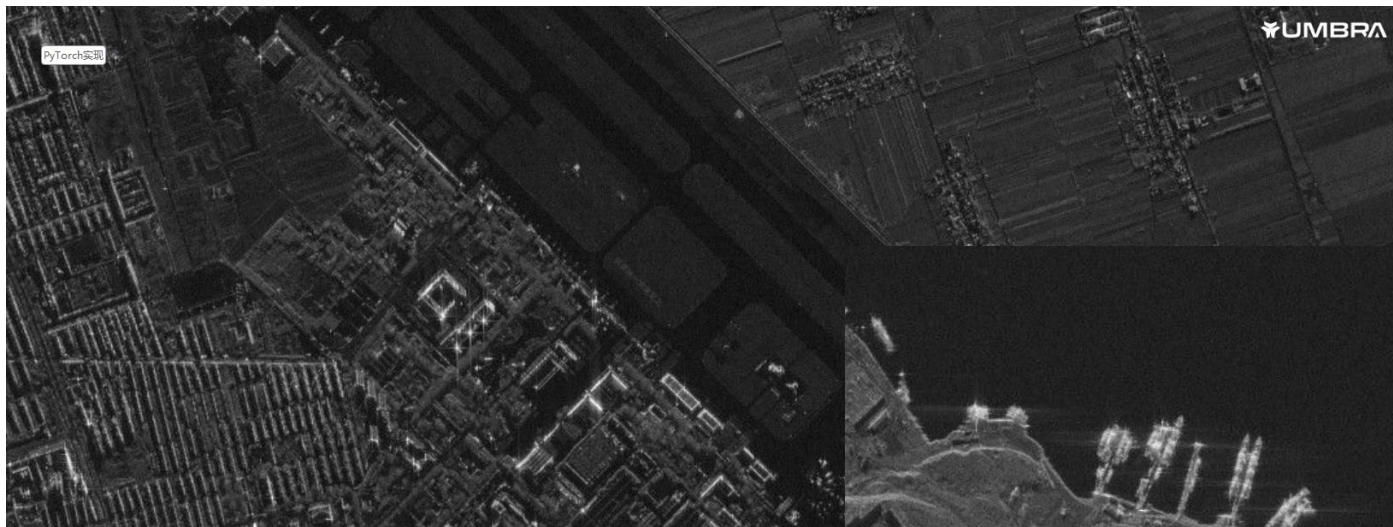
相比光学传感器，合成孔径雷达可以实现全天候、全天时观测



光学图片



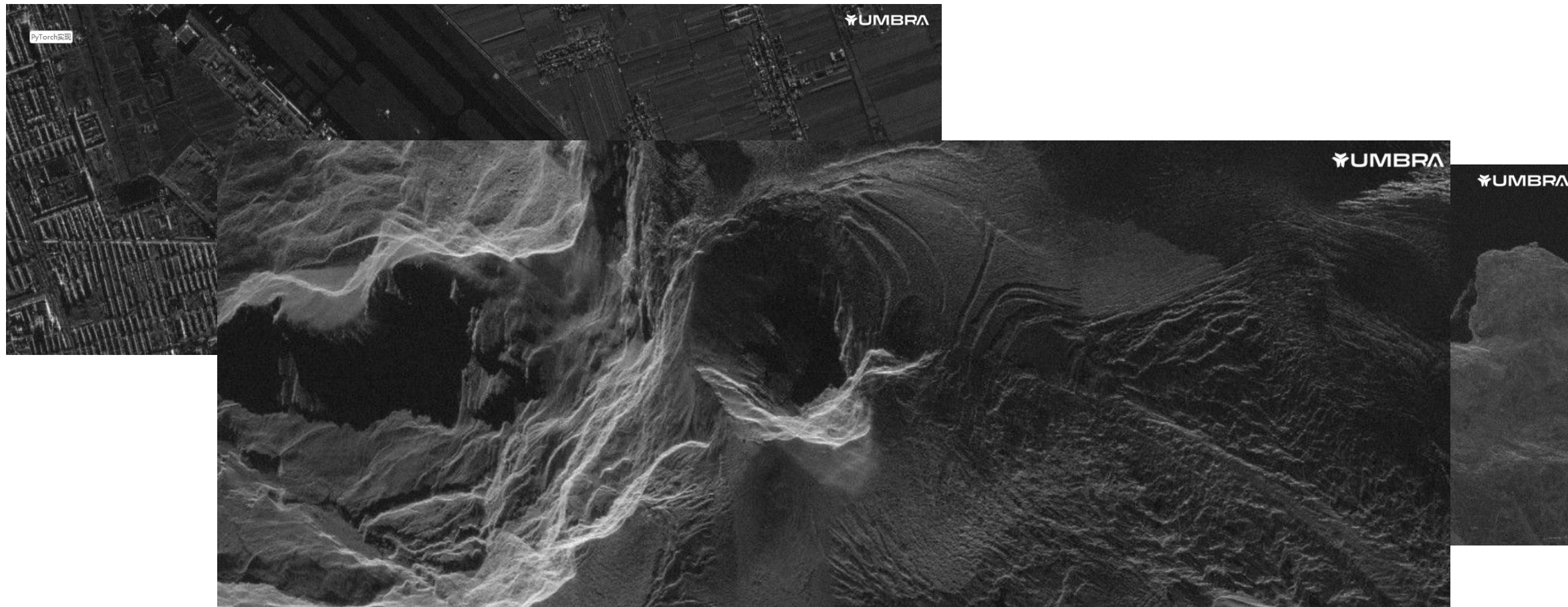
合成孔径雷达图片



地面

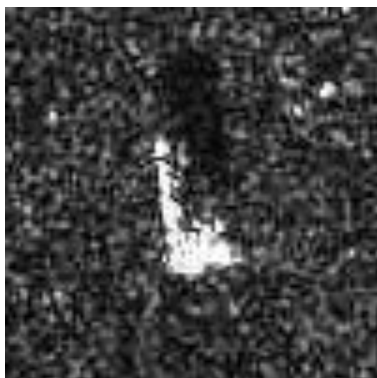


港口

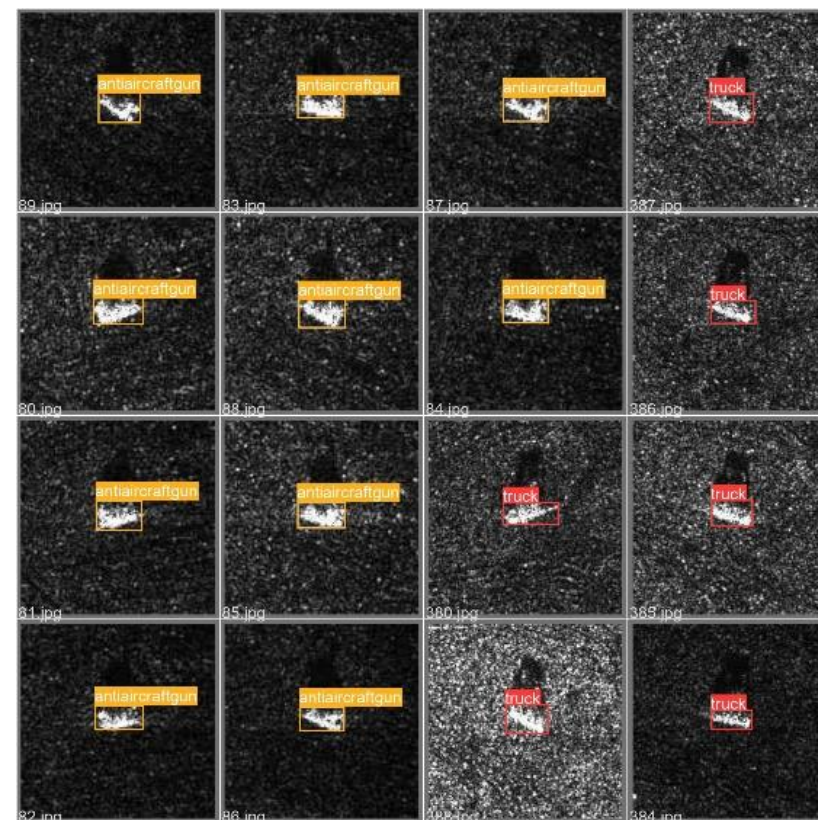


山地

MSTAR数据集：是一个专用于合成孔径雷达图像识别的公开数据集，它包含一些典型的车辆目标合成孔径雷达成像图。

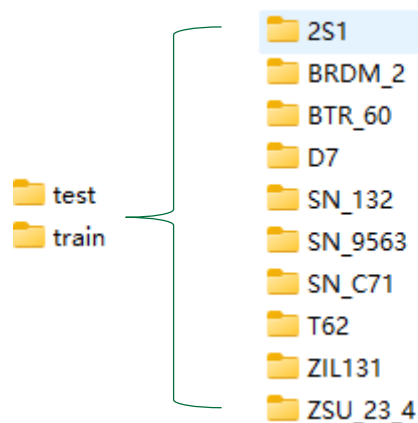


数据集中的图片样例



YOLOv5检测识别结果

MSTAR数据集：是一个专用于合成孔径雷达图像识别的公开数据集，它包含一些典型的车辆目标合成孔径雷达成像图。



数据集共有7类车辆目标，已经划分好了训练集和测试集

任务：

- 使用PyTorch框架，搭建一个深度神经网络，实现对MSTAR数据集中的车辆目标SAR图像做识别分类；
- 识别率>80%；
- 尝试使用不同的网络结构进行识别（选做）

德以明理 学以精工

谢谢